

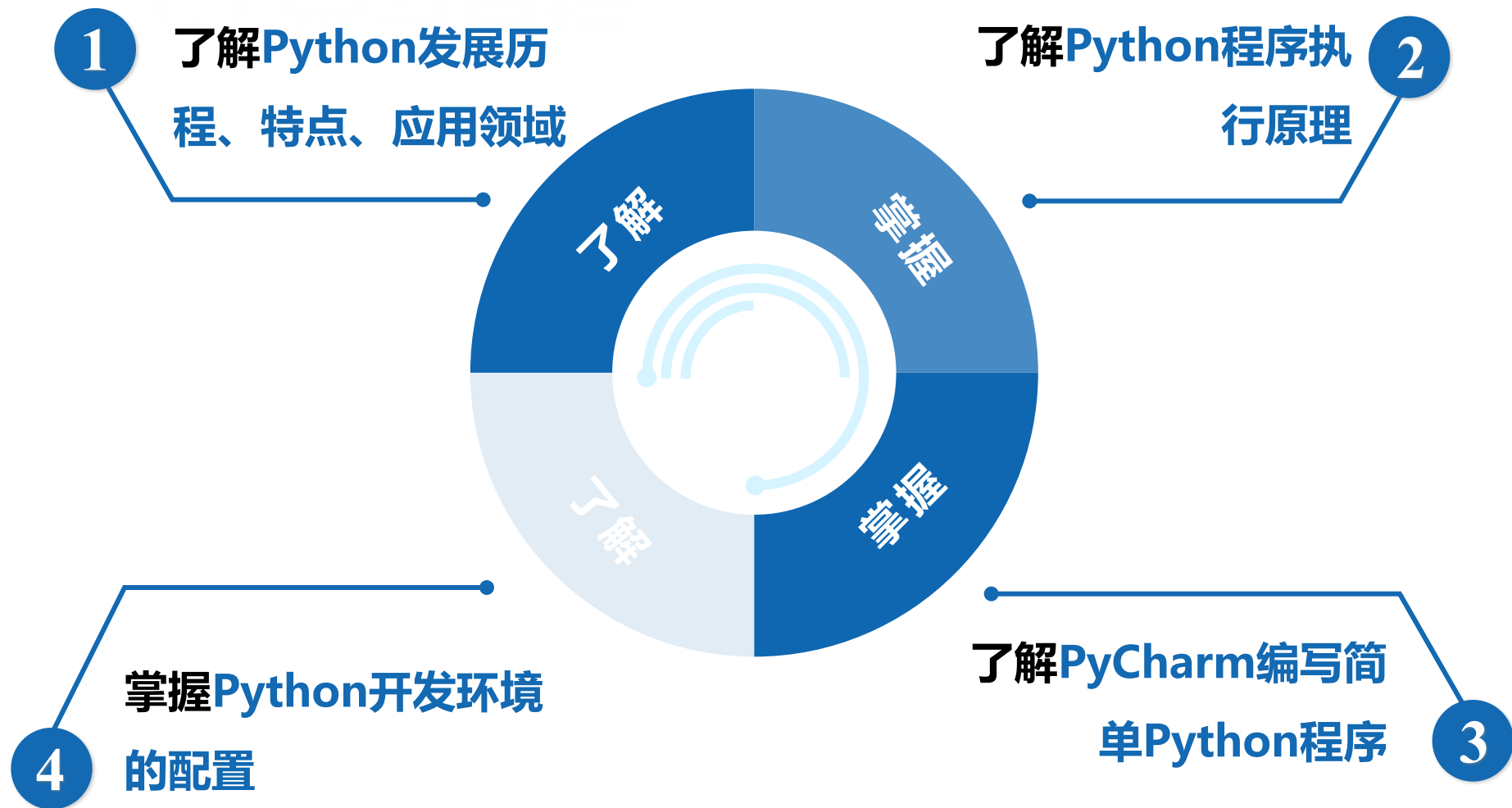
Python3

基础教程

第1章 Python概述

- Python发展历程
- Python开发环境的配置
- Python特点和应用领域
- Python程序执行原理

学习目标



目录页



01 认识Python

02 Python的安装

03 IPython-交互式解释器

04 集成开发环境-PyCharm

05 Python程序执行原理

过渡页



01 认识Python

02 Python的安装

03 IPython-交互式解释器

04 继承开发环境-PyCharm

05 Python程序执行原理

Python的由来



Python的创始人为荷兰的Guido

1989年，Guido为了打发圣诞节的无趣，决心开发一个新的脚本解释程序，做为ABC 语言的一种继承。之所以选中Python作为该编程语言的名字，是因为他是一个叫Monty Python的喜剧团体的爱好者。

Python的特点

- 易于学习
- 易于阅读
- 易于维护
- 强大标准库
- 互动模式
- 可移植
- 可扩展
- 数据库操作
- GUI编程
- 可嵌入

十大
特点

Python的应用领域

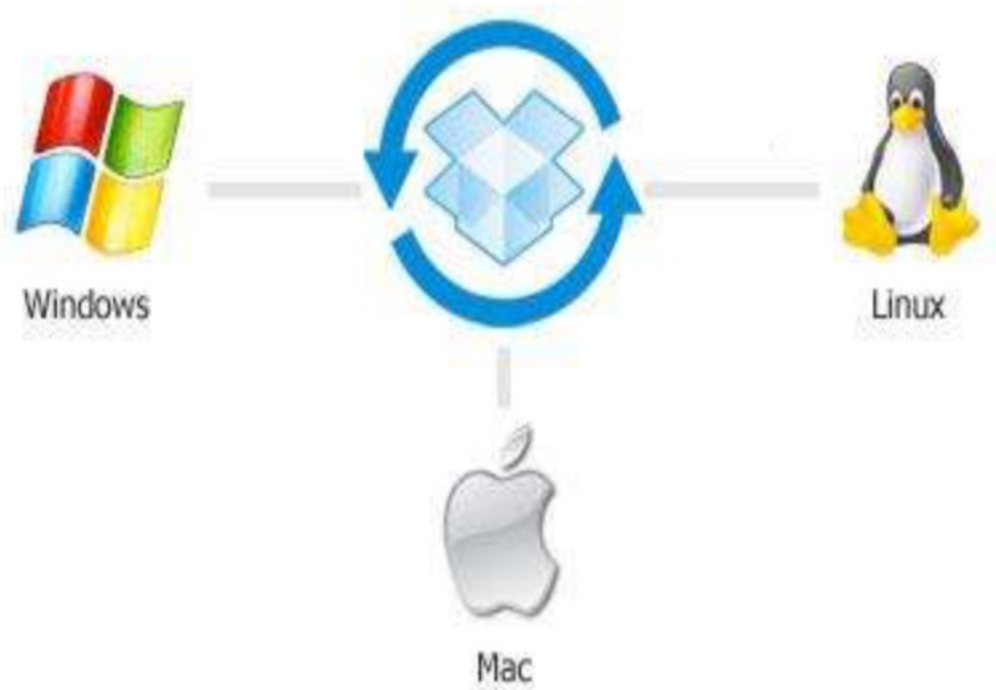
- Web开发领域

豆瓣 **douban**



Python的应用领域

- 操作系统管理、服务器运维的自动化脚本



Python的应用领域

- 科学计算



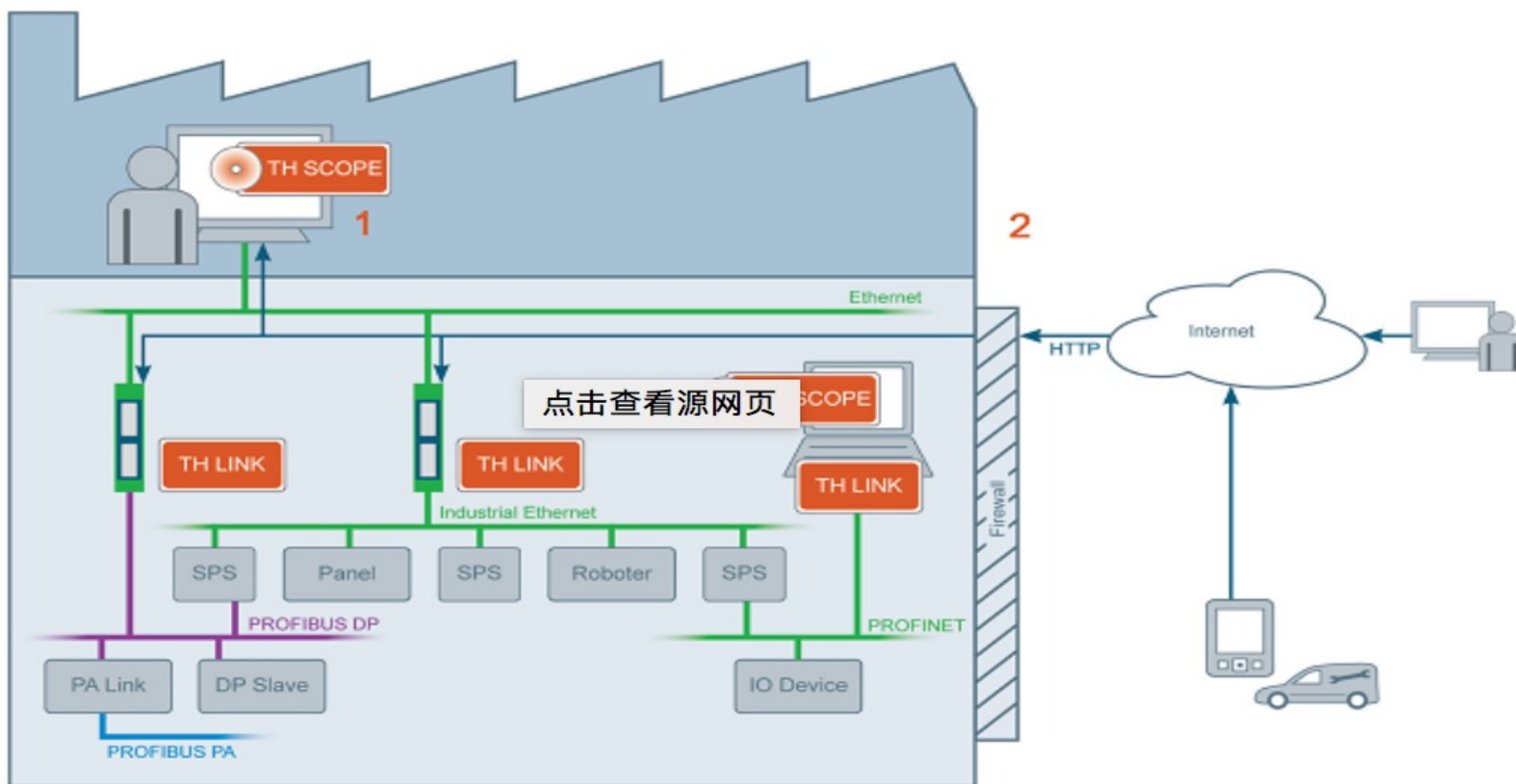
Python的应用领域

- 桌面应用



Python的应用领域

- 服务器软件（网络软件）



Python的应用领域

- 游戏



Python的发展

Jan 2016	Jan 2015	Change	Programming Language	Ratings	Change
1	2	▲	Java	21.465%	+5.94%
2	1	▼	C	16.036%	-0.67%
3	4	▲	C++	6.914%	+0.21%
4	5	▲	C#	4.707%	-0.34%
5	8	▲	Python	3.854%	+1.24%
6	6		PHP	2.706%	-1.08%
7	16	▲▲	Visual Basic .NET	2.582%	+1.51%
8	7	▼	JavaScript	2.565%	-0.71%
9	14	▲▲	Assembly language	2.095%	+0.92%
10	15	▲▲	Ruby	2.047%	+0.92%
11	9	▼	Perl	1.841%	-0.42%
12	20	▲▲	Delphi/Object Pascal	1.786%	+0.95%
13	17	▲▲	Visual Basic	1.684%	+0.61%
14	25	▲▲	Swift	1.363%	+0.62%
15	11	▼▼	MATLAB	1.228%	-0.16%
16	30	▲▲	Pascal	1.194%	+0.52%
17	82	▲▲	Groovy	1.182%	+1.07%
18	3	▼▼	Objective-C	1.074%	-5.88%
19	18	▼	R	1.054%	+0.01%
20	10	▼▼	PL/SQL	1.016%	-1.00%

Dec 2017	Dec 2016	Change	Programming Language	Ratings	Change
1	1		Java	13.268%	-4.59%
2	2		C	10.158%	+1.43%
3	3		C++	4.717%	-0.62%
4	4		Python	3.777%	-0.46%
5	6	▲	C#	2.822%	-0.35%
6	8	▲	JavaScript	2.474%	-0.39%
7	5	▼	Visual Basic .NET	2.471%	-0.83%
8	17	▲▲	R	1.906%	+0.08%
9	7	▼	PHP	1.590%	-1.33%
10	18	▲▲	MATLAB	1.569%	-0.25%
11	13	▲	Swift	1.566%	-0.57%
12	11	▼	Objective-C	1.497%	-0.83%
13	9	▼▼	Assembly language	1.471%	-1.07%
14	10	▼▼	Perl	1.437%	-0.90%
15	12	▼	Ruby	1.424%	-0.72%
16	15	▼	Delphi/Object Pascal	1.395%	-0.55%
17	16	▼	Go	1.387%	-0.55%
18	25	▲▲	Scratch	1.374%	+0.19%
19	20	▲	PL/SQL	1.368%	-0.13%
20	14	▼▼	Visual Basic	1.347%	-0.62%

Python的发展

Sep 2018	Sep 2017	Change	Programming Language	Ratings	Change
1	1		Java	17.436%	+4.75%
2	2		C	15.447%	+8.06%
3	5	^	Python	7.653%	+4.67%
4	3	v	C++	7.394%	+1.83%
5	8	^	Visual Basic .NET	5.308%	+3.33%
6	4	v	C#	3.295%	-1.48%
7	6	v	PHP	2.775%	+0.57%
8	7	v	JavaScript	2.131%	+0.11%
9	-	>>	SQL	2.062%	+2.06%
10	18	>>	Objective-C	1.509%	+0.00%
11	12	^	Delphi/Object Pascal	1.292%	-0.49%
12	10	v	Ruby	1.291%	-0.64%
13	16	^	MATLAB	1.276%	-0.35%
14	15	^	Assembly language	1.232%	-0.41%
15	13	v	Swift	1.223%	-0.54%
16	17	^	Go	1.081%	-0.49%
17	9	>>	Perl	1.073%	-0.88%
18	11	>>	R	1.016%	-0.80%
19	19		PL/SQL	0.850%	-0.63%
20	14	>>	Visual Basic	0.682%	-1.07%

Python的版本

历史版本

版本号	发布时间	拥有者	GPL 兼容
0.9.0~1.2	1991~1995	CWI	是
1.3~1.5.2	1995~1999	CNRI	是
1.6	2000	CNRI	否
2.0	2000	BeOpen.com	否
1.6.1	2001	CNRI	否
2.1	2001	PSF	否
2.0.1	2001-06-22	PSF	是
2.2~2.7.11	2001~2015	PSF	是
2.7.12	2016-06	PSF	是
2.7.13	2016-12	PSF	是
3.x	2008~至今	PSF	是

Python2.x和Python3.x

1. print函数替代了print语句

```
>>> print(3,4)
```

```
(3, 4)
```

Python2.x

```
>>> print(3,4)
```

```
3 4
```

Python3.x

Python2.x和Python3.x

2. python3.x默认使用UTF-8编码

```
>>> str = "我爱北京天安门"  
>>> str  
'\xe6\x88\x91\xe7\x88\xb1\xe5\x8c\x97\xe4\xba\xac\xe5\xa4\xa9\xe5\xae\x89\xe9\x97\xa8'
```

Python2.x

```
>>> str = "我爱北京天安门"  
>>> str  
'我爱北京天安门'
```

Python3.x

-*- coding: utf-8 -*-

Python2.x和Python3.x

3. 除法运算符/

```
>>> 1 / 2
```

```
0
```

```
>>> 1.0 / 2.0
```

```
0.5
```

Python2.x

```
>>> 1/2
```

```
0.5
```

Python3.x

Python2.x和Python3.x

4. 异常

– 捕获异常的语法是
`except exc,var .`

Python2.x

– 被抛出捕获异常的语法
变更为
`except exc as var`

Python3.x

Python2.x和Python3.x

5. 八进制字面量表示0b\0o\64\0x

```
>>> 0o1000
512
>>> 01000
512
```

Python2.x

```
>>> 01000
File "<stdin>", line 1
    01000
      ^
SyntaxError: invalid token
>>> 0o1000
512
```

Python3.x

Python2.x和Python3.x

6. 不等运算符

```
>>> 1!=2
True
>>> 1<>2
True
```

Python2.x

```
>>> 1!=2
True
>>> 1<>2
File "<stdin>", line 1
    1<>2
      ^
SyntaxError: invalid syntax
```

Python3.x

Python2.x和Python3.x

7. 数据类型

– 有long类型

Python2.x

– 没有long，只有int

Python3.x

过渡页



01 认识Python

02 Python安装

03 IPython-交互式解释器

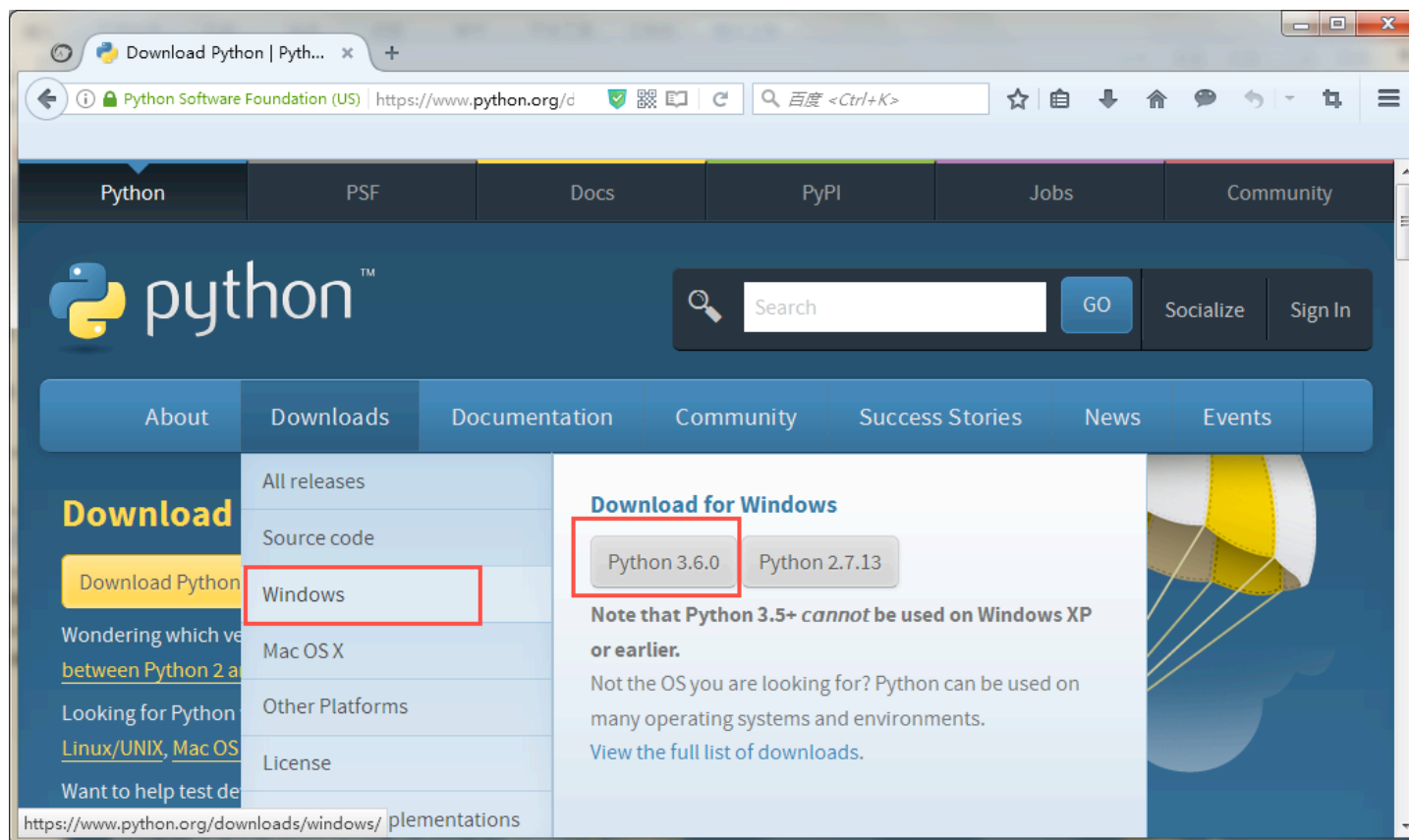
04 集成开发环境-PyCharm

05 Python程序执行原理

Window安装Python

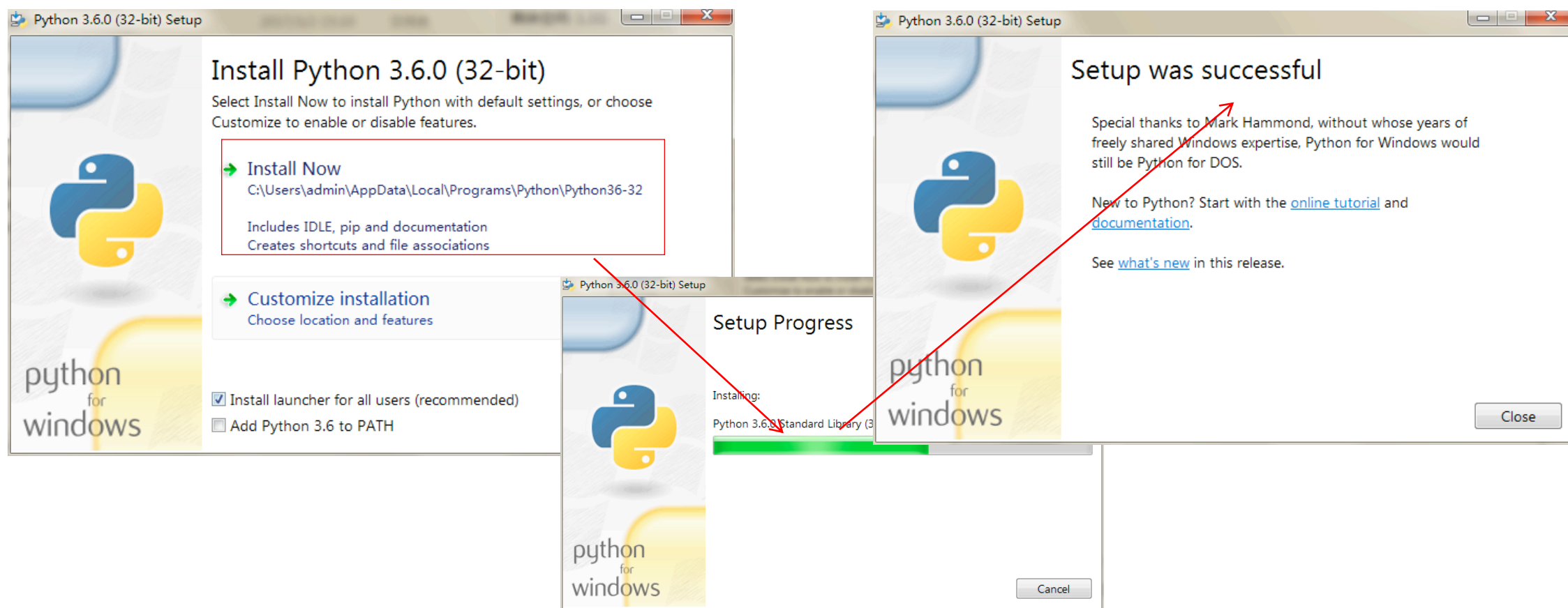
(1) 访问<http://www.python.org/download/>。

选择安装包。



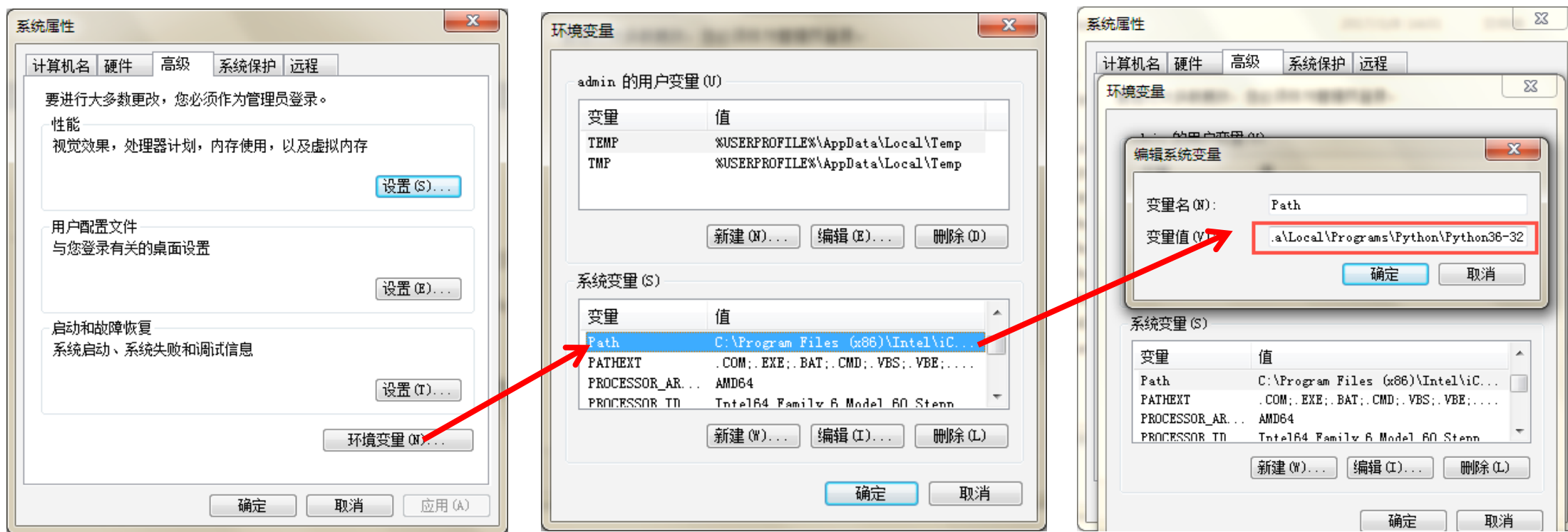
Window安装Python

(2) 下载并完成Python的安装。



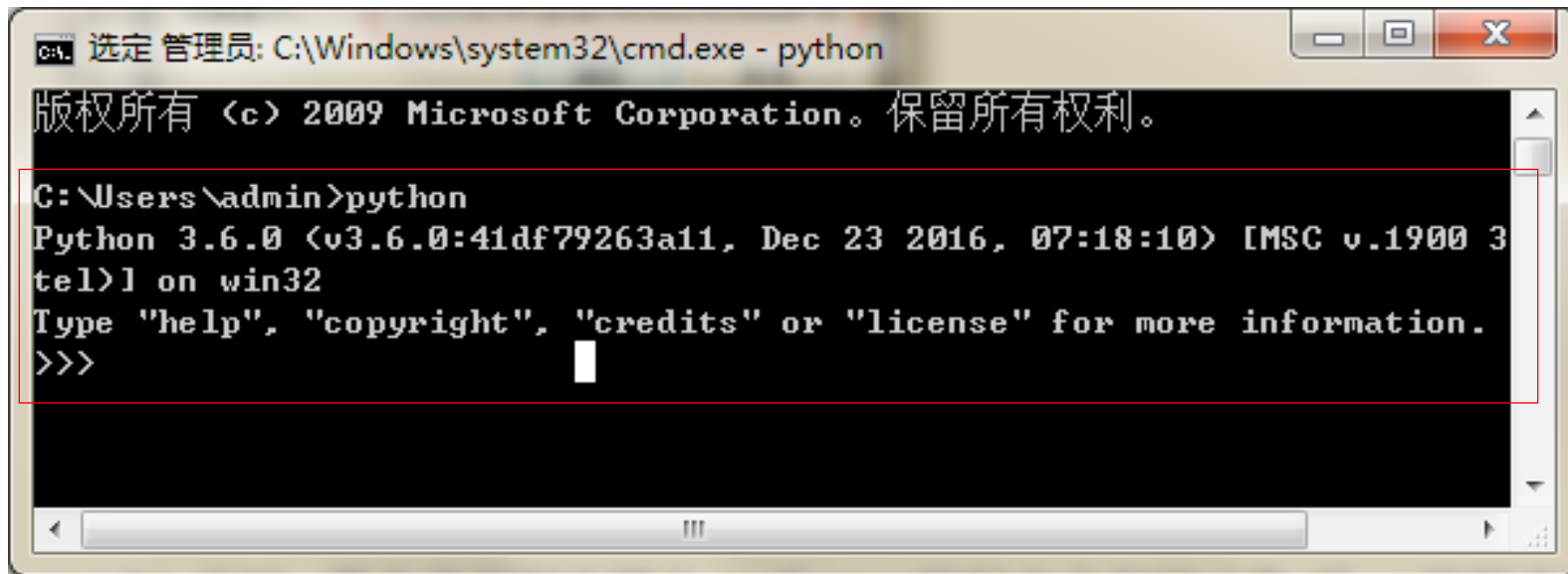
Window安装Python

(3) 配置环境变量。



Window安装Python

(4) 环境变量配置成功



The screenshot shows a Windows command prompt window titled "选定 管理员: C:\Windows\system32\cmd.exe - python". The window contains the following text:

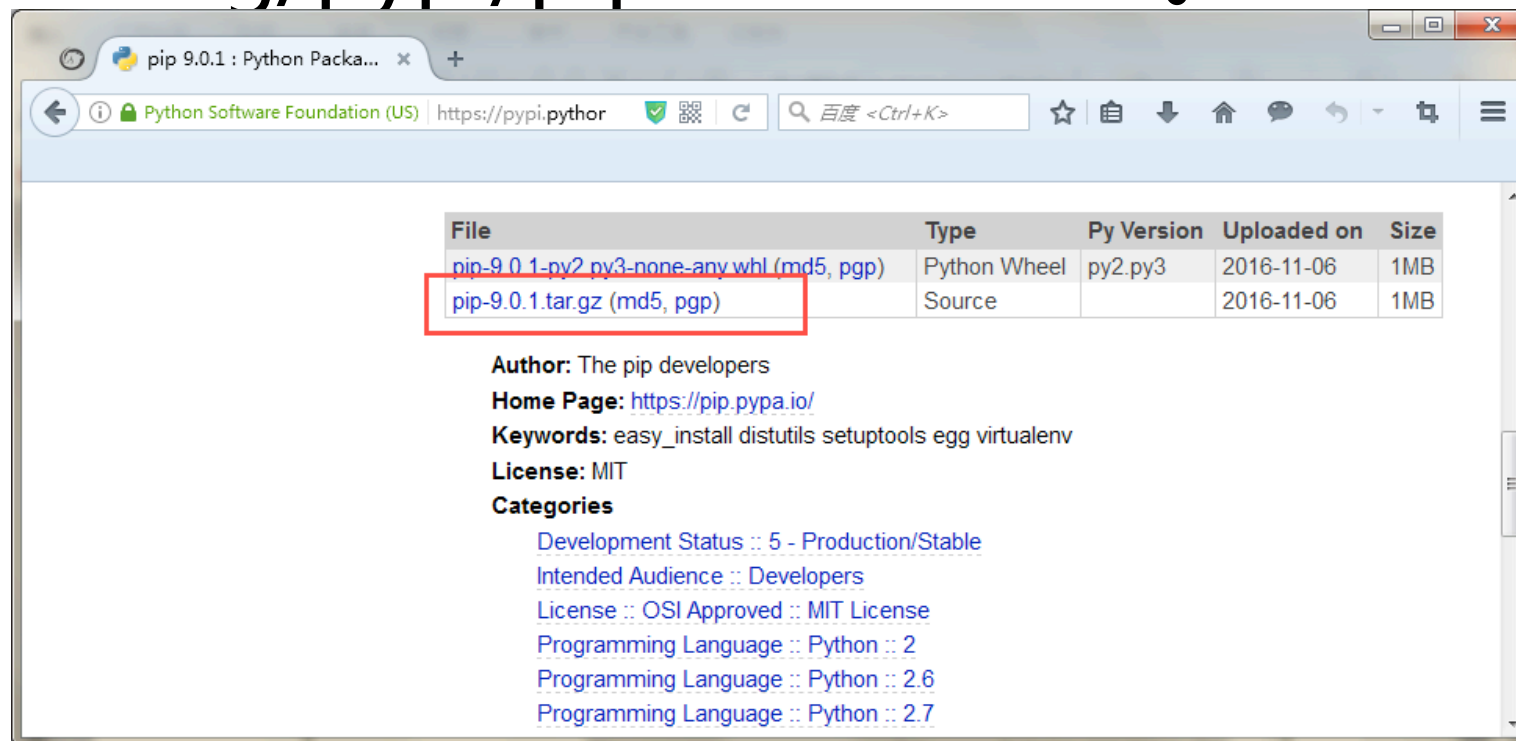
```
版权所有 (c) 2009 Microsoft Corporation。保留所有权利。  
C:\Users\admin>python  
Python 3.6.0 (v3.6.0:41df79263a11, Dec 23 2016, 07:18:10) [MSC v.1900 3  
tel) on win32  
Type "help", "copyright", "credits" or "license" for more information.  
>>>
```

A red rectangular box highlights the output of the 'python' command, from the prompt 'C:\Users\admin>' to the '>>>' prompt, indicating that the Python environment is correctly configured and accessible from the system path.

Window安装Python

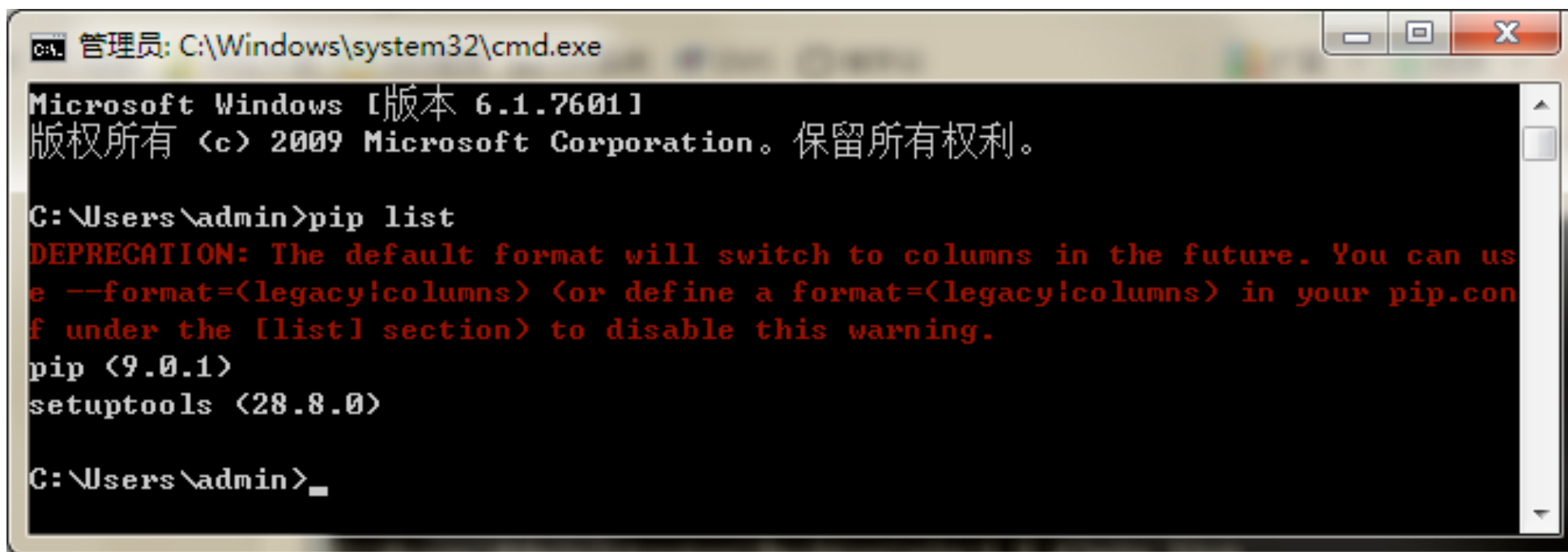
(5) 下载pip。下载网站：

<https://pypi.python.org/pypi/pip#downloads>。



Window安装Python

(6) 解压文件后，在Path最后面添加Scripts文件路径。再次打开控制台，输入pip list，出现下列信息表示pip安装成功。



```
C:\Windows\system32\cmd.exe
Microsoft Windows [版本 6.1.7601]
版权所有 (c) 2009 Microsoft Corporation。保留所有权利。

C:\Users\admin>pip list
DEPRECATION: The default format will switch to columns in the future. You can use --format=<legacy!columns> (or define a format=<legacy!columns> in your pip.conf under the [list] section) to disable this warning.
pip <9.0.1>
setuptools <28.8.0>

C:\Users\admin>
```

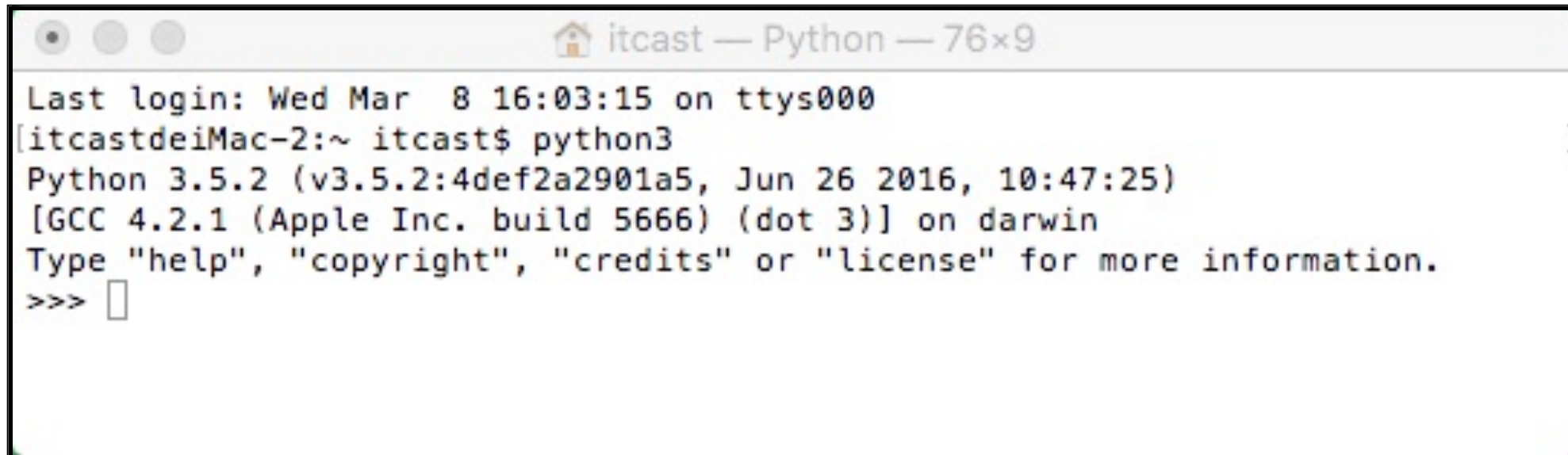
Linux安装Python

- (1) 打开WEB浏览器访问<http://www.python.org/download/>
- (2) 选择适用于Unix/Linux的源码压缩包。
- (3) 下载及解压压缩包。
- (4) 如果你需要自定义一些选项修改Modules/Setup。
- (5) 执行 `./configure` 脚本。
- (6) `make`
- (7) `make install`

Mac OS安装Python

如果我们希望安装Python，只需要访问网站

<http://www.python.org/download/>，下载最新版本的dmg文件，双击按照提示完成安装即可。

A screenshot of a macOS terminal window titled "itcast — Python — 76x9". The terminal shows the output of running "python3", displaying the Python version (3.5.2), build information, and the prompt "Type 'help', 'copyright', 'credits' or 'license' for more information." followed by a blank prompt line ">>>".

```
itcast — Python — 76x9
Last login: Wed Mar  8 16:03:15 on ttys000
[itcastdeiMac-2:~ itcast$ python3
Python 3.5.2 (v3.5.2:4def2a2901a5, Jun 26 2016, 10:47:25)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> 
```

过渡页



01 认识Python

02 Python的安装

03 Ipython-交互式解释器

04 集成开发环境-PyCharm

05 Python程序执行原理

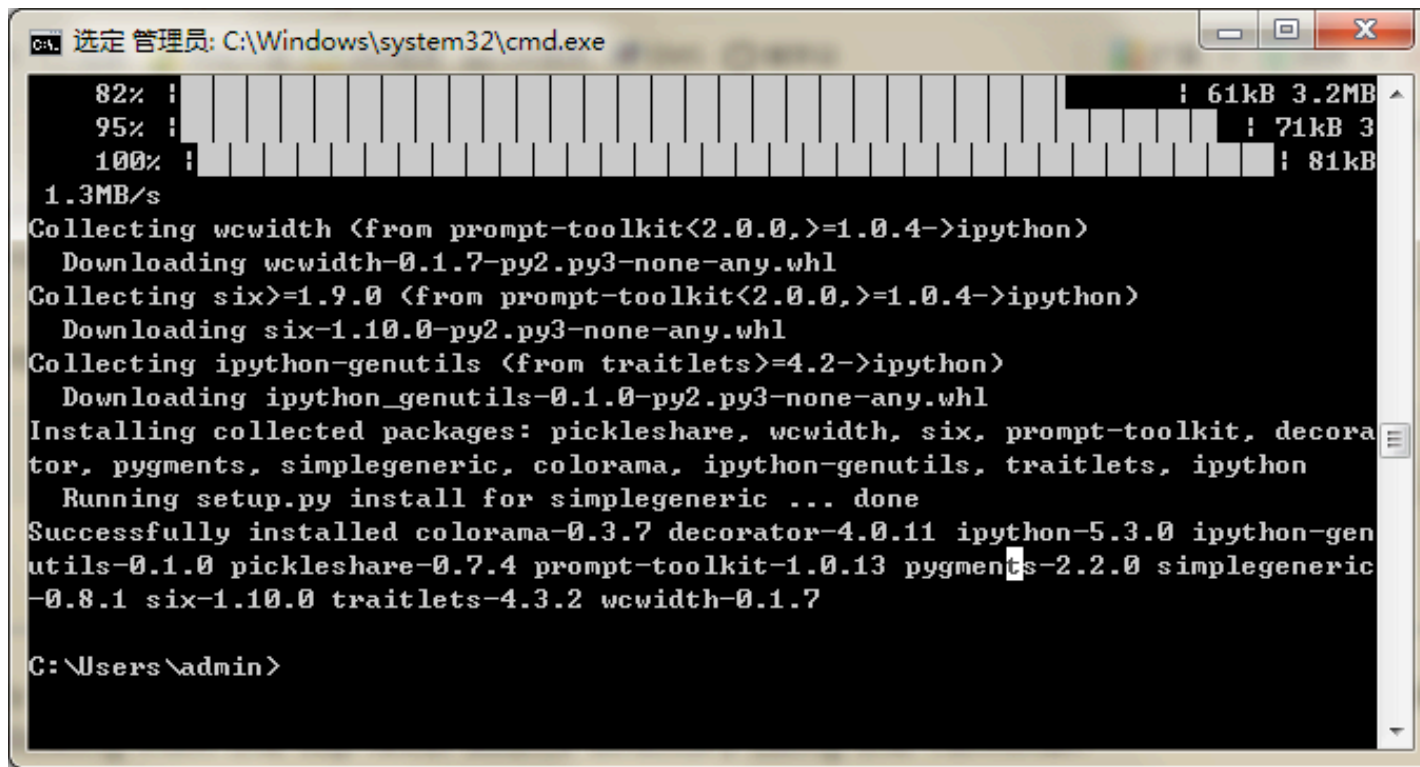
什么是IPython

IPython是一种交互式解释器。

- IPython的性能优于标准Python的shell。
- IPython支持变量自动补全，自动缩进，支持 bash shell 命令，内置了许多很有用的功能和函数。
- 使用%run命令运行脚本。
- 使用%timeit命令快速测量代码运行时间。

Ipython的安装

控制台输入命令：**pip install ipython**自动安装



```
选定 管理员: C:\Windows\system32\cmd.exe

 82% | 61kB 3.2MB
 95% | 71kB 3
100% | 81kB
1.3MB/s
Collecting wcwidth <from prompt-toolkit<2.0.0,>=1.0.4->ipython>
  Downloading wcwidth-0.1.7-py2.py3-none-any.whl
Collecting six>=1.9.0 <from prompt-toolkit<2.0.0,>=1.0.4->ipython>
  Downloading six-1.10.0-py2.py3-none-any.whl
Collecting ipython-genutils <from traitlets>=4.2->ipython>
  Downloading ipython_genutils-0.1.0-py2.py3-none-any.whl
Installing collected packages: pickleshare, wcwidth, six, prompt-toolkit, decora
tor, pygments, simplegeneric, colorama, ipython-genutils, traitlets, ipython
  Running setup.py install for simplegeneric ... done
Successfully installed colorama-0.3.7 decorator-4.0.11 ipython-5.3.0 ipython-gen
utils-0.1.0 pickleshare-0.7.4 prompt-toolkit-1.0.13 pygments-2.2.0 simplegeneric
-0.8.1 six-1.10.0 traitlets-4.3.2 wcwidth-0.1.7

C:\Users\admin>
```



Mac OS安装Python

```
brew install python  
pip3 install ipython
```

```
[fyj@bogon bin % ipython  
Python 3.12.5 (v3.12.5:ff3bc82f7c9, Aug 7 2024, 05:32:06) [Clang 13.0.0 (clang-  
1300.0.29.30)]  
Type 'copyright', 'credits' or 'license' for more information  
IPython 8.27.0 -- An enhanced Interactive Python. Type '?' for help.]
```

IPython的使用

1. 启动IPython

```
IPython: C:\Users\admin
Microsoft Windows [版本 6.1.7601]
版权所有 (c) 2009 Microsoft Corporation。保留所有权利。

C:\Users\admin>ipython
Python 3.6.0 (v3.6.0:41df79263a11, Dec 23 2016, 07:18:10) [MSC v.1900 32 bit <
tel>]
Type "copyright", "credits" or "license" for more information.

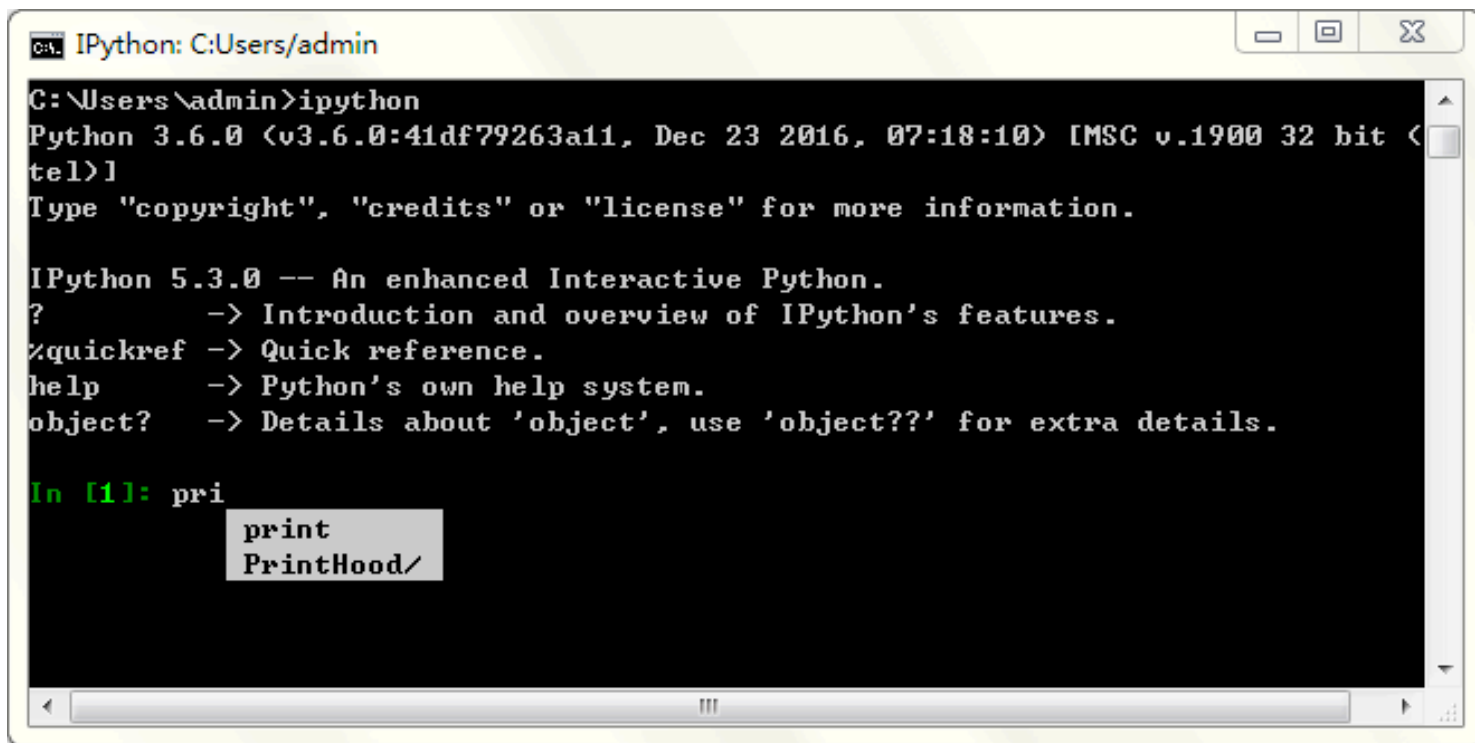
IPython 5.3.0 -- An enhanced Interactive Python.
?          -> Introduction and overview of IPython's features.
%quickref  -> Quick reference.
help       -> Python's own help system.
object?    -> Details about 'object', use 'object??' for extra details.

In [1]:
```



IPython的使用

2. Ipython自动补全



```
IPython: C:\Users\admin
C:\Users\admin>ipython
Python 3.6.0 (v3.6.0:41df79263a11, Dec 23 2016, 07:18:10) [MSC v.1900 32 bit <
tel>]
Type "copyright", "credits" or "license" for more information.

IPython 5.3.0 -- An enhanced Interactive Python.
?          -> Introduction and overview of IPython's features.
%quickref  -> Quick reference.
help       -> Python's own help system.
object?    -> Details about 'object', use 'object??' for extra details.

In [1]: pri
        print
        PrintHood/
```



过渡页



01 认识Python

02 Python的安装

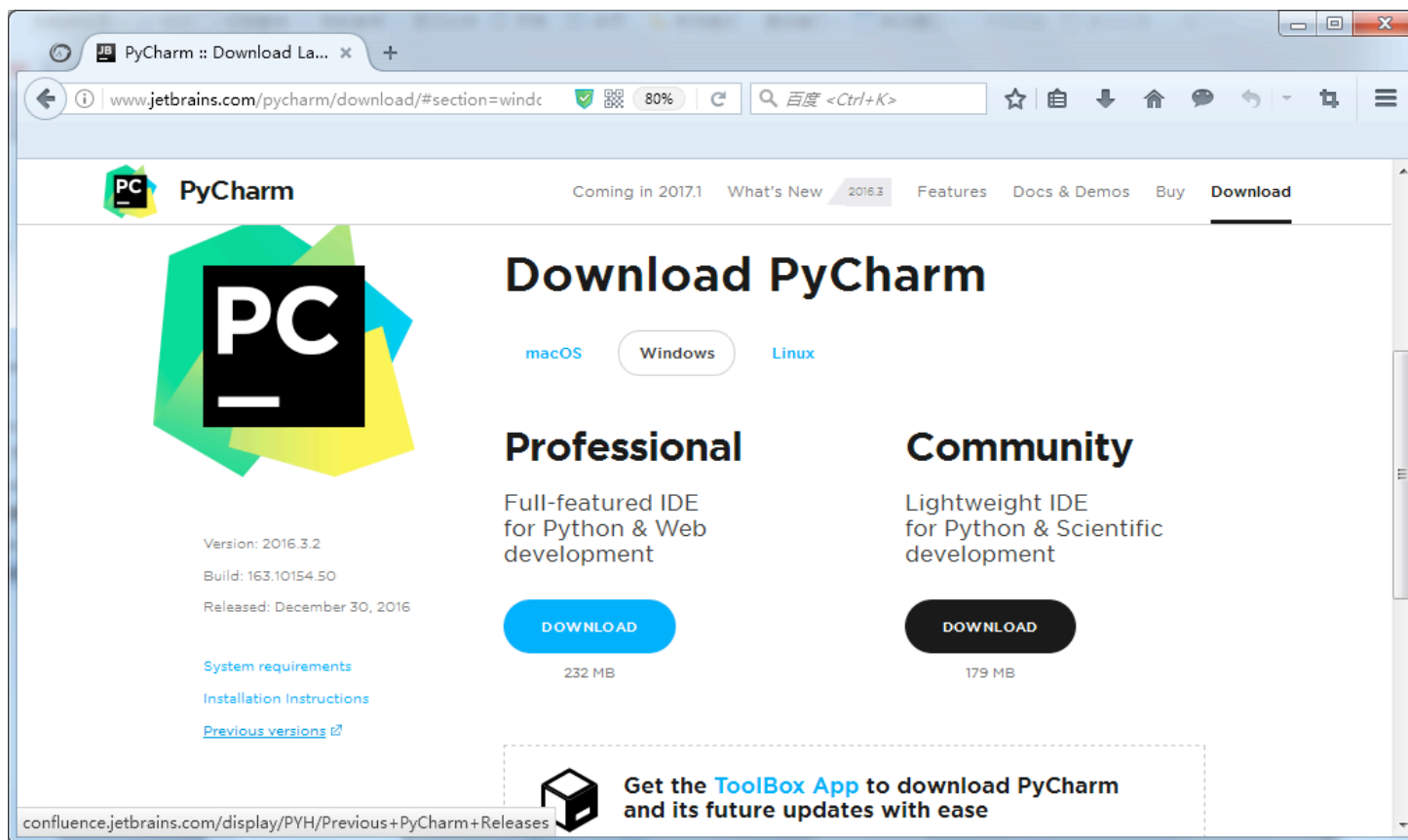
03 IPython-交互式解释器

04 集成开发环境-Pycharm

05 Python程序执行原理

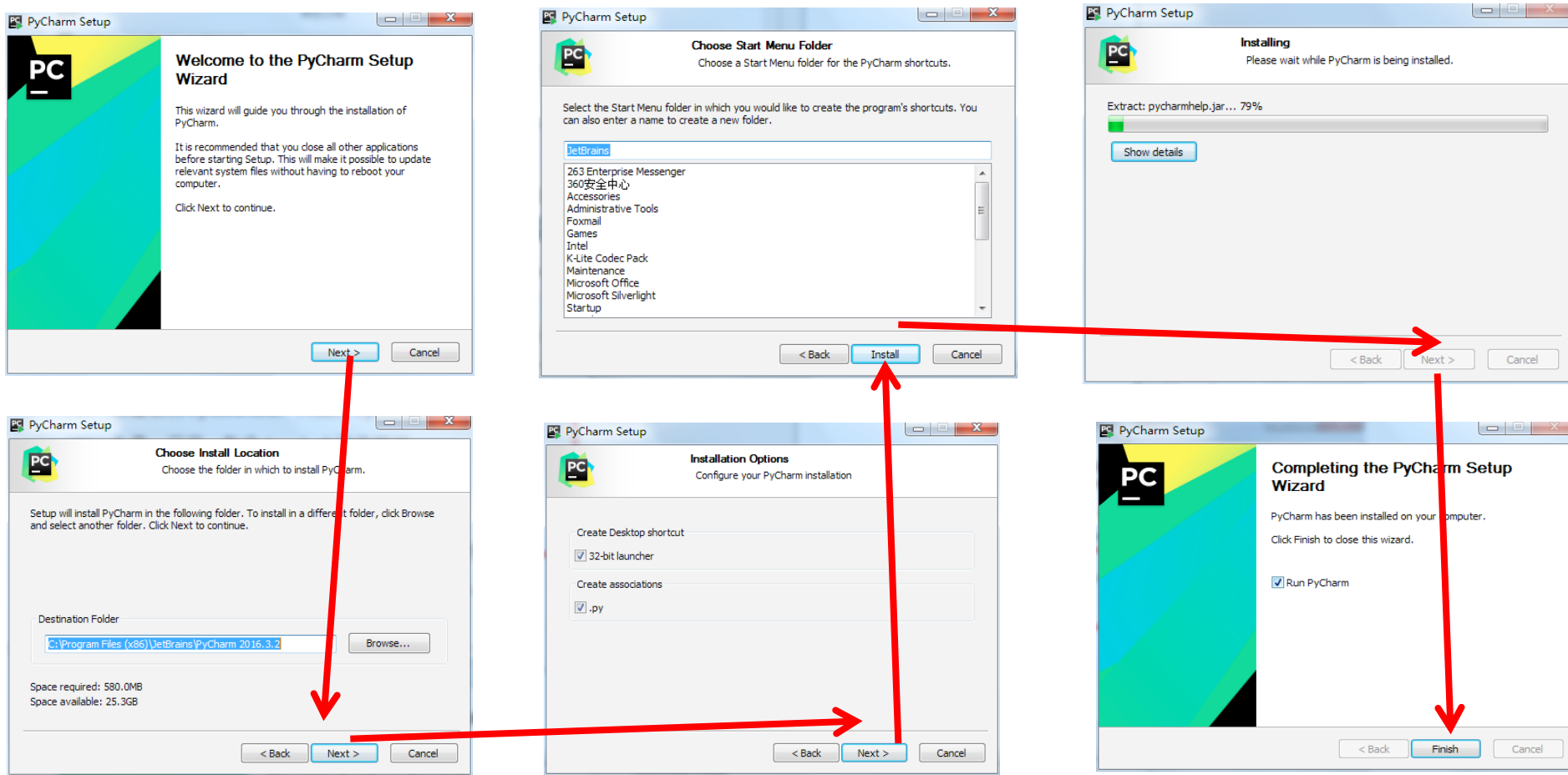
Pycharm的下载安装

(1) 访问PyCharm官方进入Pycharm的下载页面。



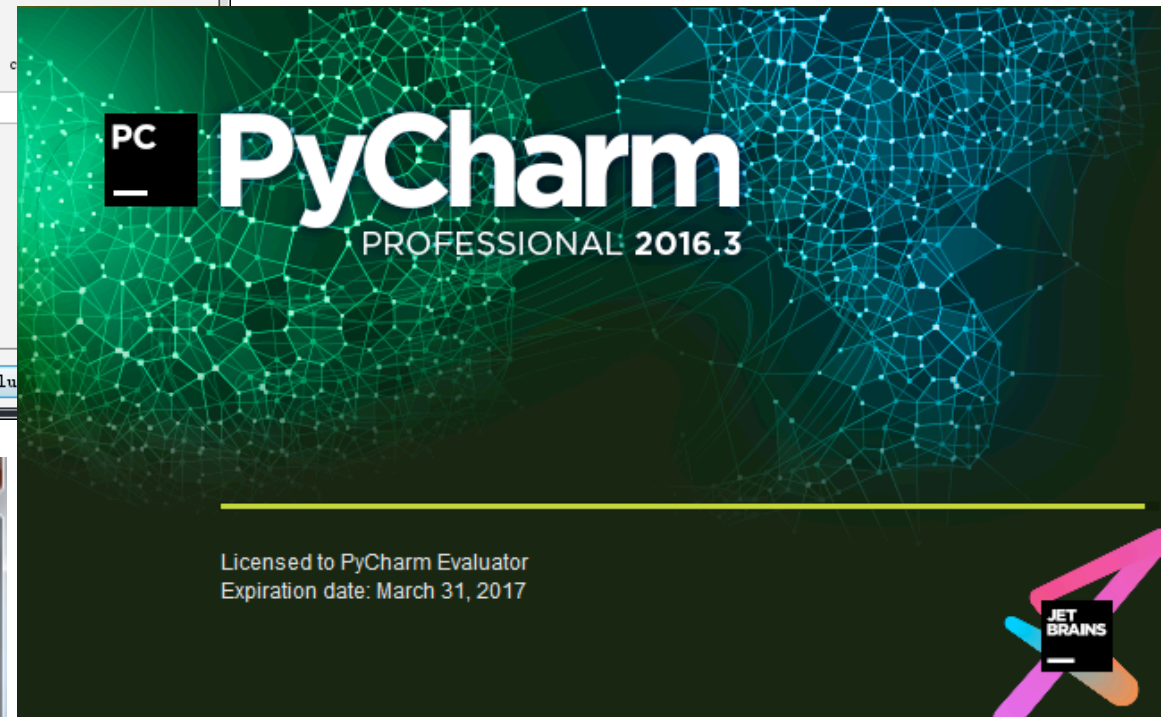
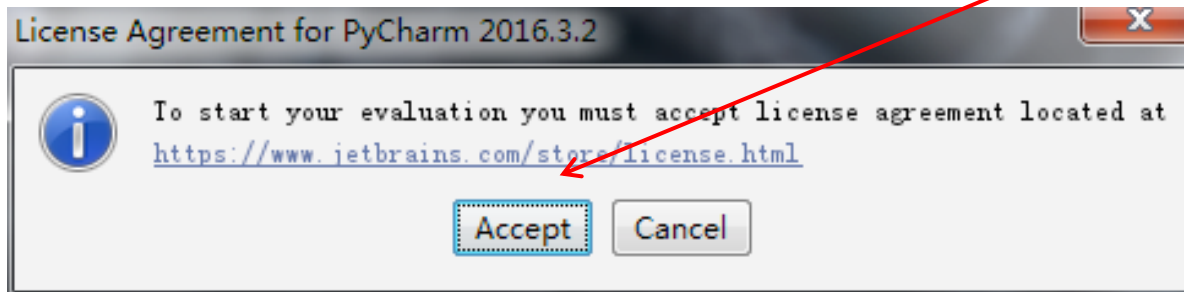
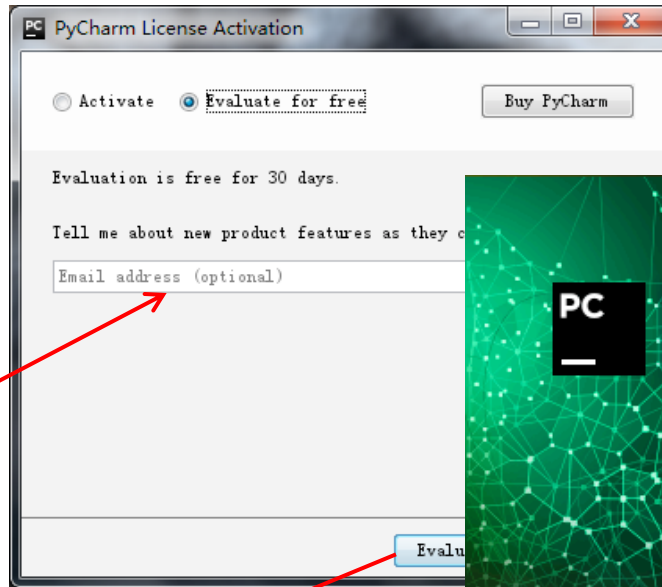
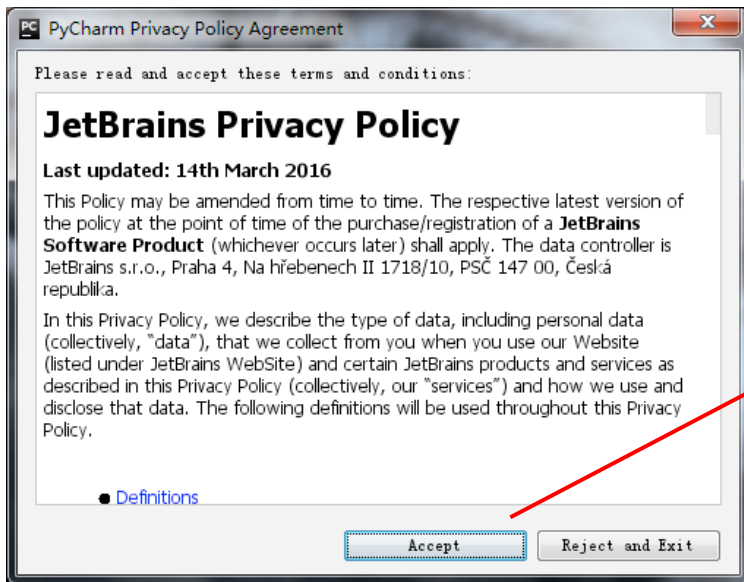
Pycharm的下载安装

(2) 双击exe下载文件，开始安装。



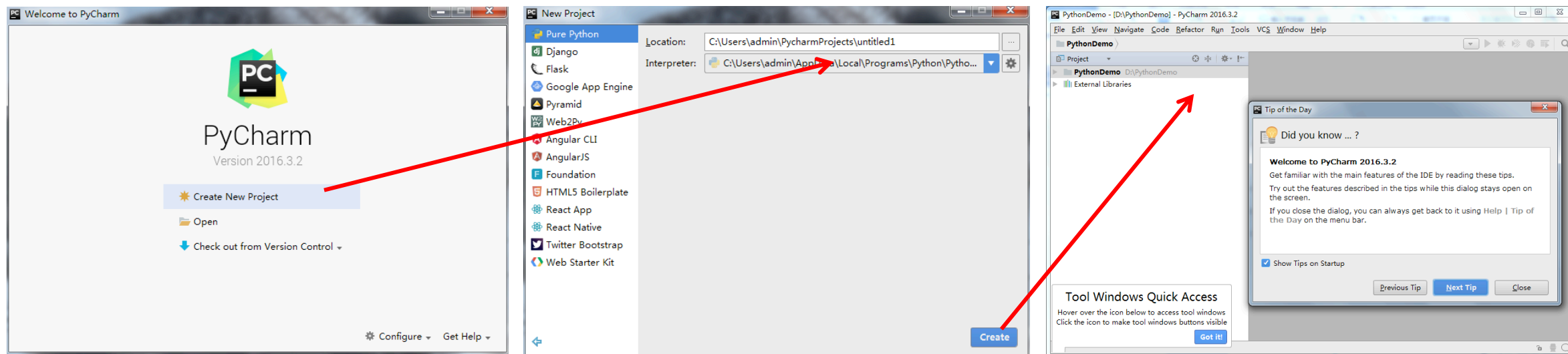
Pycharm的使用

完成PyCharm的安装后，就可以打开并使用PyCharm了。



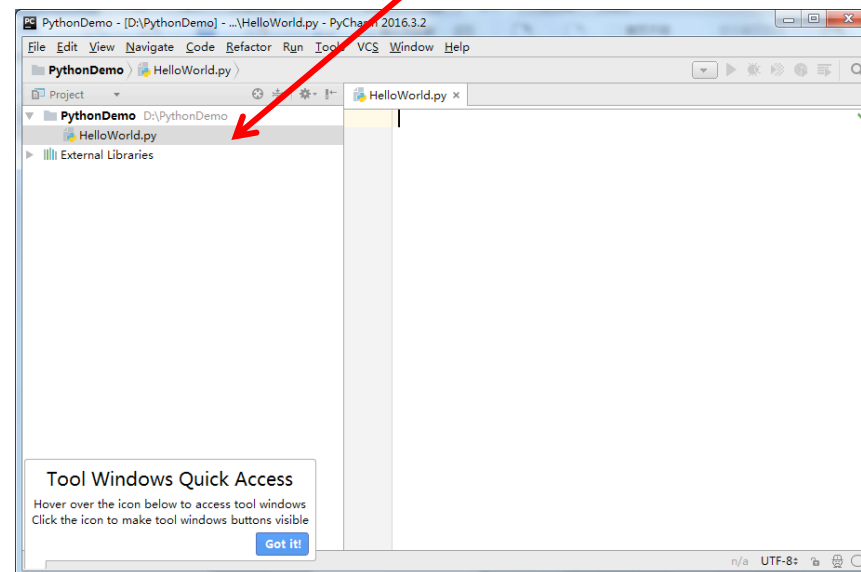
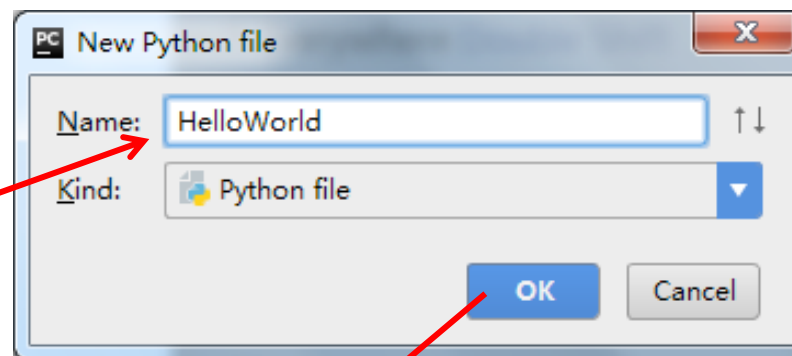
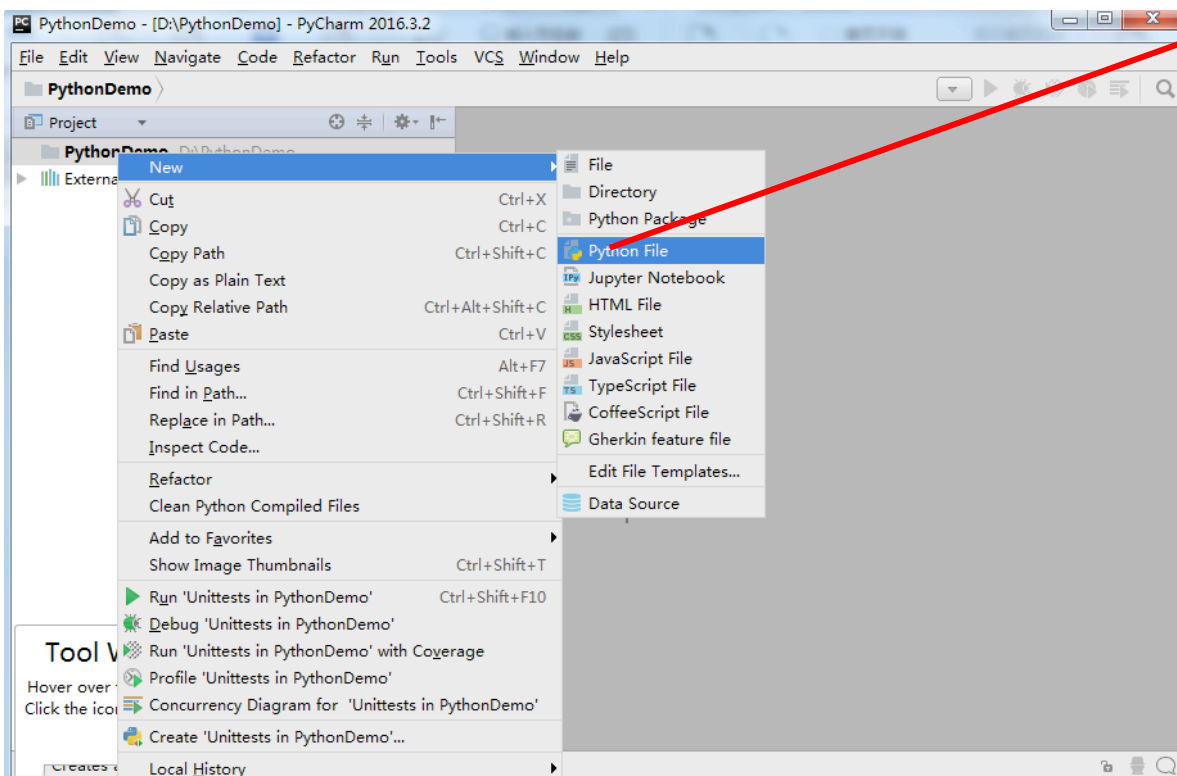
Pycharm的使用

1. 创建项目



Pycharm的使用

2. 创建Python文件



过渡页



01 认识Python

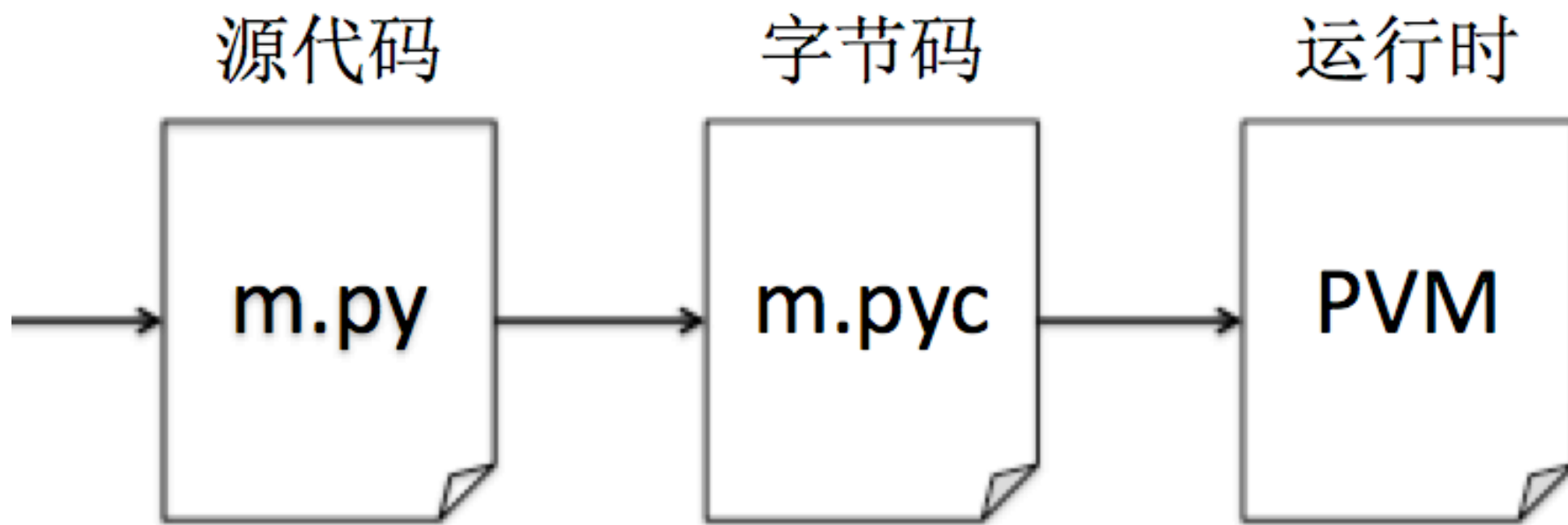
02 Python的安装

03 IPython-交互式解释器

04 集成开发环境-PyCharm

05 Python程序执行原理

Python程序执行原理



- (1) 把源代码编译成字节码
- (2) 把编译好的字节码转发到Python虚拟机（PVM）中执行

本章小结

- 首先带大家认识Python，对Python的发展历程、特点、应用领域、Python版本以及不同版本的异同进行了讲解；然后讲解了Python在不同平台下的安装方式，接着介绍了IPython和PyCharm的使用，最后带大家分析了Python程序的执行原理。
- 通过本章的学习，希望大家能够对Python有一个初步的认识，能够独立完成Python开发工具的安装和基本使用，为后面学习Python开发做好准备。

Python3

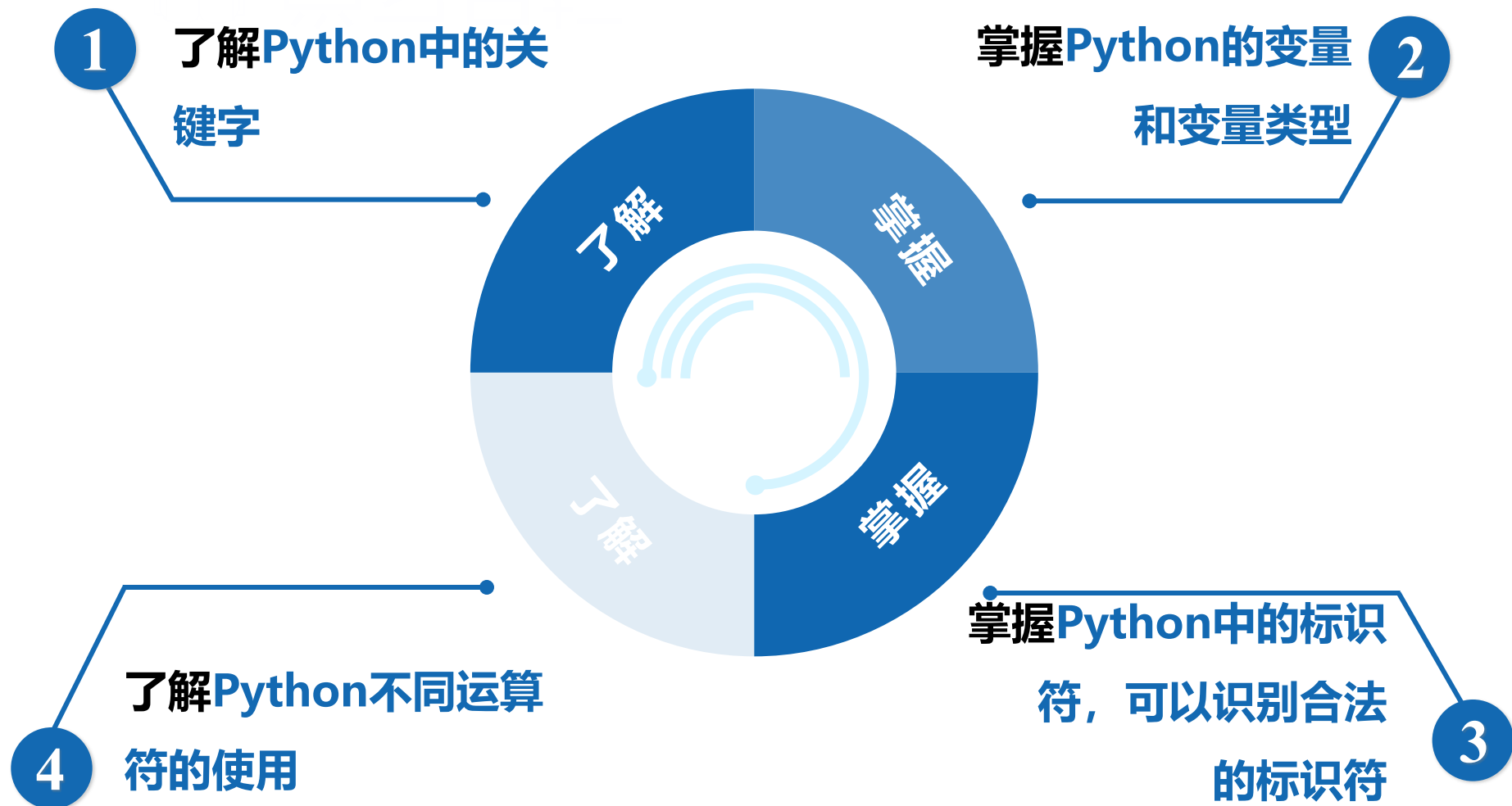
基础教程

第2章 Python语法基础

- 变量和变量类型
- 标识符和关键字
- 简单数值类型
- 运算符



学习目标



目录页



- 01** 基本语法
- 02** 变量和数据类型
- 03** 标识符和关键字
- 04** 简单数值类型
- 05** 运算符
- 06** 位运算
- 07** 运算符优先级

过渡页



01 基本语法

02 变量和数据类型

03 标识符和关键字

04 简单数值类型

05 运算符

06 位运算

07 运算符优先级

注释

Python中的单行注释以#开头!

第一个注释

```
print (“Hello, Python!”) # 第二个注释
```

注释

多行注释可以使用**三引号**作为开头和结束符号

```
"""
```

```
print(value, ..., sep=' ', end='\n',  
      file=sys.stdout, flush=False)
```

```
"""
```


行与缩进

python最具特色的就是使用**缩进来表示代码块**

if True:

 print ("True")

else:

 print ("False")

 print ("False")



if True:

 print ("True")

else:

 print ("False")

 print ("hello")



语句换行

Python 通常是一行写完一条语句，但如果语句很长，我们需要换行，这时可以使用圆括号来实现。

```
str = ('Python是一种面向对象、解释型计算机程序设计语言，'  
      '由Guido van Rossum于1989年底发明。'  
      '第一个公开发行人版发行于1991年，'  
      '源代码同样遵循 GPL(GNU General Public License)协议。')
```

语句换行

Python 通常是一行写完一条语句，但如果语句很长，我们需要换行，这时可以使用\来实现。

```
str = 'Python是一种面向对象、解释型计算机程序设计语言，'\n      '由Guido van Rossum于1989年底发明。'\n      '第一个公开发行人版发行于1991年，'\n      '源代码同样遵循 GPL(GNU General Public License)协议。'
```

语句换行

需要注意的是，在 [], {}, 或 () 中的语句，不需要使用圆括号进行换行。

```
total = ['item_one', 'item_two', 'item_three',  
        'item_four', 'item_five']
```

过渡页



01 基本语法

02 变量和数据类型

03 标识符和关键字

04 简单数值类型

05 运算符

06 位运算

07 运算符优先级

变量和赋值

Python中的变量用来存储数据，其类型和值在赋值的那一刻被初始化。

```
num1 = 100
```

```
num2 = 87
```

```
result = num1 + num2
```

变量的类型

1. 数字类型
 - 整型
 - 复数
 - 浮点型
2. 布尔类型
3. 字符串类型
4. 列表类型
5. 元组类型
6. 字典类型
7. 集合



过渡页



01 基本语法

02 变量和数据类型

03 标识符和关键字

04 简单数值类型

05 运算符

06 位运算

07 运算符优先级

标识符

水果名称

现实生活中，人们常用一些名称来标记事物，例如，现实生活中每种水果都有一个名称来标识。



ORANGE



APPLE



LEMON



KIWI



MELON



POMEGRANATE



FIGS



PEACH



WATERMELON

标识符

若希望在程序中表示一些事物，需要开发人员自定义一些符号和名称，这些符号和名称叫做**标识符**。

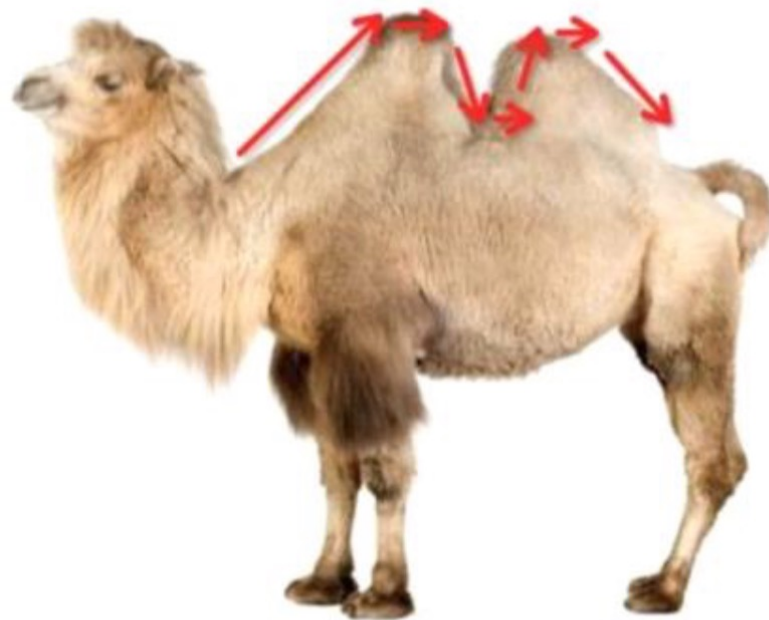
命名规则

- 标识符由字母、下划线和数字组成，且数字不能开头。
- Python中的标识符是区分大小写的。
- python中的标识符不能使用关键字

标识符

为了规范命名标识符，关于标识符的命名提以下建议：

1. 见名之意
2. 驼峰式



如：

userName

userLoginFlag

关键字

关键字指的是具有特殊功能的标识符。

```
>>> help()          # 进入帮助系统  
help> keywords      # 查看所有的关键字列表  
help> return         # 查看return这个关键字的说明  
help> quit           # 退出帮助系统
```

过渡页



01 基本语法

02 变量和数据类型

03 标识符和关键字

04 简单数值类型

05 运算符

06 位运算

07 运算符优先级

整型

整数类型 (int) 简称整型，它用于表示整数。

– 二进制: 0b10100

– 八进制: 0o10111

– 十进制: 64

– 十六进制: 0x14

浮点型

- 浮点型 (Float) 用于表示实数。
- 浮点型字面值可以用十进制或科学计数法表示。

<实数>E或者e<整数>

E或e表示基是10，后面的整数表示指数，指数的正负使用+或-表示。

布尔类型

- 布尔型数据只有两个取值：True和False
- bool值没有操作
- 所有的空都是False，所有的非空都是True

```
a = 0 # 整数0, False
print(bool(a)) s = "" # 空字符串, False
print(bool(s)) lst = [] # 空列表, False
print(bool(lst)) dic = {} # 空字典, False
print(bool(dic)) a = None # None表示空, False
print(bool(a))
```


布尔类型

– 以下对象的布尔值都是False:

- None
- False (布尔型)
- 0 (整型0)
- 0.0 (浮点型0)
- 0.0+0.0j (复数0)
- "" (空字符串)
- [] (空列表)
- () (空元组)
- {} (空字典)

复数类型

复数类型，用于表示数学中的复数，例如， $5+3j$ 。

两大特点

- (1) 复数由实数部分和虚数部分构成，表示为：
 $\text{real}+\text{imag}j$ 或 $\text{real}+\text{imag}J$
- (2) 复数的实数 real 和虚数 imag 都是浮点型

数字类型转换

函数	说明
<code>int(x [,base])</code>	将x转换为一个整数
<code>float(x)</code>	将x转换到一个浮点数， 默认六位小数
<code>complex(real [,imag])</code>	创建一个复数

过渡页



- 01** 基本语法
- 02** 变量和数据类型
- 03** 标识符和关键字
- 04** 简单数值类型
- 05** 运算符
- 06** 位运算
- 07** 运算符优先级

算术运算符

运算符	相关说明
+	加：两个对象相加
-	减：得到负数或一个数减去另一个数
*	乘：两个数相乘或是返回一个被重复若干次的字符串
/	除：x除以y
%	取余：返回除法的余数
**	幂：返回x的y次幂
//	取整除：返回商的整数部分

赋值运算符

赋值运算符只有一个，即`=`，它的作用是把等号右边的值赋给左边。例如，`x=1`

`anInt=12`

`anFloat=2.2`

`anStr='string'`

`aList=['a','a','a']`

`anArray=(1,2,3)`

`aMap={1:'a',2:'b',3:'c',}`

为多个变量赋同一个值：`x=y=z=1`

增量赋值

`x+=1` `x=x+1`

将多个值赋值给多个变量

`a, b = 1, 2`

变量交换

`X= 2`

`y =3`

`x,y = y,x`

复合赋值运算符

运算符	相关说明	实例
<code>+=</code>	加法赋值运算符	<code>c+=a</code> 等效于 <code>c=c+a</code>
<code>-=</code>	减法赋值运算符	<code>c-=a</code> 等效于 <code>c=c-a</code>
<code>*=</code>	乘法赋值运算符	<code>c*=a</code> 等效于 <code>c=c*a</code>
<code>/=</code>	除法赋值运算符	<code>c/=a</code> 等效于 <code>c=c/a</code>
<code>%=</code>	取模赋值运算符	<code>c%=a</code> 等效于 <code>c=c%a</code>
<code>**=</code>	幂赋值运算符	<code>c**=a</code> 等效于 <code>c=c**a</code>
<code>//=</code>	取整除赋值运算符	<code>c//=a</code> 等效于 <code>c=c//a</code>

复合赋值运算符

```
a = 21  
b = 10  
c = 0
```

```
c = a + b  
print ("1 - c 的值为:", c)
```

```
c += a  
print ("2 - c 的值为:", c)
```

```
c *= a  
print ("3 - c 的值为:", c)
```

```
c /= a  
print ("4 - c 的值为:", c)
```

```
c = 2  
c %= a  
print ("5 - c 的值为:", c)
```

```
c **= a  
print ("6 - c 的值为:", c)
```

```
c //= a  
print ("7 - c 的值为:", c)
```

1 - c 的值为:	31
2 - c 的值为:	52
3 - c 的值为:	1092
4 - c 的值为:	52.0
5 - c 的值为:	2
6 - c 的值为:	2097152
7 - c 的值为:	99864

比较运算符

运算符	相关说明
<code>==</code>	检查两个操作数的值是否相当
<code>!=</code>	检查两个操作数的值是否相等
<code>></code>	检查左操作数的值是否大于右操作数的值
<code><</code>	检查左操作数的值是都小于右操作数的值
<code>>=</code>	检查左操作数的值是否大于或等于右操作数的值
<code><=</code>	检查左操作数的值是否小于或等于右操作数的值

逻辑运算符

运算符	逻辑表达式	描述
and	x and y	布尔“与”，如果x为False，x and y返回False，否则它返回y的计算值
or	x or y	布尔“或”，如果x为True，它返回True，否则返回y的计算值
not	not x	布尔“非”，如果x为True，返回False，如果x为False，它返回True

逻辑运算符

```
a = 10
b = 20

if ( a and b ):
    print ("1 - 变量 a 和 b 都为 true")
else:
    print ("1 - 变量 a 和 b 有一个不为 true")

if ( a or b ):
    print ("2 - 变量 a 和 b 都为 true，或其中一个变量为 true")
else:
    print ("2 - 变量 a 和 b 都不为 true")
```

```
a = 0
if ( a and b ):
    print ("3 - 变量 a 和 b 都为 true")
else:
    print ("3 - 变量 a 和 b 有一个不为 true")

if ( a or b ):
    print ("4 - 变量 a 和 b 都为 true，或其中一个变量为 true")
else:
    print ("4 - 变量 a 和 b 都不为 true")

if not( a and b ):
    print ("5 - 变量 a 和 b 都为 false，或其中一个变量为 false")
else:
    print ("5 - 变量 a 和 b 都为 true")
```

- 1 - 变量 a 和 b 都为 true
- 2 - 变量 a 和 b 都为 true，或其中一个变量为 true
- 3 - 变量 a 和 b 有一个不为 true
- 4 - 变量 a 和 b 都为 true，或其中一个变量为 true
- 5 - 变量 a 和 b 都为 false，或其中一个变量为 false

成员运算符

运算符	描述	实例
in	如果在指定的序列中找到值返回 True，否则返回 False。	x 在 y 序列中，如果 x 在 y 序列中返回 True。
not in	如果在指定的序列中没有找到值返回 True，否则返回 False。	x 不在 y 序列中，如果 x 不在 y 序列中返回 True。

成员运算符

```
3 a = 10
4 b = 20
5 list = [1, 2, 3, 4, 5 ];
6
7 if ( a in list ):
8     print ("1 - 变量 a 在给定的列表中 list 中")
9 else:
10    print ("1 - 变量 a 不在给定的列表中 list 中")
11
12 if ( b not in list ):
13    print ("2 - 变量 b 不在给定的列表中 list 中")
14 else:
15    print ("2 - 变量 b 在给定的列表中 list 中")
16
17 # 修改变量 a 的值
18 a = 2
19 if ( a in list ):
20    print ("3 - 变量 a 在给定的列表中 list 中")
21 else:
22    print ("3 - 变量 a 不在给定的列表中 list 中")
```



过渡页



01 基本语法

02 变量和数据类型

03 标识符和关键字

04 简单数值类型

05 运算符

06 位运算

07 运算符优先级

位运算符

运算符	描述
&	按位与运算符：参与运算的两个值,如果两个相应位都为1,则该位的结果为1,否则为0
	按位或运算符：只要对应的二个二进位有一个为1时，结果位就为1。
^	按位异或运算符：当两对应的二进位相异时，结果为1
~	按位取反运算符：对数据的每个二进制位取反,即把1变为0,把0变为1。 <code>~x</code> 类似于 <code>-x-1</code>
<<	左移动运算符：运算数的各二进位全部左移若干位，由"<<"右边的数指定移动的位数，高位丢弃，低位补0。
>>	右移动运算符：把">>"左边的运算数的各二进位全部右移若干位，">>"右边的数指定移动的位数

按位与

参与运算的两个数各对应的二进位进行“与”的操作。
只有对应的两个二进位都是1时，结果位就为1，否则结果位为0

按位与 (&)

0	0	0	0	1	0	0	1
---	---	---	---	---	---	---	---

0	0	0	0	0	0	1	1
---	---	---	---	---	---	---	---

运算结果

0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---

按位或

按位或指的是参与运算的两个数各对应的二进位进行“或”的操作。只要对应的两个二进位有一个为1时，结果位就为1

按位或 (|)

运算结果

0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---

0	0	0	0	0	0	1	1
---	---	---	---	---	---	---	---

0	0	0	0	1	0	1	1
---	---	---	---	---	---	---	---

按位异或

- 按位异或就是将参与运算的两个数对应的二进制位进行比较，如果一个位为1，另一个位为0，则结果为1，否则，结果位为0。

	0	0	0	0	1	0	0	0
按位异或 (^)	0	0	0	0	0	1	0	0
运算结果	0	0	0	0	1	1	0	0

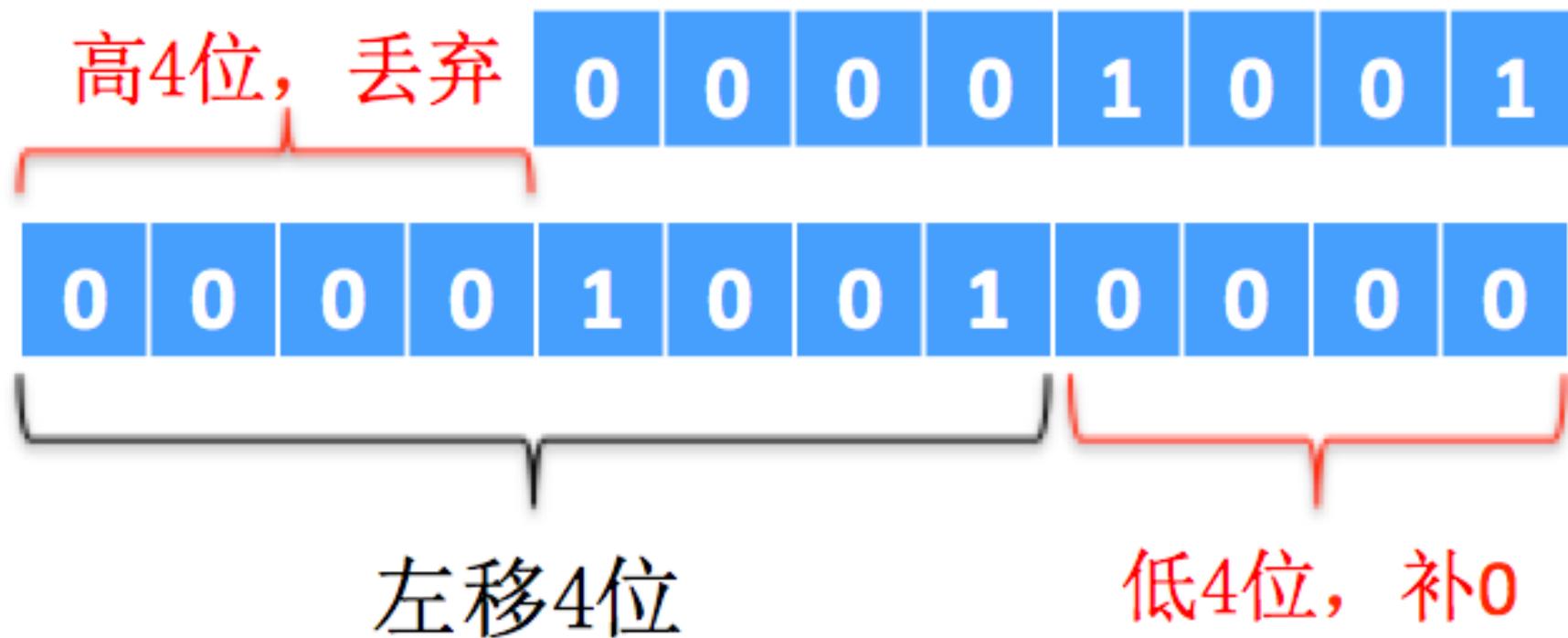
按位取反

- 按位取反就是将二进制的每一位进行取反；
- 0取反为1，1取反为0

9的二进制	0	0	0	0	1	0	0	1
按位取反	1	1	1	1	0	1	1	0

按位左移

按位左移指的是二进制全部左移 n 位，高位丢弃，低位补0。



按位右移

按位右移指的是将二进制全部右移 n 位，移出的位丢弃，移进的位补符号位。



位运算

```
a = 60          # 60 = 0011 1100
b = 13          # 13 = 0000 1101
c = 0

c = a & b;      # 12 = 0000 1100
print ("1 - c 的值为:", c)

c = a | b;      # 61 = 0011 1101
print ("2 - c 的值为:", c)

c = a ^ b;      # 49 = 0011 0001
print ("3 - c 的值为:", c)

c = ~a;         # -61 = 1100 0011
print ("4 - c 的值为:", c)

c = a << 2;      # 240 = 1111 0000
print ("5 - c 的值为:", c)

c = a >> 2;      # 15 = 0000 1111
print ("6 - c 的值为:", c)
```

1 - c 的值为: 12

2 - c 的值为: 61

3 - c 的值为: 49

4 - c 的值为: -61

5 - c 的值为: 240

6 - c 的值为: 15

过渡页



- 01** 基本语法
- 02** 变量和数据类型
- 03** 标识符和关键字
- 04** 简单数值类型
- 05** 运算符
- 06** 位运算
- 07** 运算符优先级

运算符优先级

运算符	描述
**	指数（最高优先级）
~ + -	按位翻转，一元加号和减号（最后两个的方法名为+@ 和 -@）
* / % //	乘，除，取模和取整除
+ -	加法减法
>> <<	右移，左移运算符
&	位 'AND'
^	位运算符
<= < > >=	比较运算符
<> == !=	等于运算符
= %= /= //= -= += *= **=	赋值运算符
is is not	身份运算符
not or and	逻辑运算符

假设：

$a = 20, b = 10, c = 15$

$d = 5, e = 0$

那么

$e = (a + b) * c / d = 90$

$e = ((a + b) * c) / d = 90$

$e = (a + b) * (c / d) = 90$

$e = a + (b * c) / d = 50$

本章小结

- 本章主要讲解了Python中的变量、数据类型以及运算符，这种知识都是最基础的语法，也比较容易理解。希望大家在初学Python时期，多动手写代码进行练习，加深印象，为后期的深入学习打好扎实功底。

Python3

基础教程

第3章 Python常用语句

- 判断语句

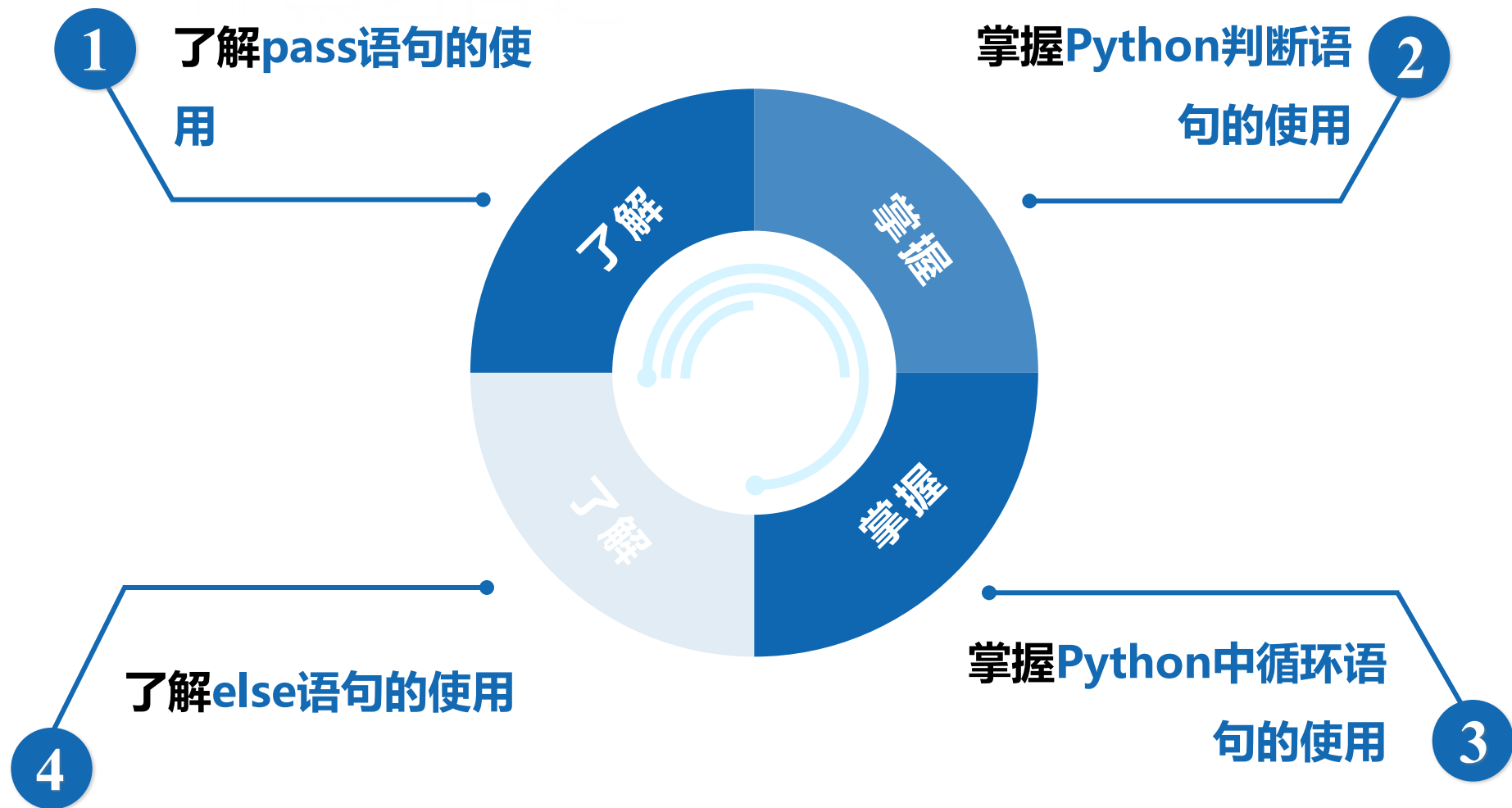
- 循环语句

- break语句

- continue语句



学习目标



目录页



01 判断语句

02 循环语句

03 **Python**的其他语句

过渡页



01 判断语句

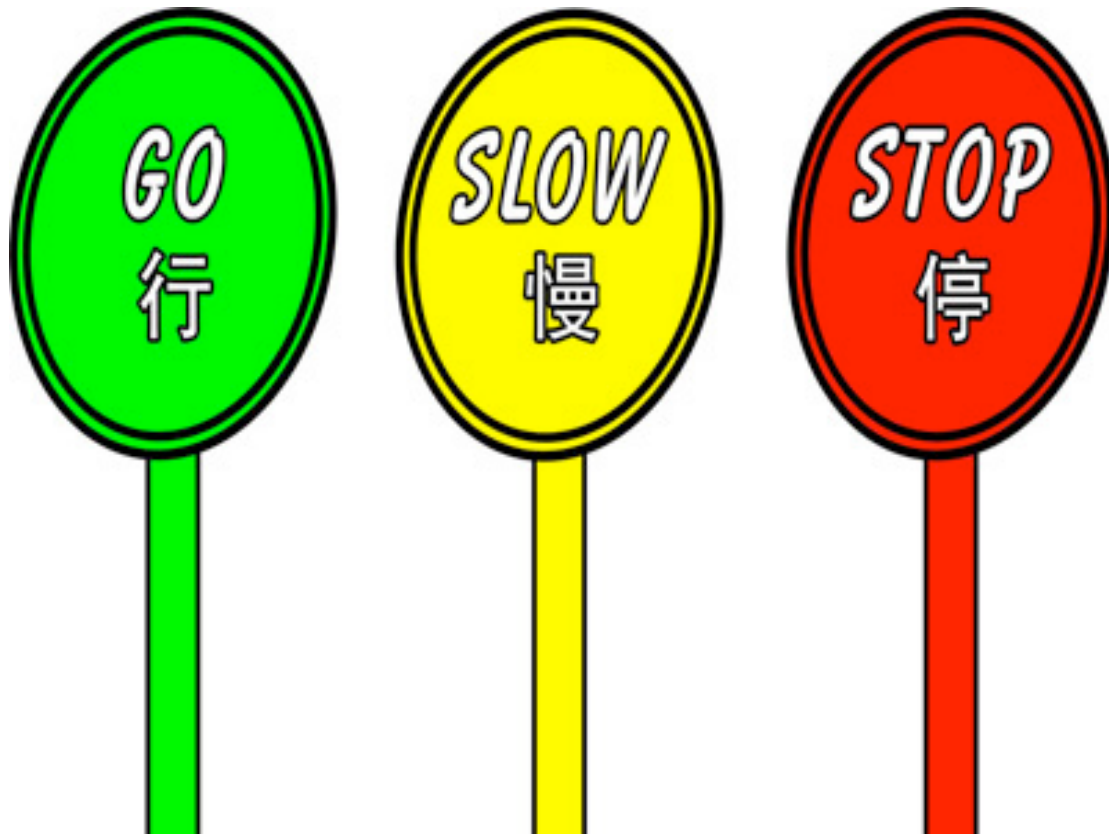
02 循环语句

03 Python的其他语句

判断语句

红灯停，绿灯行

现实生活中，过马路要看红绿灯，如果是绿灯才能过马路，否则需要停止等待。



if 语句

if语句是最简单的条件判断语句，
它可以控制程序的执行流程。

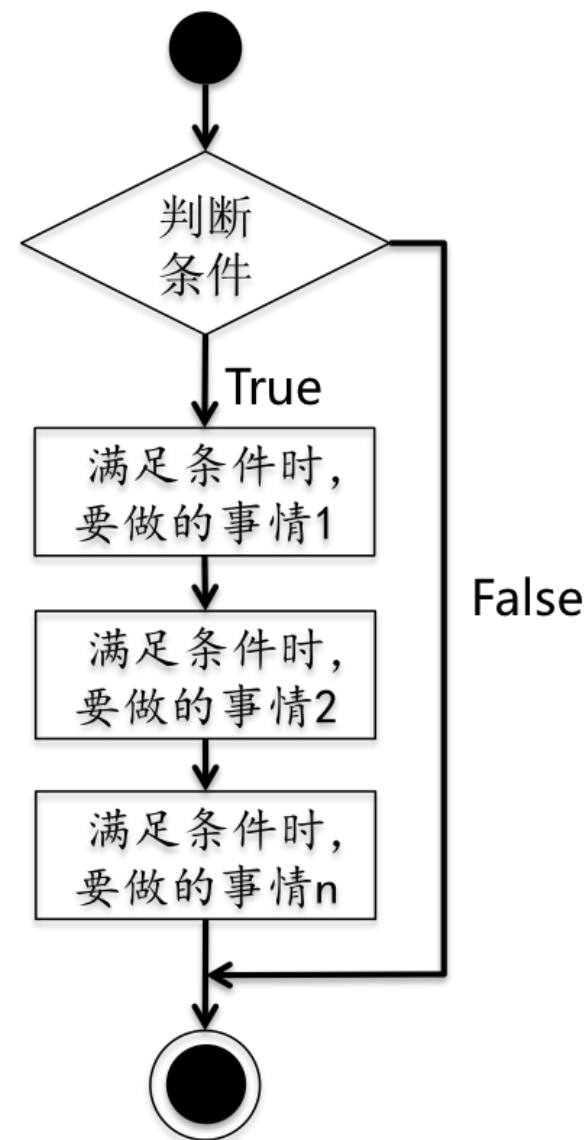
if 判断条件:

满足条件时要做的事情1...

满足条件时要做的事情2...

...(省略)...

满足条件时要做的事情n...



if-else语句

if 条件:

满足条件时要做的事情1

满足条件时要做的事情2

...(省略)...

满足条件时要做的事情3

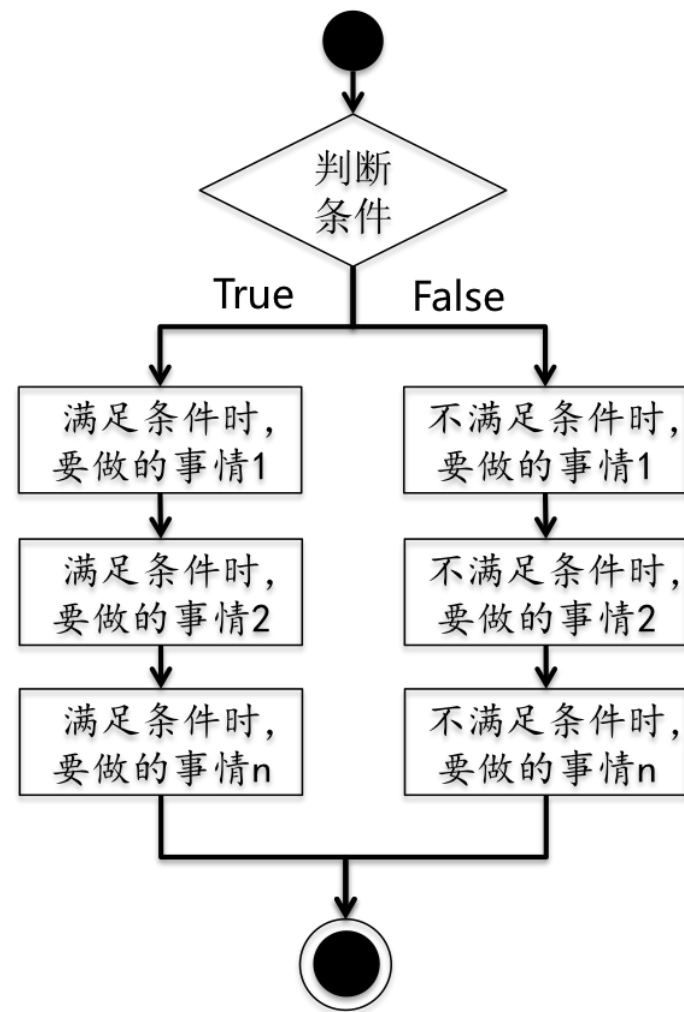
else:

不满足条件时要做的事情1

不满足条件时要做的事情2

...(省略)...

不满足条件时要做的事情n



if-elif语句

if-elif判断语句，该语句可以判断多种情况。

if 判断条件1:

 满足条件1时要做的事情

elif 判断条件2:

 满足条件2时要做的事情

elif 判断条件3:

 满足条件3时要做的事情

- 当满足判断条件1时，执行满足条件1时要做的事情，然后整个if结束；
- 如果不满足判断条件1，那么判断是否满足条件2，如果满足判断条件2，就执行满足条件2时要做的事情，然后整个if结束
- 当不满足判断条件1和判断条件2，如果满足判断条件3，则执行满足判断条件3时要做的事情，然后整个if结束。

if嵌套



购票

安检



上车



检票



if嵌套

if嵌套指的是在if或者if-else语句里面包含if或者if-else。

if 条件1:

 满足条件1 做的事情1

 满足条件1 做的事情2

...(省略)...

if 条件2:

 满足条件2 做的事情1

 满足条件2 做的事情2

...(省略)...



判断语句

例1: if 基本用法

```
flag = False
name = 'luren'
if name == 'python':           # 判断变量否为'python'
    flag = True                # 条件成立时设置标志为真
    print 'welcome boss'      # 并输出欢迎信息
else:
    print name                  # 条件不成立时输出变量名称
```

luren

输出结果

```
#!/usr/bin/python
# -*- coding: UTF-8 -*-
# 例2: elif用法
```

```
num = 5
if num == 3:                    # 判断num的值
    print 'boss'
elif num == 2:
    print 'user'
elif num == 1:
    print 'worker'
elif num < 0:                   # 值小于零时输出
    print 'error'
else:
    print 'roadman'            # 条件均不成立时输出
```

roadman

输出结果

判断语句

```
num = 9
if num >= 0 and num <= 10:    # 判断值是否在0~10之间
    print 'hello'
# 输出结果: hello

num = 10
if num < 0 or num > 10:    # 判断值是否在小于0或大于10
    print 'hello'
else:
    print 'undefine'
# 输出结果: undefine

num = 8
# 判断值是否在0~5或者10~15之间
if (num >= 0 and num <= 5) or (num >= 10 and num <= 15):
    print 'hello'
else:
    print 'undefine'
# 输出结果: undefine
```

过渡页



01 判断语句

02 循环语句

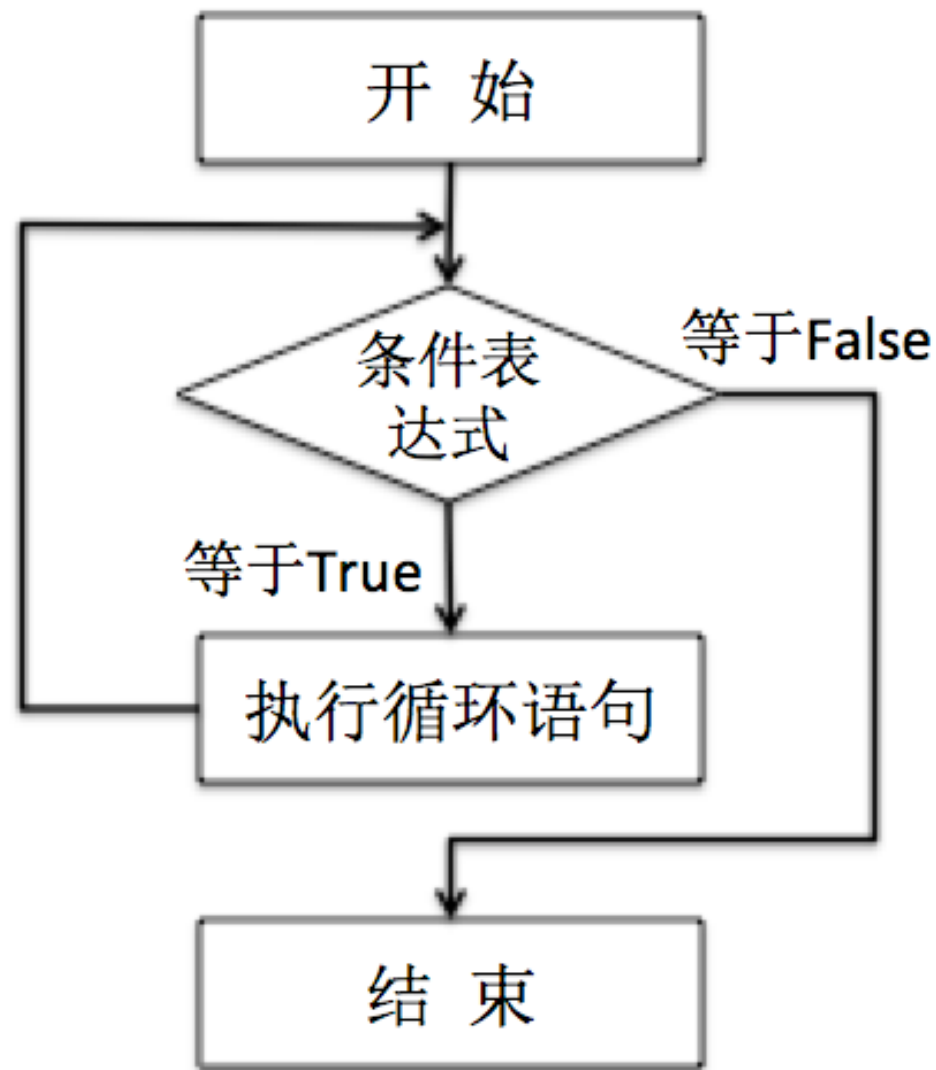
03 Python的其他语句

while循环

while循环语法格式

while 条件表达式:
 条件满足, 执行循环语句

条件表达式永远为true, 无限循环。



while循环

```
#!/usr/bin/python

count = 0
while (count < 9):
    print 'The count is:', count
    count = count + 1

print "Good bye!"
```

The count is: 0

The count is: 1

The count is: 2

The count is: 3

The count is: 4

The count is: 5

The count is: 6

The count is: 7

The count is: 8

Good bye!

for循环

Python中的for循环可以遍历任何序列的项目：

语
法
格
式

```
for 变量 in 序列:  
    循环语句
```

例如：

```
for i in [0,1,2]:  
    print(i)
```

for循环

```
for letter in 'Python':    # 第一个实例
    print '当前字母 :', letter

fruits = ['banana', 'apple', 'mango']
for fruit in fruits:        # 第二个实例
    print '当前水果 :', fruit

print "Good bye!"
```

当前字母 : P

当前字母 : y

当前字母 : t

当前字母 : h

当前字母 : o

当前字母 : n

当前水果 : banana

当前水果 : apple

当前水果 : mango

Good bye!

for循环

考虑到我们使用的数值范围经常变化，Python提供了一个内置range()函数，它可以生成一个数字序列。

语法格式

```
for i in range(start,end):  
    执行循环语句
```

程序在执行for循环时：

- 循环计时器变量i被设置为start;
- 执行循环语句;
- i递增
- 每设置一个新值都会执行一次循环
- 当i等于end时，循环结束。

for循环

```
fruits = ['banana', 'apple', 'mango']
for index in range(len(fruits)):
    print '当前水果 :', fruits[index]

print "Good bye!"
```

当前水果 : banana

当前水果 : apple

当前水果 : mango

Good bye!

```
for num in range(10,20): # 迭代 10 到 20 之间的数字
    for i in range(2,num): # 根据因子迭代
        if num%i == 0: # 确定第一个因子
            j=num/i # 计算第二个因子
            print '%d 等于 %d * %d' % (num,i,j)
            break # 跳出当前循环
    else: # 循环的 else 部分
        print num, '是一个质数'
```

10 等于 2 * 5

11 是一个质数

12 等于 2 * 6

13 是一个质数

14 等于 2 * 7

15 等于 3 * 5

16 等于 2 * 8

17 是一个质数

18 等于 2 * 9

19 是一个质数

while嵌套

while嵌套指的是while循环里面还包含while循环。

while 条件1:

条件1满足时，做的事情1

条件1满足时，做的事情2

条件1满足时，做的事情3

...(省略)...

while 条件2:

条件2满足时，做的事情1

条件2满足时，做的事情2

条件2满足时，做的事情3

...(省略)...

while嵌套

```
i = 2
while(i < 100):
    j = 2
    while(j <= (i/j)):
        if not(i%j): break
        j = j + 1
    if (j > i/j) : print i, " 是素数"
    i = i + 1

print "Good bye!"
```

2 是素数

3 是素数

5 是素数

7 是素数

11 是素数

13 是素数

17 是素数

19 是素数

23 是素数

29 是素数

31 是素数

过渡页



01 判断语句

02 循环语句

03 Python的其他语句

break语句

break语句用于结束整个循环。

```
for i in range(5):  
    print("-----")  
    print (i)
```

这两个结果有什么不同？

```
i=1  
for i in range(5):  
    i+=1  
    print("-----")  
    if i==3:  
        break  
    print(i)
```


continue语句

continue的作用是用来结束本次循环，紧接着执行下一次的循环。

```
for i in range(5):  
    print("-----")  
    print i
```

这两个结果有什么不同？

```
i=1  
for i in range(5):  
    i+=1  
    print("-----")  
    if i==3:  
        continue  
    print(i)
```

pass语句

pass是空语句，它是为了保持程序结构完整性。

```
for letter in 'Runoob':  
    if letter == 'o':  
        pass  
        print ('执行 pass 块')  
    print ('当前字母:', letter)  
print ("Good bye!")
```

pass语句不做任何事情，
用作占位。

else语句

else语句可以和循环语句结合使用，并且else语句旨在循环完成后执行。

```
count = 0
while count < 5:
    print(count, " is less than 5")
    count = count + 1
else:
    print(count, " is not less than 5")
```

运行结果是什么？

本章小结

本章主要介绍的是Python中的常用语句，包括判断语句、循环语句以及其他子句。其中，判断语句主要是if语句，循环语句主要是for语句和while语句。在Python开发中，这些语句的使用频率非常高，希望读者可以多加以理解，并熟练掌握它们的使用。

Python3

基础教程

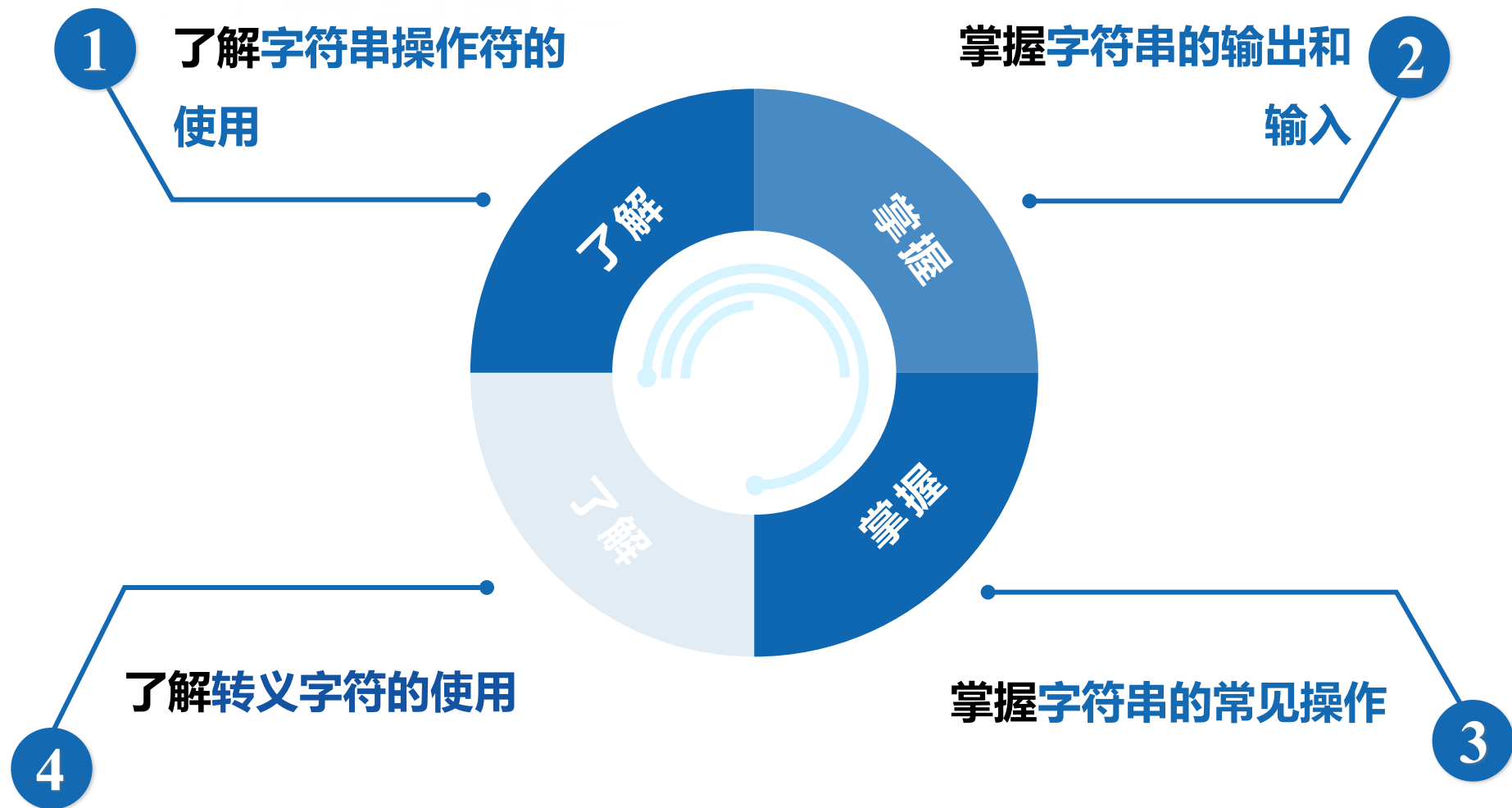


第4章 字符串

- 字符串输入输出
- 字符串的内建函数
- 切片
- 字符串运算符



学习目标



目录页



01 字符串介绍

02 字符串的输出和输入

03 访问字符串中的值

04 字符串内建函数

05 字符串运算符

过渡页



01 字符串介绍

02 字符串的输出和输入

03 访问字符串中的值

04 字符串内建函数

05 字符串运算符

什么是字符串

字符串是一种表示文本数据的类型。

使用单引号

'a' 、 '123'

使用双引号

"a"、 "123"

使用三引号

"""

Hello

"""

转义字符

看下面的代码：

```
>>>'let's go! go'
File "<input>", line 1
    'let's go! go'
      ^
SyntaxError: invalid syntax
```

对于单引号或者双引号这些特殊的符号，我们可以对他们进行**转义**。例如，对字符串中的单引号进行转义：

```
>>>'let\'s go! go'
"let's go! go"
```

转义字符

转义字符	代表含义
\\(在行尾时)	反斜杠符号
\\	反斜杠符号
\"	双引号
\\n	换行
\\b	退格
\\t	横向制表符

过渡页



01 字符串介绍

02 字符串的输出和输入

03 访问字符串中的值

04 字符串内建函数

05 字符串运算符

字符串的输出

比如有以下代码:

```
print("我今年10岁")  
print("我今年11岁")  
print ("我今年12岁")  
...
```

大家试想一下, 上述代码多次输出"我今年xx岁", 是否有一种简化程序的方式呢?

当然有。可以通过字符串格式化来完成。

字符串的输出

下面是字符串的格式化输出

```
name = '小明'
```

```
age = 5
```

```
print("大家好， 我叫%s ,我今年%d岁了"%(name,age))
```

字符串的输出

常见的格式化符号

格式化符号	转换
%s	通过str()字符串转换来格式化
%d	有符号十进制整数
%f	浮点实数

字符串的输入

Python3提供了 `input()` 函数从标准输入读取一行文本，默认的标准输入是键盘。`input`可以接收一个Python表达式作为输入，并将运算结果返回。

```
username=input("请输入用户名")  
print(username)
```



过渡页



01 字符串介绍

02 字符串的输出和输入

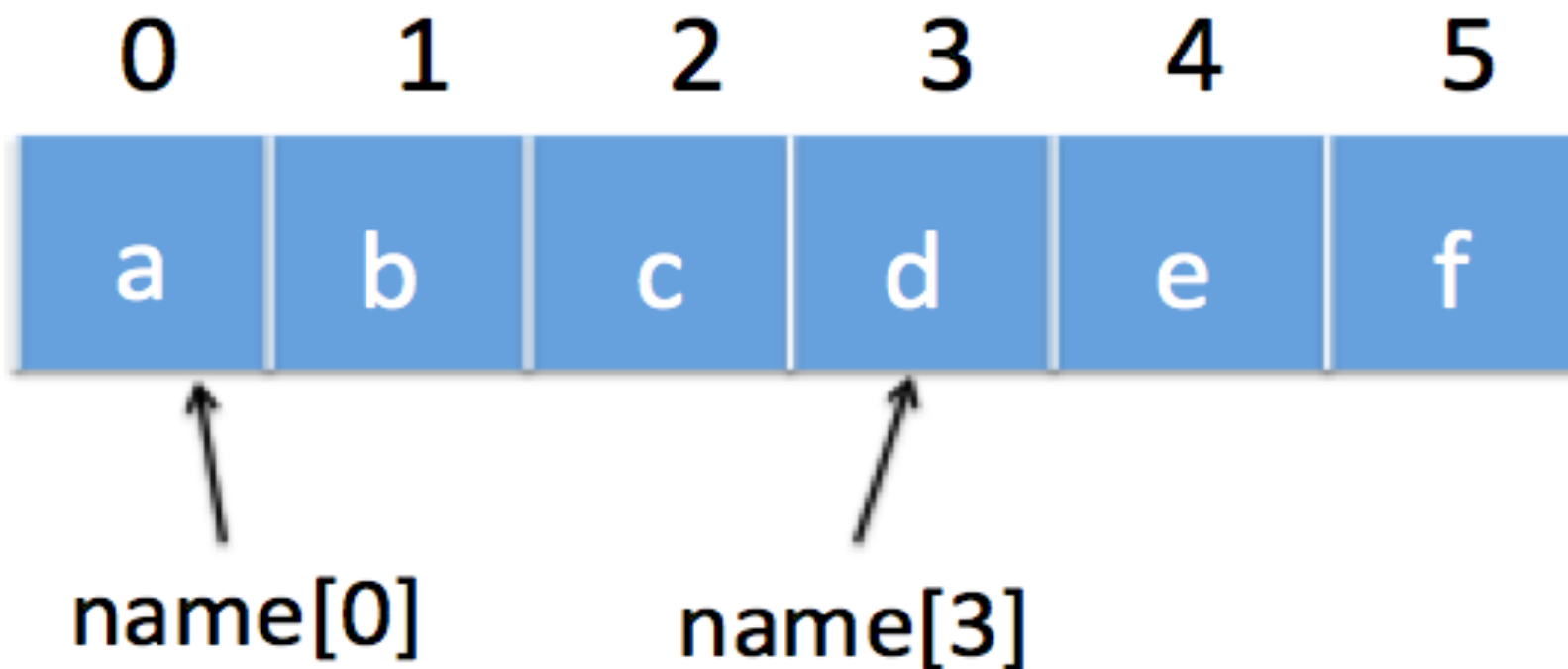
03 访问字符串中的值

04 字符串内建函数

05 字符串运算符

字符串的存储方式

字符串中的每个字符都对应一个下标，下标编号是从0开始的。



什么是切片

切片的语法格式如下所示：

[起始:结束:步长]

切片选取的区间属于左闭右开型，即从"起始"位开始，到"结束"位的前一位结束（不包含结束位本身）

使用切片截取字符串

假设有字符串 `name= "abcdef"` , 则:

<code>name[0:3]</code>	→	<code>abc</code>
<code>name[3:5]</code>	→	<code>de</code>
<code>name[1:-1]</code>	→	<code>bcde</code>
<code>name[2:]</code>	→	<code>cdef</code>
<code>name[::-2]</code>	→	<code>fdb</code>

过渡页



01 字符串介绍

02 字符串的输出和输入

03 访问字符串中的值

04 字符串内建函数

05 字符串运算符

字符串内建函数

- find函数：检测字符串是否包括子字符串

```
str.find(string, beg=0, end=len(str))
```

参数如下：

- str -- 指定检索的字符串。
- beg -- 开始索引，默认为0。
- end -- 结束索引，默认为字符串的长度。
- 返回索引，如果找不到，结果是-1

字符串内建函数

- index函数：检测字符串是否包括子字符串

```
str.index(string, beg=0, end=len(str))
```

参数如下：

- str -- 指定检索的字符串。
- beg -- 开始索引，默认为0。
- end -- 结束索引，默认为字符串的长度。
- 返回索引，如果找不到，报错

字符串内建函数

- **replace函数：将旧字符串替换为新字符串**

```
str.replace(old, new[, max])
```

参数如下：

- **old** -- 将被替换的字符串。
- **new** -- 新字符串，用于替换old字符串。
- **max** -- 可选字符串，替换不超过max次

字符串内建函数

- **count函数：统计字符串中某个字符的个数**

```
str.count(sub, start= 0, end=len(str))
```

参数如下：

- **sub** -- 搜索的子字符串
- **start** -- 字符串开始搜索的位置
- **end** -- 字符串中结束搜索的位置

字符串内建函数

- **split函数**：通过指定分隔符对字符串进行切片

```
str.split(string="", num=str.count(string))
```

参数如下：

- **str** -- 分隔符。py2默认为所有空字符，py3不能为空。
- **num** -- 分割次数
- 生成列表

字符串内建函数

- **capitalize**: 第一个字符大写, 其他字符小写

```
str.capitalize()
```

- **title**: 所有单词首字母大写, 其余字母小写

```
str.title()
```

字符串内建函数

- **startswith**: 检查字符串是否以指定子串开头

```
str.startswith(string, beg=0, end=len(str))
```

参数如下:

- **str** -- 检测的字符串。
- **strbeg** -- 可选参数用于设置字符串检测的起始位置。
- **strend** -- 可选参数用于设置字符串检测的结束位置。

字符串内建函数

- **endswith**: 检查字符串是否以制定子串结尾

```
str.endswith(suffix[, start[, end]])
```

参数如下:

- **suffix** -- 该参数可以是一个字符串或者是一个元素。
- **start** -- 字符串中的开始位置。
- **end** -- 字符串中的结束位置。

字符串内建函数

- upper: 将小写字母转为大写字母

```
str.upper()
```

```
mystr = 'hello world itheima and itheimaApp'
```

```
newStr = mystr.upper()
```

结果为: HELLO WORLD ITHEIMA AND ITHEIMAAPP

字符串内建函数

- ljust: 左对齐, 使用空格填充至指定长度的新字符串

```
str.ljust(width[, fillchar])
```

参数如下:

➤width -- 指定字符串长度。

➤fillchar -- 填充字符, 默认为空格。

字符串内建函数

- `rjust`: 右对齐, 使用空格填充至指定长度的新字符串

```
str.rjust(width[, fillchar])
```

参数如下:

➤ `width` -- 指定字符串长度。

➤ `fillchar` -- 填充字符, 默认为空格。

字符串内建函数

- center: 返回一个指定的宽度 width 居中的字符串

```
str.center(width[, fillchar])
```

参数如下:

- width -- 字符串的总宽度。
- fillchar -- 填充字符。

字符串内建函数

- **lstrip**: 截掉字符串左边的空格或指定字符

```
str.lstrip([chars])
```

参数如下:

➤ **chars** --指定删除的字符。

字符串内建函数

- `rstrip`: 截掉字符串右边的空格或指定字符

```
str.rstrip([chars])
```

参数如下:

➤ `chars` - 制定删除的字符。

字符串内建函数

- strip: 截掉字符串左右边的空格或指定字符

```
str.strip([chars])
```

参数如下:

➤chars --移除字符串头尾指定的字符。

过渡页



01 字符串介绍

02 字符串的输出和输入

03 访问字符串中的值

04 字符串内建函数

05 字符串运算符

字符串运算符

操作符	描述
+	字符串连接
*	重复输出字符串
[]	通过索引获取字符串中的字符
[:]	截取字符串中的一部分

字符串运算符

操作符	描述
in	成员运算符，如果字符串中包含给定的字符，返回True
not in	成员运算符。如果字符串中不包含给定字符，返回True
r/R	原始字符串：所有的字符串都是直接按照字面的意思来使用，没有转义特殊或不能打印的字符。

练习-作业1

1.打印九九乘法表

2.求 $1-2+3-4+5 \dots 99$ 的所有数的和

3.输入一行字符， 分别统计出其中
英文字母、空格、数字和其它字符的个数。

本章小结

本章首先对字符串进行了讲解，介绍了什么是字符串以及对特殊字符的转义；接着讲解了字符串的格式化输入和输出、字符串的访问；最后讲解了字符串的内建函数以及字符串运算符。通过本章的学习，希望读者能结合案例多加练习，熟练掌握字符串操作的相关函数。

Python3

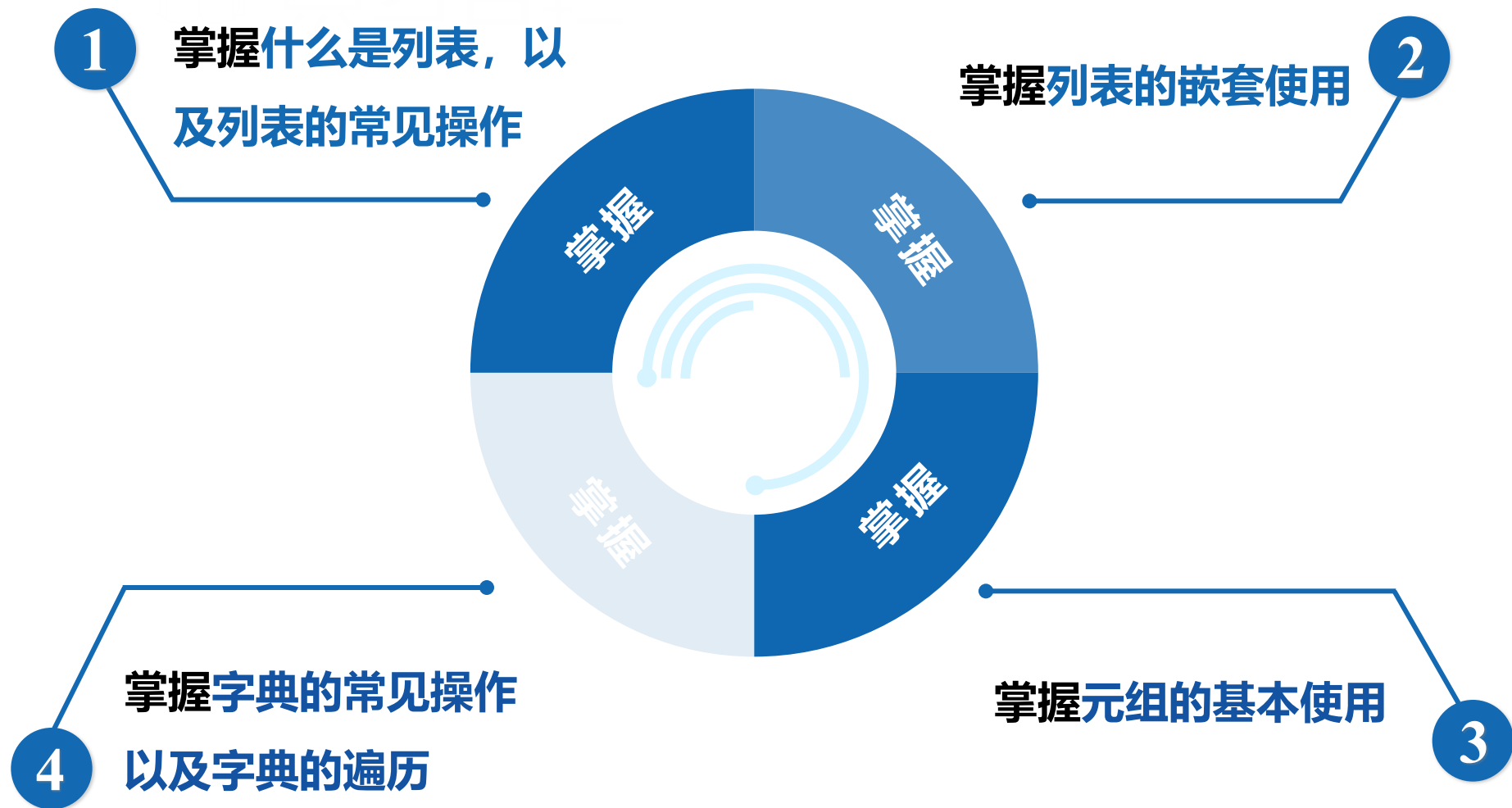
基础教程

第5章 列表、元组和字典

- 列表
- 元组
- 字典



学习目标



目录页



- 01** 列表概述
- 02** 列表的循环遍历
- 03** 列表的常见操作
- 04** 列表的嵌套
- 05** 元组
- 06** 字典

过渡页



01 列表概述

02 列表的循环遍历

03 列表的常见操作

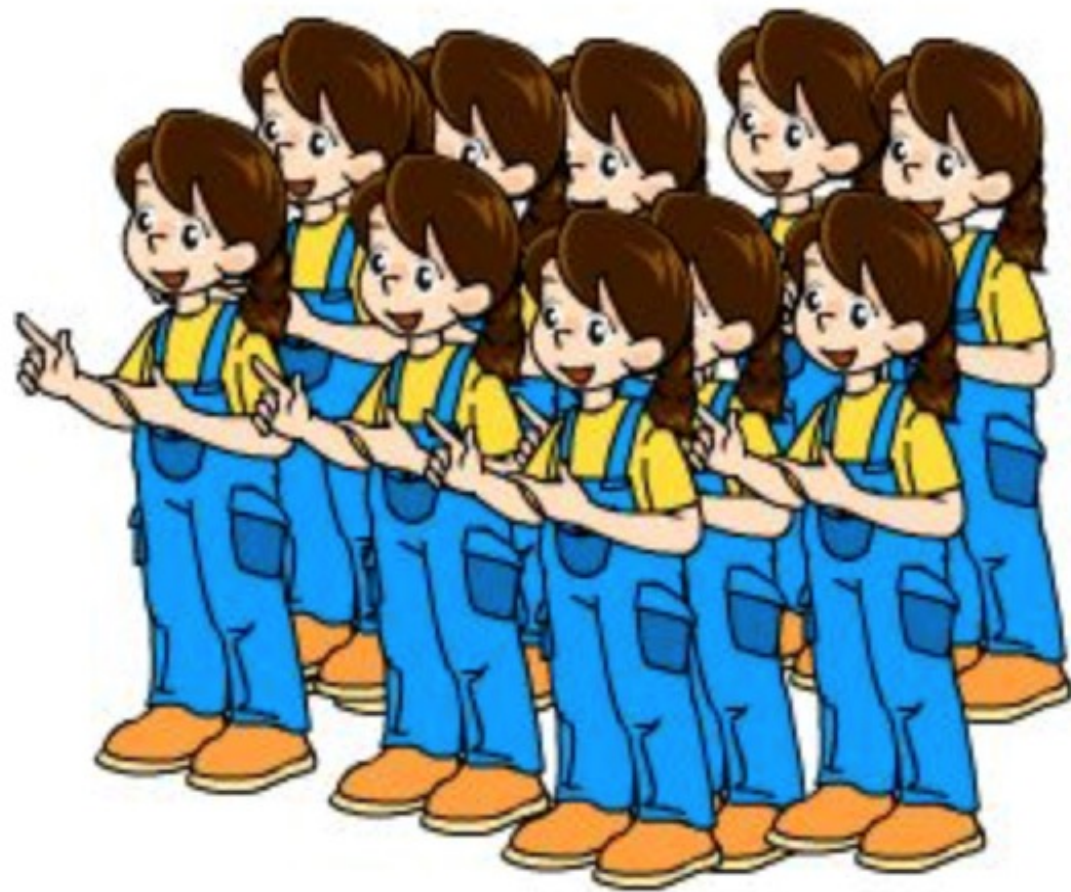
04 列表的嵌套

05 元组

06 字典

列表概述

假设一个班有100个学生，如果每个变量存放一个学生的姓名，是不是很方便？如果有一千个学生甚至更多，那该怎么办呢？



列表概述

列表是Python中的一种数据结构，它可以存储不同类型的数据。例如：

```
A = [1, 'xiaoWang', 'a', [2, 'b']]
```

列表概述

列表索引是从0开始的，我们可以通过下标索引的方式来访问列表中的值。

```
A = ['xiaoWang', 'xiaoZhang', 'xiaoHua']  
print(A[0])  
print(A[1])
```


过渡页



01 列表概述

02 列表的循环遍历

03 列表的常见操作

04 列表的嵌套

05 元组

06 字典

列表的循环遍历

1. 使用for循环遍历列表

```
namesList = ['xiaoWang','xiaoZhang','xiaoHua']  
for name in namesList:  
    print(name)
```

列表的循环遍历

2. 使用while循环遍历列表

```
namesList = ['xiaoWang', 'xiaoZhang', 'xiaoHua']  
length = len(namesList)  
i = 0  
while i < length:  
    print(namesList[i])  
    i += 1
```

过渡页



01 列表概述

02 列表的循环遍历

03 列表的常见操作

04 列表的嵌套

05 元组

06 字典

在列表中增加元素

在列表中增加元素的方式有多种，具体如下：

- 通过 **append** 可以向列表添加元素
- 通过 **extend** 可以将另一个列表的元素添加到列表中。
- 通过 **insert** 在指定位置 `index` 前插入元素 `object`。

在列表中查找元素

在列表中查找元素的方法包括：

- `in`（存在）,如果存在那么结果为`true`，否则为`false`。
- `not in`（不存在）， 如果不存在那么结果为`true`， 否则`false`。

在列表中修改元素

列表元素的修改，也是通过下标来实现的。

```
A = ['xiaoWang','xiaoZhang','xiaoHua']
```

```
A[1] = 'xiaoLu'
```

在列表中删除元素

列表元素的常用删除方法有三种，具体如下：

- `del`：根据下标进行删除
- `pop`：删除最后一个元素
- `remove`：根据元素的值进行删除

列表的排序操作

列表的排序可以通过下面两个方法实现：

sort方法：列表的元素按照特定顺序排列。

reverse方法：将列表逆置。

过渡页



01 列表概述

02 列表的循环遍历

03 列表的常见操作

04 列表的嵌套

05 元组

06 字典

列表的嵌套

列表的嵌套指的是一个列表的元素又是一个列表。

```
schoolNames = [['北京大学','清华大学'],  
                ['南开大学','天津大学','天津师范大学'],  
                ['山东大学','中国海洋大学']]
```

过渡页



01 列表概述

02 列表的循环遍历

03 列表的常见操作

04 列表的嵌套

05 元组

06 字典

什么是元组

Python的元组与列表类似，不同之处在于元组的元素不能修改。元组使用小括号，列表使用方括号。

```
tup1 = ('physics', 'chemistry', 1997, 2000)
```

```
tup2 = (1, 2, 3, 4, 5 )
```

```
tup3 = "a", "b", "c", "d"
```

元组的相关操作

1. 访问元组

元组可以使用下标索引来访问元组中的值

```
tuple=('hello',100,4.5)
```

```
print(tuple[0])
```

```
print(tuple[1])
```

```
print(tuple[2])
```

元组的相关操作

2. 元组不允许修改元组

```
tup1 = (12, 34.56);  
tup2 = ('abc', 'xyz')  
# 以下修改元组元素操作是非法的。  
tup1[0] = 100
```



元组的相关操作

3. 元组的遍历

```
a_tuple = (1, 2, 3, 4, 5)
for num in a_tuple:
    print(num,end=" ")
```


元组的相关操作

4. 元组的内置函数

方法	描述
<code>len(tuple)</code>	计算元组元素个数
<code>max(tuple)</code>	返回元组中元素最大值
<code>min(tuple)</code>	返回元组中元素最小值
<code>tuple(seq)</code>	将列表转为元组

过渡页



01 列表概述

02 列表的循环遍历

03 列表的常见操作

04 列表的嵌套

05 元组

06 字典

字典介绍

字典是一种存储数据的容器，它和列表一样，都可以存储多个数据。

```
info = {'name':'班长', 'sex':'f', 'address':'北京'}
```

每个元素都是由两部分组成的，分别是键和值。

‘name’ 为键， ‘班长’ 为值。



字典的常见操作

1. 根据键访问值

```
info = {'name':'班长', 'id':100, 'sex':'f', 'address':'北京'}  
print(info['name'])  
print(info['address'])
```

注意：如果使用的是不存在的键，则程序会报错。

字典的常见操作

如果我们不确定字典中是否存在某个键而又想获取其值时，可以使用get方法，还可以设置默认值。

```
info = {'name':'班长', 'id':100, 'sex':'f', 'address':'北京'}  
age = info.get('age')  
print(age) # 'age'键不存在，所以age为None  
print(type(age))  
age = info.get('age', 18) # 若info不存在'age'，返回默认值18  
print(age)
```

字典的常见操作

2. 修改字典的元素

```
info = {'name':'班长', 'id':100, 'sex':'f', 'address':'北京'}  
newId = input('请输入新的学号')  
info['id'] = int(newId)  
print('修改之后的id为: %d'%info['id'])
```

字典的常见操作

3. 添加字典元素

```
info = {'name':'班长', 'id':100, 'sex':'f', 'address':'北京'}  
newId = input('请输入新的学号')  
info['id'] = newId  
print(info)
```

字典的常见操作

4. 删除字典元素

- **del**: 用于删除字典；删除后，字典完全不存在了，无法再根据键访问字典的值。
- **clear**: 只是清空字典中的数据，字典还存在，只不过没有元素。

字典的常见操作

5. 计算字典中键值对的个数

```
dict = {'Name': 'Zara', 'Age': 7}  
print("Length : %d" % len (dict))
```

字典的常见操作

6. 获取字典中键的列表

keys()方法返回在字典中的所有可用的键的列表。

```
dict = {'Name': 'Zara', 'Age': 7};  
print(dict.keys())
```

字典的常见操作

7. 获取字典中值的列表

values()方法返回在字典中的所有可用的值的列表

```
dict = {'Name': 'Zara', 'Age': 7};  
print(dict.values())
```

字典的常见操作

8. 计算字典中键值对的个数

items()方法返回字典的(键, 值)元组对的列表

```
dict = {'Name': 'Zara', 'Age': 7}  
print("Value : %s" % dict.items())
```

字典的遍历

1. 遍历字典的键key

```
dict = {'Name': 'Zara', 'Age': 7}
for key in dict.keys():
    print(key)
```

字典的遍历

2. 遍历字典的值value

```
dict = {'Name': 'Zara', 'Age': 7}

for value in dict.values():

    print(value)
```

字典的遍历

3. 遍历字典的元素

```
dict = {'Name': 'Zara', 'Age': 7}

for item in dict.items():
    print(item)
```

字典的遍历

4. 遍历字典的键值对

```
dict = {'Name': 'Zara', 'Age': 7}

for key,value in dict.items():

    print("key=%s,  value=%s"%(key,value))
```


本章小结

本章主要介绍了列表、元组和字典三种类型，希望大家通过本章的学习，能够清楚的知道这三种类型各自的特点，这样在后续开发过程中，可以选择合适的类型对数据进行操作。

Python3

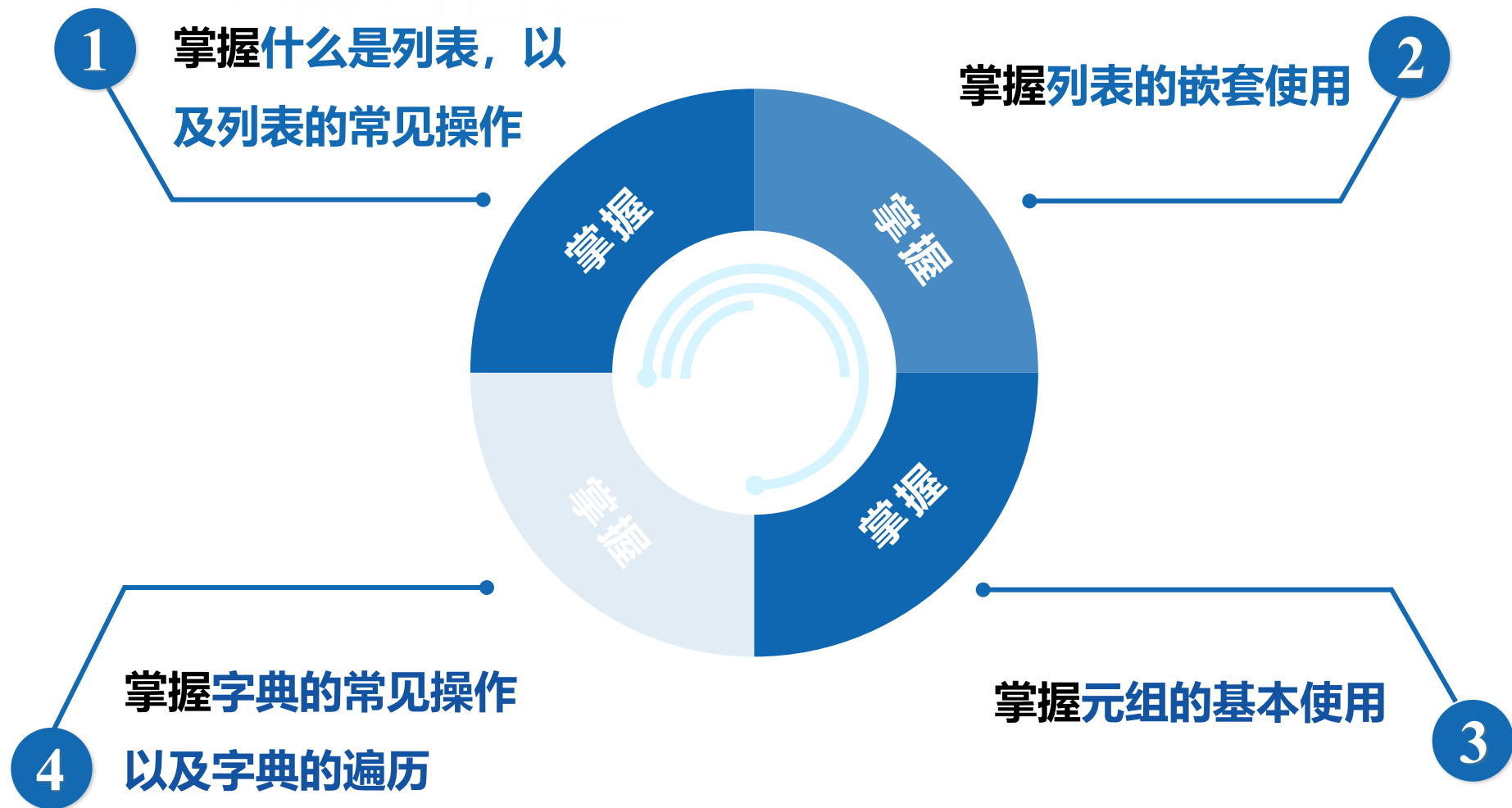
基础教程

第6章 函数

- 函数的定义和调用
- 函数的参数
- 函数的返回值
- 变量作用域
- 递归函数和匿名函数
- 日期时间、随机数函数



学习目标



目录页



- 01** 什么是函数
- 02** 函数的定义和调用
- 03** 函数的参数
- 04** 函数的返回值
- 05** 函数的四种类型

目录页



06 函数的嵌套调用

07 变量的作用域

08 递归函数和匿名函数

09 日期时间函数

10 随机数函数

过渡页



01 什么是函数

02 函数的定义和调用

03 函数的参数

04 函数的返回值

05 函数的四种类型

什么是函数？

函数是组织好的，可重复使用的，用来实现单一或相关联功能的代码段，它能够提高应用的模块性和代码的重复利用率。

```
print(" * ")  
print(" *** ")  
print("*****")
```

如果在一个程序的不同地方需要输出这个图形，每次使用print语言输出的做法显然不可取。

过渡页



01 什么是函数

02 函数的定义和调用

03 函数的参数

04 函数的返回值

05 函数的四种类型

函数的定义和调用

Python定义函数使用`def`关键字，格式如下：

```
def 函数名（参数列表）：
```

```
    函数体
```

函数的定义和调用

这是一个自定义的函数：

```
def printInfo():  
    print('-----')  
    print('    生命苦短，我用Python    ')  
    print('-----')
```

函数的定义和调用

定义了函数之后，想要让这些代码能够执行，需要调用函数。通过 “**函数名()**” 即可完成调用。

```
# 调用刚才定义的函数  
printInfo()
```

过渡页



01 什么是函数

02 函数的定义和调用

03 函数的参数

04 函数的返回值

05 函数的四种类型

函数的返回值

先看一段代码：

```
def add2num():  
    c = 11+22  
    print(c)
```

这个函数计算的只是固定的两个数，没有什么意义。

函数的返回值

如果希望定义的函数，可以计算任何两个数和，我们在定义函数的时候，让函数接收数据，这就是函数的参数。

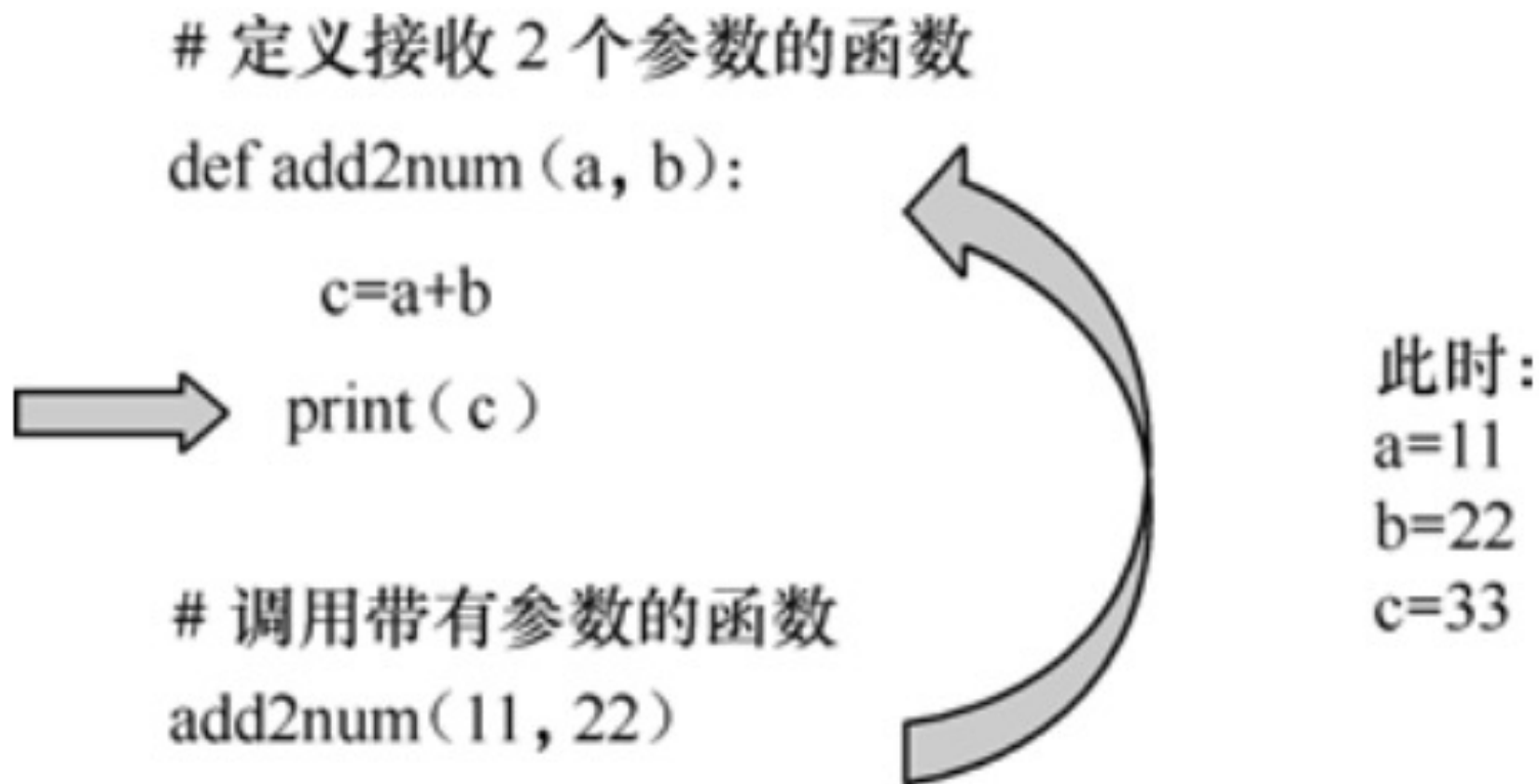
```
def add2num(a, b):  
    c = a+b  
    print(c)
```

a和b就是函数的参数，调用的时候，可以传入任何两个数。

```
add2num(11, 22)
```

函数的返回值

参数调用过程



默认参数

调用函数时，如果没有传递参数，则会使用默认参数。

```
def printinfo( name, age = 35 ):  
    print("Name:", name)  
    print("Age:", age)  
    printinfo(name="miki" )  
    printinfo(age=9,name="miki" )
```

带有默认值的参数
一定要位于参数列表的后面。否则
程序会报错。

不定长参数

有时可能需要一个函数能处理比当初声明时更多的参数，这些参数叫做**不定长参数**，声明时不会命名。

```
def functionname([formal_args,] *args, **kwargs):  
    "函数_文档字符串"  
    function_suite  
    return [expression]
```

- 加了星号 (*) 的变量args会存放所有未命名的变量参数，args为元组；
- 加**的变量kwargs会存放命名参数，即形如key=value的参数，kwargs为字典。

不定长参数

下面代码的运行结果是怎样的呢？

```
def test(a,b,*args):  
    print(a)  
    print(b)  
    print(args)  
test(11,22)
```

```
def test(a,b,*args):  
    print(a)  
    print(b)  
    print(args)  
test(11,22,33,44,55,66,77,88,99)
```

过渡页



01 什么是函数

02 函数的定义和调用

03 函数的参数

04 函数的返回值

05 函数的四种类型

函数的返回值

函数的返回值是使用`return`语句来完成的。

```
def add2num(a, b):  
    c = a+b  
    return c
```

函数`add2num`中包含`return`,
意味着这个函数有一个返回值,
其返回值就是`a`和`b`相加的结果。

过渡页



01 什么是函数

02 函数的定义和调用

03 函数的参数

04 函数的返回值

05 函数的四种类型

函数的四种类型

根据函数的参数和返回值，函数可以分为四种类型：

- 无参数，无返回值的函数
- 无参数，有返回值的函数
- 有参数，无返回值的函数
- 有参数，有返回值的函数

无参数，无返回值的函数

```
def printMenu():  
    print('-----')  
    print('    xx涮涮锅 点菜系统')  
    print(' 1. 羊肉涮涮锅')  
    print(' 2. 牛肉涮涮锅')  
    print(' 3. 猪肉涮涮锅')  
    print('-----')  
printMenu()
```

```
-----  
    xx涮涮锅 点菜系统  
 1. 羊肉涮涮锅  
 2. 牛肉涮涮锅  
 3. 猪肉涮涮锅  
-----
```

无参数，有返回值的函数

```
# 获取温度
```

```
def getTemperature():
```

```
    #这里是获取温度的一些处理过程
```

```
    #为了简单起见，先模拟返回一个数据
```

```
    return 24
```

```
temperature = getTemperature()
```

```
print('当前的温度为:',temperature)
```



当前的温度为： 24

有参数，无返回值的函数

该类函数使用较少，了解即可。

```
def test(num1,num2):  
    result=num1+num2  
    print('计算结果为： %d'%result)
```

有参数，有返回值的函数

```
def calculateNum(num):  
    result = 0  
    i = 1  
    while i<=num:  
        result = result + i  
        i+=1  
    return result  
result = calculateNum(100)  
print('1~100的累积和为:', result)
```

像数据处理并需要结果的应用，可以使用此类函数。



过渡页



06 函数的嵌套调用

07 变量的作用域

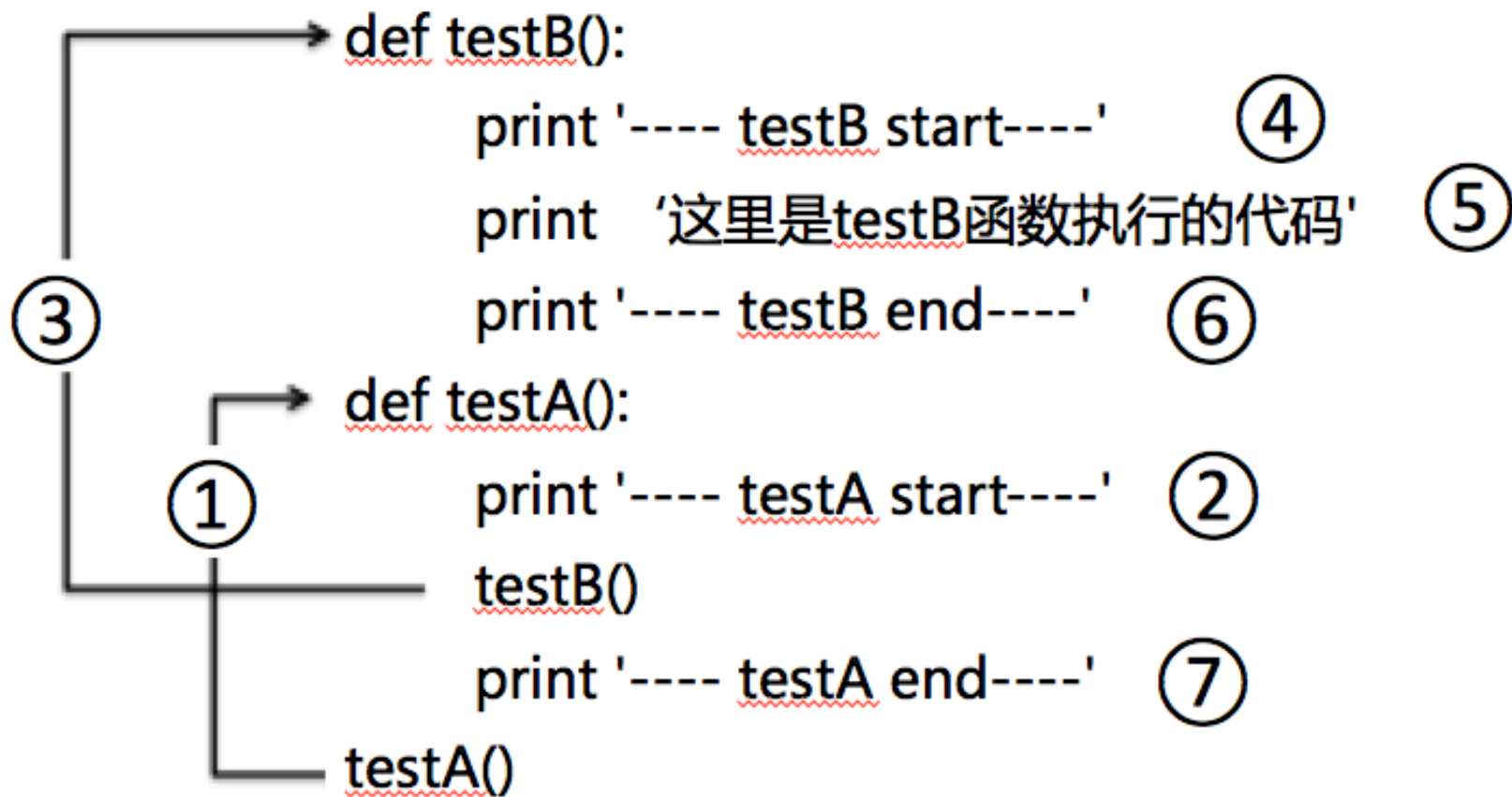
08 递归函数和匿名函数

09 日期时间函数

10 随机数函数

函数的嵌套调用

示例



过渡页



06 函数的嵌套调用

07 变量的作用域

08 递归函数和匿名函数

09 日期时间函数

10 随机数函数

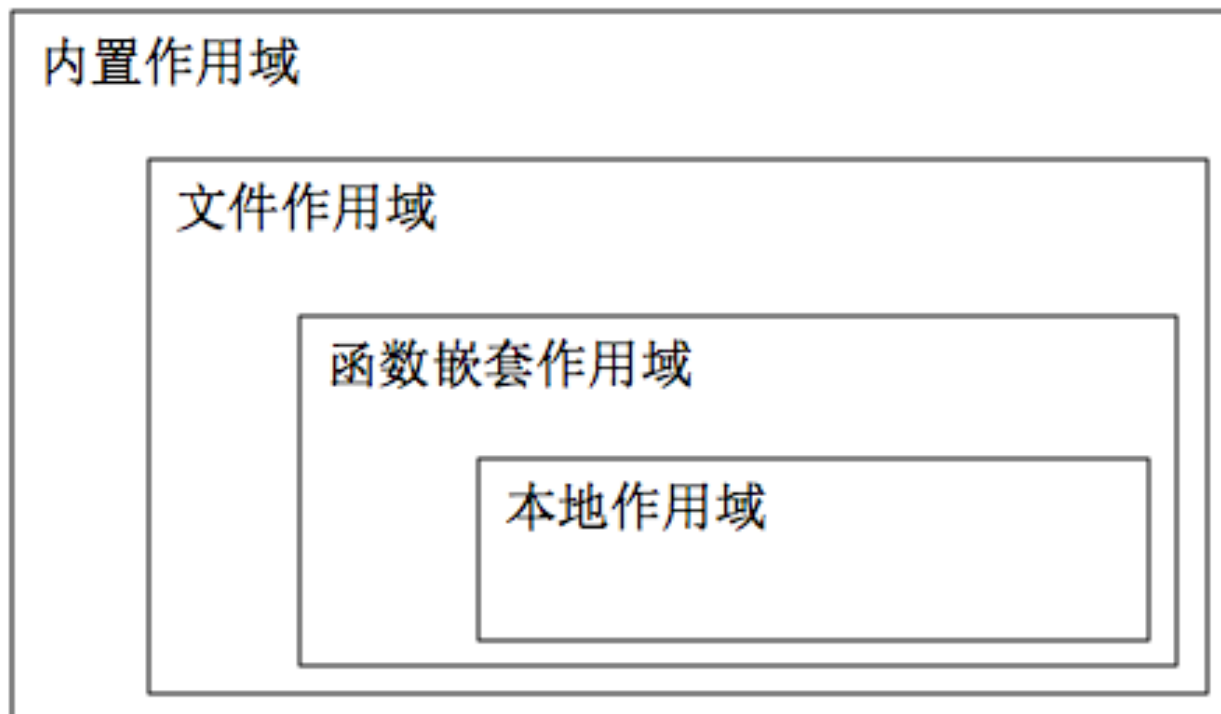
作用域

- 为了避免变量的名称发生冲突，所以Python引入了命名空间的概念。
- 命名空间指的是名称到对象的映射，类似于字典，键名是变量的名字，值是变量的值。

作用域

命名空间是相互独立存在的，而且它们被安排在某个特定的层次，把这些层次结构组合起来就是**作用域**。

- 本地作用域
- 函数嵌套作用域
- 文件作用域
- 内置作用域



局部变量

- 所谓**局部变量**，就是在函数内部定义的变量。
- **局部变量的作用域是函数内部**，意味着它只在定义它的函数中有效，一旦函数结束就会消失。

全局变量

- 定义在函数外的拥有全局作用域。
- 全局变量可以在整个程序范围内访问。
- 如果出现全局变量和局部变量名字相同的情况，则在函数中访问的是局部变量。

过渡页



06 函数的嵌套调用

07 变量的作用域

08 递归函数和匿名函数

09 日期时间函数

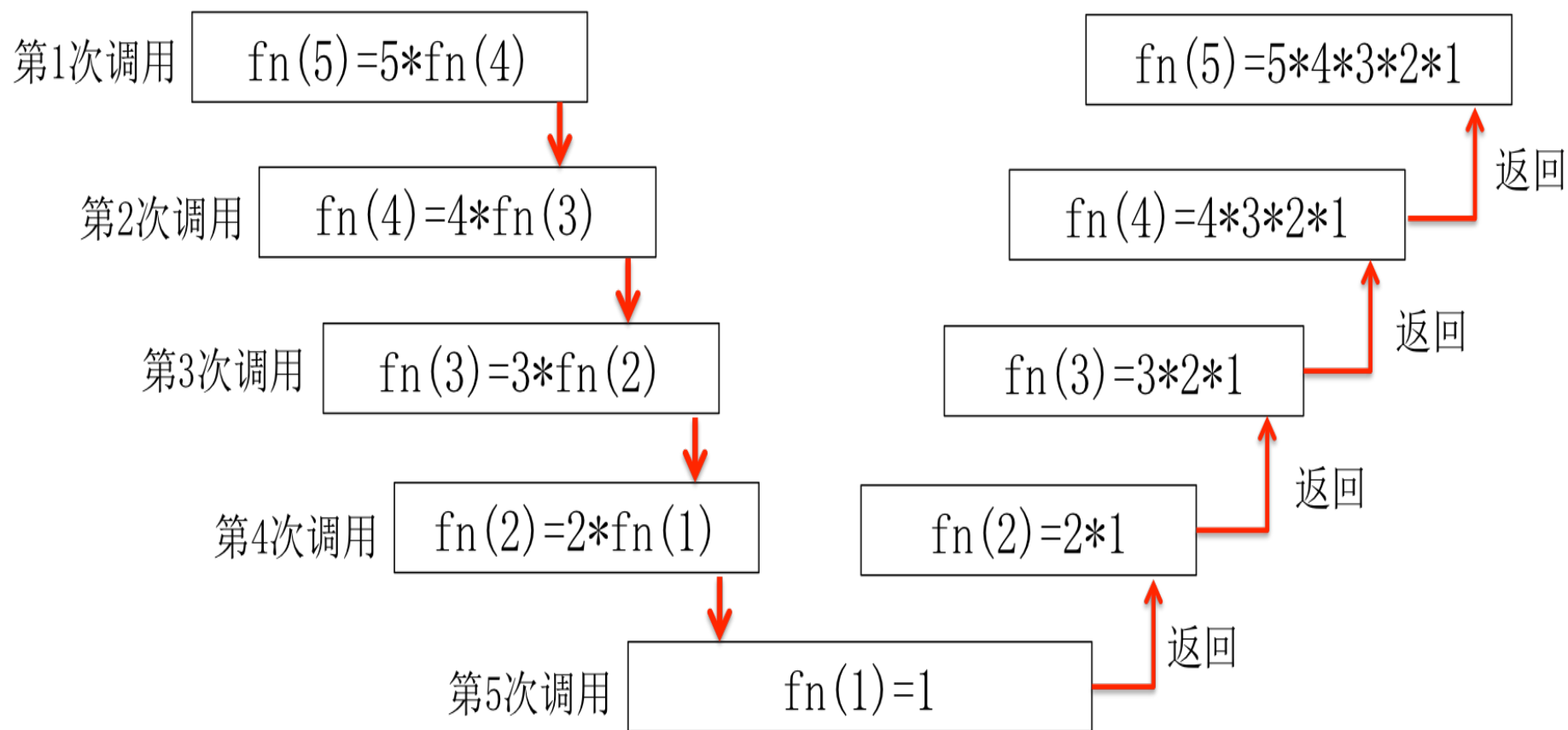
10 随机数函数

递归函数

一个函数的内部可以调用其他函数。但是，如果一个函数在内部不调用其它的函数，而是自己本身的话，这个函数就是递归函数。

递归函数

使用递归，实现阶乘 $n! = 1 * 2 * 3 * \dots * n$ 的计算。



递归函数

使用递归，实现阶乘 $n! = 1 * 2 * 3 * \dots * n$ 的计算。

```
def fact(n):  
    if n==1:  
        return 1  
    return n * fact(n - 1)
```

Fact(5)

120

$5*4*3*2*1$

匿名函数

匿名函数就是没有名称的函数，也就是不再使用def语句定义的函数。如果要声明匿名函数，则需要使用lambda关键字，匿名函数的声明格式如下所示：

```
lambda [arg1 [,arg2,....argn]]:expression
```

匿名函数

匿名函数示例代码：

```
sum = lambda arg1, arg2: arg1 + arg2  
print("运行结果： ", sum( 10, 20 ))  
print("运行结果： ", sum( 20, 20 ))
```

匿名函数

注意：

使用Lambda声明的匿名函数能接收任何数量的参数，但只能返回一个表达式的值。匿名函数不能直接调用print，因为lambda需要一个表达式。

过渡页



06 函数的嵌套调用

07 变量的作用域

08 递归函数和匿名函数

09 日期时间函数

10 随机数函数

时间函数

在Python中，通常有如下几种方式表示时间：

- (1) 时间戳；
- (2) 格式化的时间字符串；
- (3) 时间元组 (`struct_time`) 。

时间戳

通常来讲，**时间戳**表示的是从**1970年1月1日00:00:00**开始按秒计算的**偏移量**。

```
import time; # 引入time模块  
ticks = time.time()  
print("当前时间戳为:", ticks)
```



格式化的时间字符串

我们可以使用`time`模块的`strftime`方法来格式化日期。

```
import time  
  
# 格式化成2016-03-20 11:45:39形式  
print(time.strftime("%Y-%m-%d %H:%M:%S", time.localtime()))
```

格式化的时间字符串

格式化符号	含义
%y	两位数的年份表示 (00-99)
%Y	四位数的年份表示 (000-9999)
%m	月份 (01-12)
%d	月内中的一天

格式化的时间字符串

格式化符号	含义
%H	24小时小时数 (0-23)
%I	12小时制小时数 (01-12)
%M	分钟数 (00-59)
%S	秒 (00-59)

时间元组

返回struct_time的函数主要有gmtime()、localtime()和strptime(), struct_time元组。

序号	字段	值
0	tm_year	2008
1	tm_mon	1到12
2	tm_mday	1到31
3	tm_hour	0到23

序号	字段	值
4	tm_min	0到59
5	tm_sec	0到61
6	tm_wday	0到6
7	tm_yday	1到366
8	tm_isdst	-1, 0, 1, -1是决定是否夏令时的旗帜

时间元组

- `time.altzone`

返回格林威治西部的夏令时地区的偏移秒数。如果该地区在格林威治东部会返回负值（如西欧，包括英国）。对夏令时启用地区才能使用。

时间元组

- `time.asctime([tupletime])`

接受时间元组并返回一个可读的形式为"Tue Dec 11 18:07:14 2008" (2008年12月11日 周二18时07分14秒) 的24个字符的字符串。

时间元组

- `time.asctime([tupletime])`

接受时间元组并返回一个可读的形式为"Tue Dec 11 18:07:14 2008" (2008年12月11日 周二18时07分14秒) 的24个字符的字符串。

时间元组

- `time.clock()`

用以浮点数计算的秒数返回当前的CPU时间。用来衡量不同程序的耗时，比`time.time()`更有用。

- `time.ctime([secs])`

作用相当于`asctime(localtime(secs))`，未给参数相当于`asctime()`。

时间元组

- `time.gmtime([secs])`

接收时间辏（1970纪元后经过的浮点秒数）并返回格林威治天文时间下的时间元组t。

时间元组

- `time.localtime([secs])`

接收时间辍（1970纪元后经过的浮点秒数）并返回
当地时间下的时间元组t（t.tm_isdst可取0或1，
取决于当地当时是不是夏令时）

时间元组

- `time.localtime([secs])`

接收时间辏（1970纪元后经过的浮点秒数）并返回
当地时间下的时间元组t

- `time.mktime(tupletime)`

接受时间元组并返回时间辏

时间元组

- `time.sleep(secs)`

推迟调用线程的运行，secs指秒数。

- `time.strftime(fmt[,tupletime])`

接收以时间元组，并返回以可读字符串表示的当地时间，格式由fmt决定。

时间元组

- `time.strptime(str,fmt='%a %b %d %H:%M:%S %Y')`

根据fmt的格式把一个时间字符串解析为时间元组。

时间元组

- `time.time()`

返回当前时间的时间戳

- `time.tzset()`

根据环境变量TZ重新初始化时间相关设置。

时间元组

- `time.timezone`

表示当地时区（未启动夏令时）距离格林威治的偏移秒数（ >0 ，美洲； ≤ 0 大部分欧洲，亚洲，非洲）

时间元组

- `time.tzname`

包含一对根据情况的不同而不同的字符串，分别是带夏令时的本地时区名称，和不带的。

日历函数

- `calendar.calendar(year,w=2,l=1,c=6)`

返回一个多行字符串格式的`year`年年历，3个月一行，间隔距离为`c`。每日宽度间隔为`w`字符。
每行长度为 $21 * W + 18 + 2 * C$ 。`l`是每星期行数。

日历函数

- **calendar.firstweekday** ()

返回当前每周起始日期的设置。默认情况下，首次载入calendar模块时返回0，即星期一。

- **calendar.isleap(year)**

如果是闰年返回True，否则为false。

日历函数

- **calendar.leapdays(y1,y2)**

返回在Y1， Y2两年之间的闰年总数。

- **calendar.month(year,month,w=2,l=1)**

返回一个多行字符串格式的year年month月日历，
两行标题，一周一行。每日宽度间隔为w字符。
每行的长度为 $7 * w + 6$ 。l是每星期的行数。

日历函数

- **calendar.monthcalendar(year,month)**

返回一个整数的单层嵌套列表。每个子列表装载代表一个星期的整数。Year年month月外的日期都设为0;范围内的日子都由该月第几日表示,从1开始。

日历函数

- **calendar.monthrange(year, month)**

返回两个整数。第一个是该月的星期几的日期码，第二个是该月的日期码。日从0（星期一）到6（星期日）；月从1到12。

- **calendar.prcal(year, w=2, l=1, c=6)**

相当于 `print(calendar.calendar(year, w, l, c))`

日历函数

- **calendar.setfirstweekday(weekday)**

设置每周的起始日期码。0（星期一）到6（星期日）

- **calendar.prmonth(year,month,w=2,l=1)**

相当于 `print(calendar.calendar(year, w, l, c))`

日历函数

- **calendar.timegm(tupletime)**

和`time.gmtime`相反：接受一个时间元组形式，返回该时刻的时间戳（1970纪元后经过的浮点秒数）

- **calendar.weekday(year,month,day)**

返回给定日期的日期码。0（星期一）到6（星期日）。月份为 1（一月） 到 12（12月）。

过渡页



06 函数的嵌套调用

07 变量的作用域

08 递归函数和匿名函数

09 日期时间函数

10 随机数函数

随机数函数

- `random.random()`

用于生成一个0到1的随机浮点数: $0 \leq n < 1.0$ 。

```
import random
# 生成第一个随机数
print("random():", random.random())
# 生成第二个随机数
print("random():", random.random())
```

随机数函数

- `random.uniform(a,b)`

返回a,b之间的随机浮点数，范围[a,b]或[a,b]取决于四舍五入，a不一定要比b小。

```
import random  
print("random:",random.uniform(50,100))  
print("random:",random.uniform(100,50))
```

随机数函数

- `random.randint(a,b)`

返回a,b之间的整数，范围[a,b]，注意：传入参数必须是整数，a一定要比b小。

```
import random
#生成的随机数n: 12 <= n <= 20
print(random.randint(12,20))
```

随机数函数

- `random.randrange([start], stop[, step])`

返回有个区间内的整数，可以设置step。只能传入整数，`random.randrange(10, 100, 2)`，结果相当于从`[10, 12, 14, 16, ... 96, 98]`序列中获取一个随机数。

随机数函数

- `random.choice(sequence)`

从sequence（序列，是有序类型的）中随机获取一个元素，列表、元组、字符串都属于sequence。

```
random.randrange(10,100,2)
```

结果等效于

```
random.choice(range(10,100,2))
```


随机数函数

- `random.shuffle(x[,random])`

用于将列表中的元素打乱顺序，俗称为洗牌。

```
p = ["Python", "is", "powerful", "simple"]  
random.shuffle(p)
```

随机数函数

- `random.sample(sequence,k)`

从指定序列中随机获取k个元素作为一个片段返回，
sample函数不会修改原有序列

```
list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
slice = random.sample(list, 5)  
print(slice)  
print(list)
```

本章小结

本章主要针对函数进行了讲解，包括函数的定义调用、函数的参数、函数的返回值、函数的嵌套、递归函数、匿名函数、日期时间函数和随机数函数。函数作为关联功能的代码段，可以很好的提高应用的模块性，希望读者能用好这些函数，并学会查询相关的函数手册。

Python3

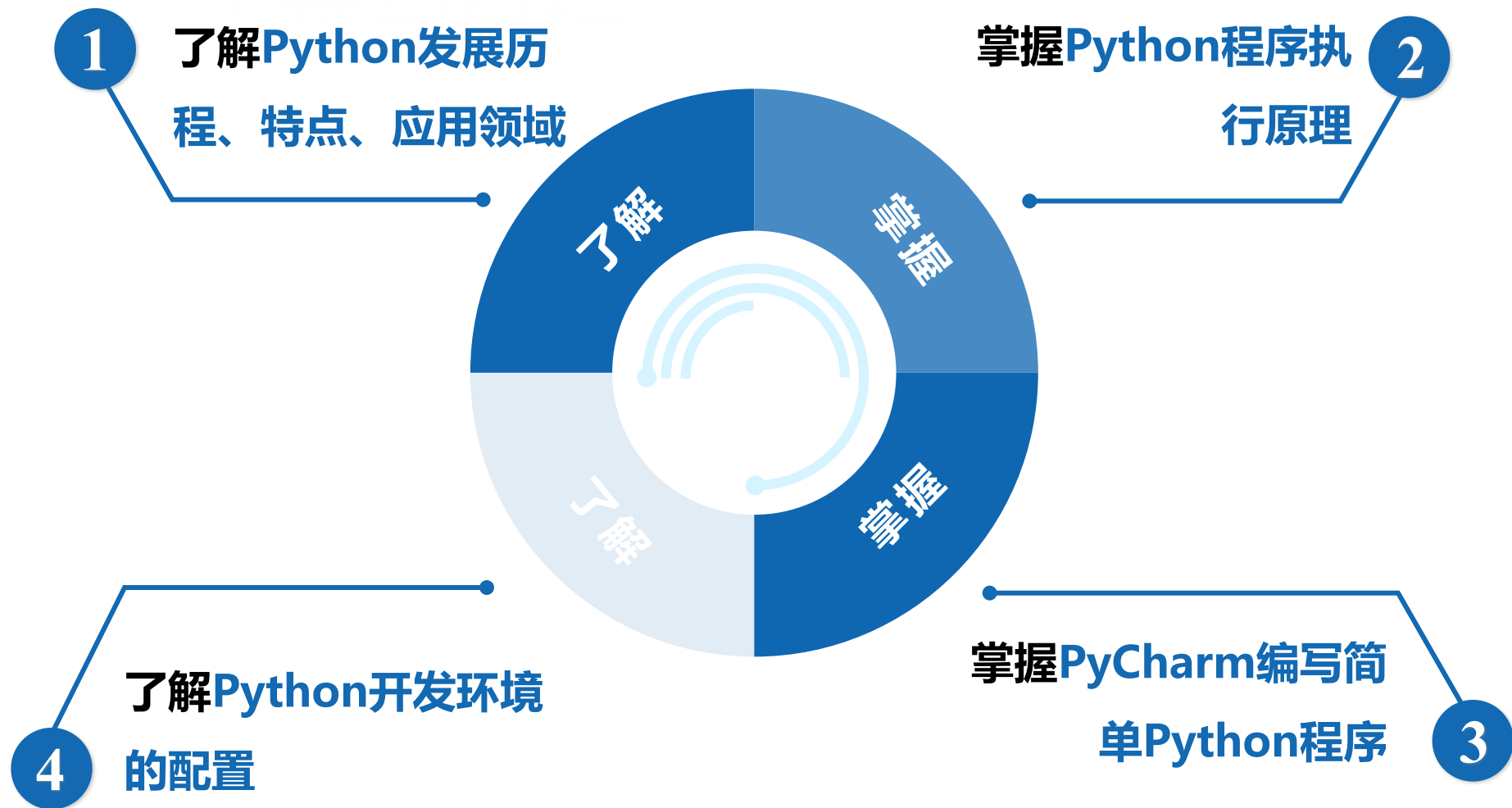
基础教程

第7章 高级函数

- Python发展历程
- Python开发环境的配置
- Python特点和应用领域
- Python程序执行原理



学习目标



目录页



01 闭包

02 装饰器

03 常见Python内置函数

过渡页



01 闭包

02 装饰器

03 常见Python内置函数

闭包

Python函数是支持嵌套的。如果在一个内部函数中对外部函数作用域（非全局作用域）的变量进行引用，那么内部函数就会被称为闭包。闭包需要满足如下3个条件：

- 1.存在于两个嵌套关系的函数中，并且闭包是内部函数；
- 2.内部函数引用了外部函数的变量（自由变量）；
- 3.外部函数会把内部函数的函数名称返回。

过渡页



01 闭包

02 装饰器

03 常见Python内置函数

装饰器的概念

假设我们已经开发了一个本有的函数，后续可能会增加临时的需求，例如插入日志，我们可以增加一个包裹函数，由它来负责这些额外的需求，这个包裹函数就是**装饰器**。

装饰器的概念

装饰器主要应用在如下场景：

- 引入日志；
- 函数执行时间统计；
- 执行函数前预备处理；
- 执行函数后清理功能；
- 权限校验；
- 缓存。

装饰器的概念

装饰器是一个函数，它需要接收一个参数，该参数表示被修饰的函数。例如，有如下一个装饰器函数：

```
•def myDectoration(func):  
•    def inner():  
•        print("正在执行内部函数")  
•        func()  
•    return inner  
  
•def printMessage():  
•    print("-----欢迎您-----")  
•pm = myDectoration(printMessage)  
•pm()
```

- 装饰器是个嵌套函数
- 内部函数是一个闭包。
- 外部函数接收的是被修饰的
- 函数（func）

装饰器的概念

通过在函数定义的前面添加@符号和装饰器名，实现装饰器对函数的包装。给f1函数加上装饰器，示例如下：

```
@w1  
def f1():  
    print('f1')
```

此时，程序会自动编译生成调用装饰器函数的代码，等价于：

```
f1 = w1(f1)
```

多个装饰器

多个装饰器应用在一个函数上，调用顺序是**从下至上**。

```
@w1  
@w2  
def f1():  
    print('---f1---')
```

执行顺序：

先执行@w2，后执行@w1

装饰有参数的函数

看看下面的代码，运行结果是什么呢？

```
def w1(func):  
    def inner(a,b):  
        print('开始验证权限')  
        func(a,b)  
    return inner
```

```
@w1  
def test(a,b):  
    print('a=%d,b=%d'%(a,b))  
test(1,2)
```


带有返回值的装饰器

示例

```
def func(functionName):  
    def func_in():  
        return functionName  
    return func_in  
  
@func  
def test():  
    return 'itheima'
```

带有参数的装饰器

如果给装饰器添加参数，需要增加一层包装，先传递参数，然后再传递函数名。示例代码如下：

```
def func_arg(args):  
    def func(functionName):  
        def func_in():  
            print('——记录日志-args=%s'%args)  
            functionName()  
        return func_in  
    return func
```

过渡页



01 闭包

02 装饰器

03 常见Python内置函数

map函数

map函数会根据提供的函数对指定的序列**做映射**。

map函数的定义如下：

```
map(function, iterable,...)
```

第1个参数是函数的名称；第2个参数表示支持迭代的容器或者迭代器。

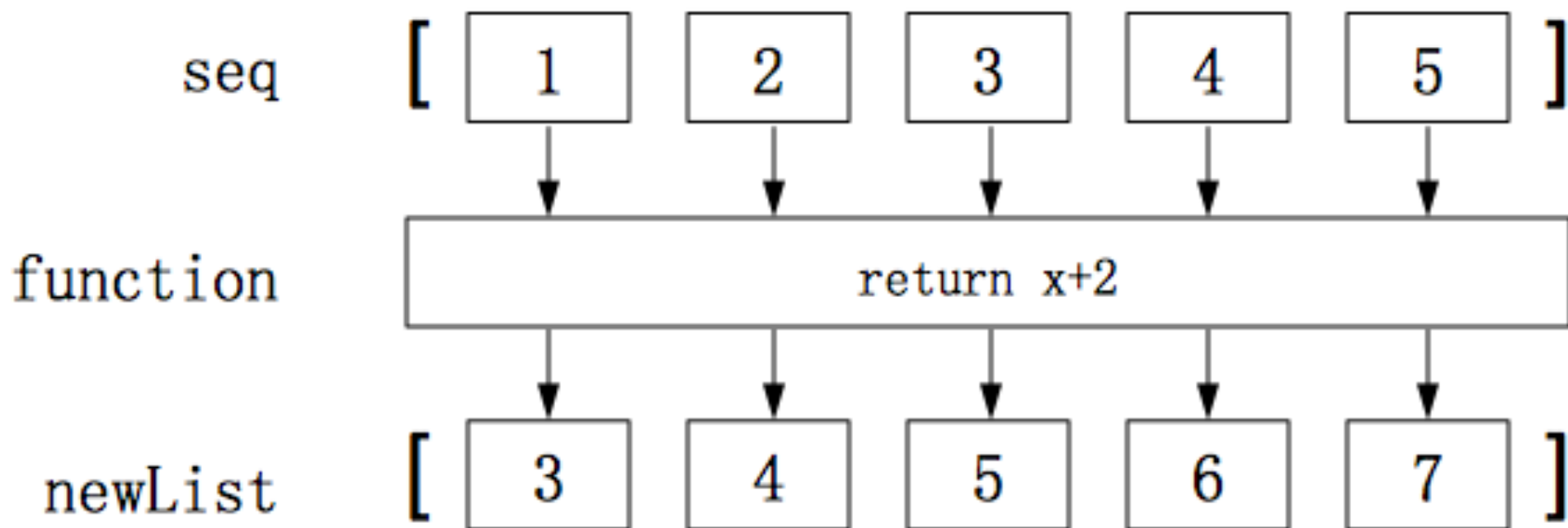
map函数

map函数的作用是以参数序列中的每个元素分别调用function函数，把每次调用后返回的结果保存为对象。

```
func = lambda x:x+2  
result = map(func, [1,2,3,4,5])  
print(list(result))
```

map函数

执行过程



filter函数

filter函数会对指定序列执行过滤操作。

filter函数的定义如下：

```
filter(function, iterable)
```

第1个参数可以是函数的名称；第2个参数表示的是序列、支持迭代的容器或迭代器。

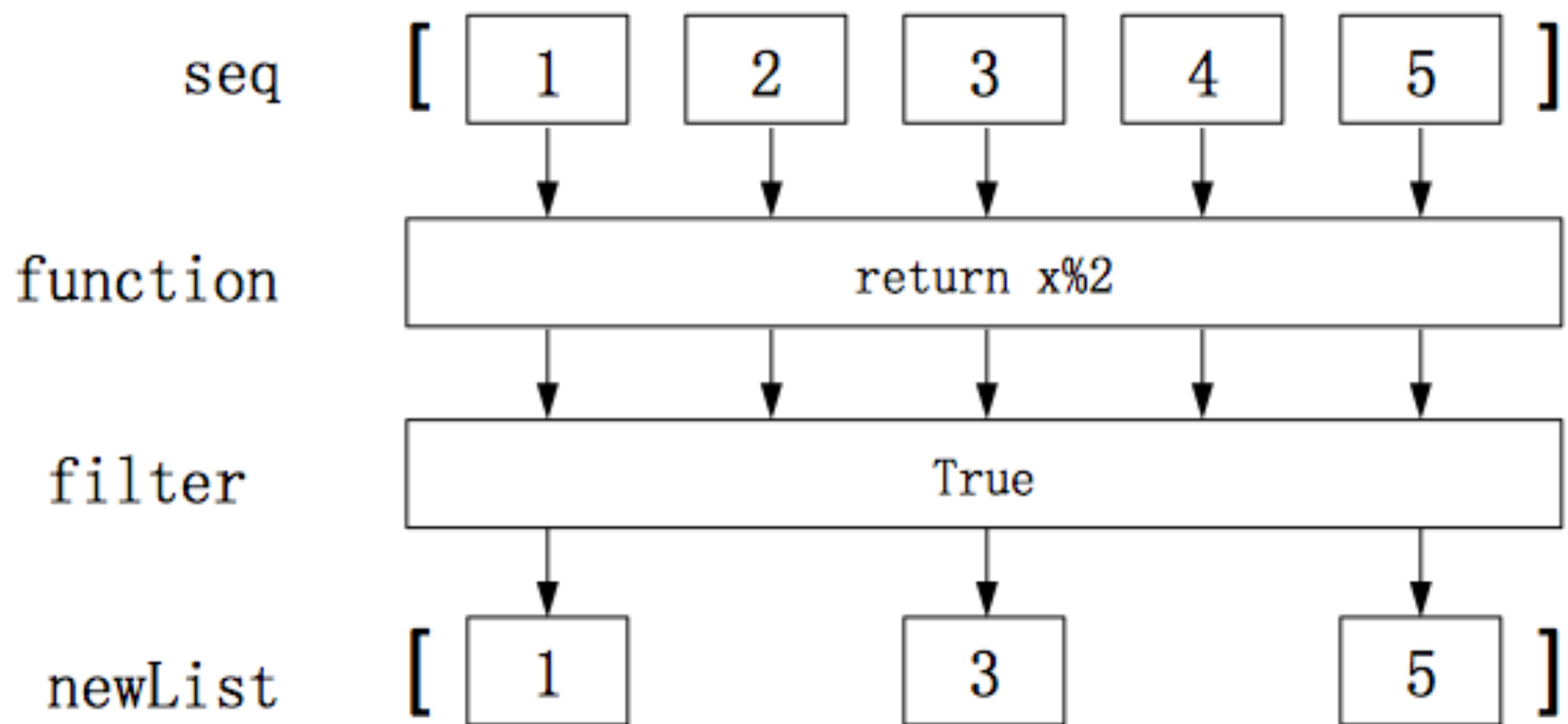
filter函数

示例代码

```
func = lambda x:x%2  
result = filter(func, [1, 2, 3, 4, 5])  
print(list(result))
```


filter函数

执行过程



本章小结

- 本章继续带大家探讨了函数的高级内容，包括闭包、装饰器，以及与序列相关的3个内置函数。通过对本章内容的学习，大家应该对函数有了更深地理解，并能够熟练地应用到开发中。

Python3

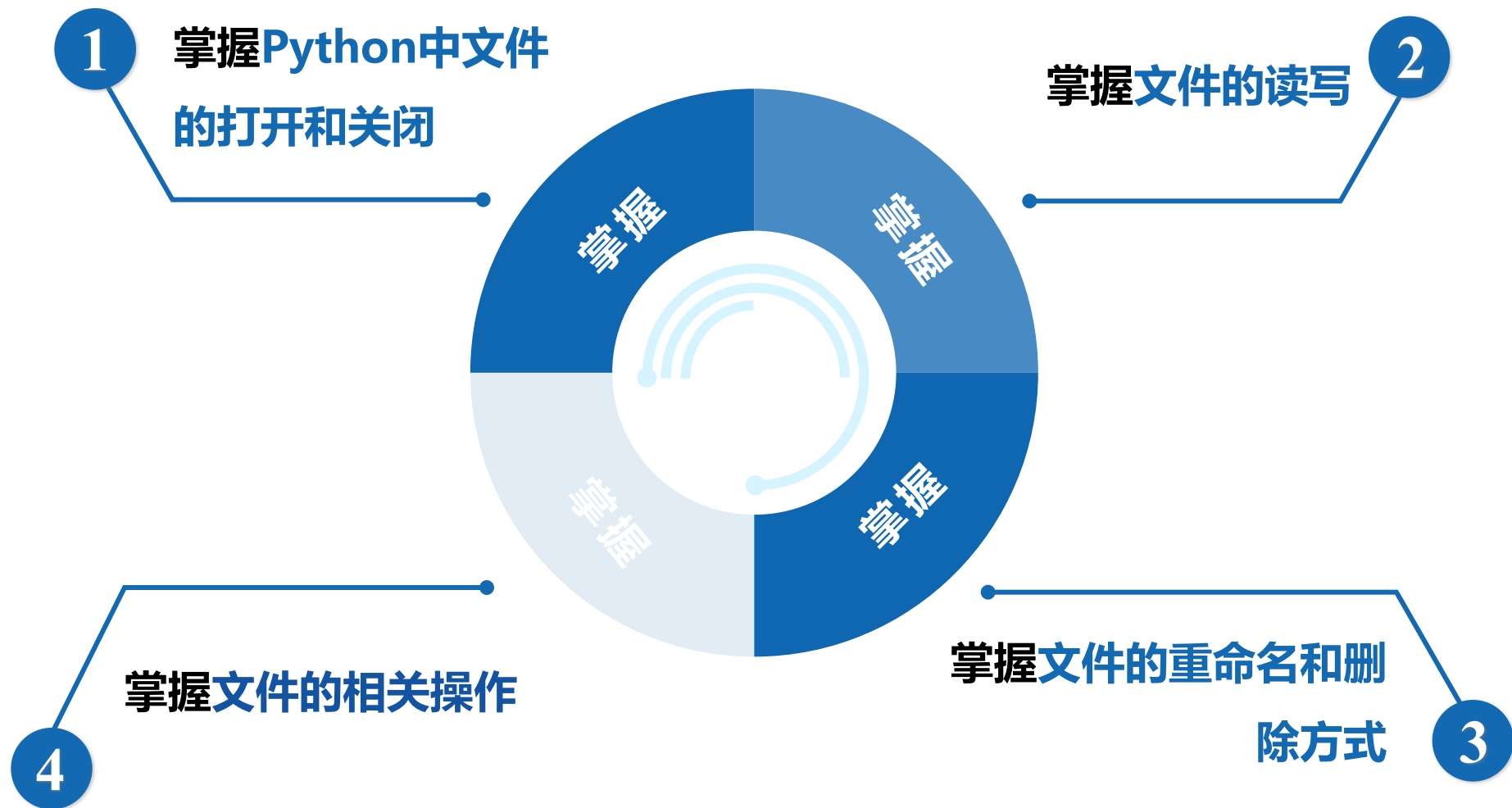
基础教程

第8章 文件操作

- 文件的打开和关闭
- 文件的相关操作
- 文件的读写
- 文件的重命名和删除



学习目标



目录页



- 01** 文件的打开和关闭
- 02** 文件的读写
- 03** 文件的重命名和删除
- 04** 文件的相关操作

过渡页



01 文件的打开和关闭

02 文件的读写

03 文件的重命名和删除

04 文件的相关操作

文件的打开

使用Word编写一份简历的**流程**：

- 打开新建一个Word文件
- 写入个人简历信息
- 保存文件
- 关闭Word软件



文件的打开

操作文件的整体过程和word写简历的过程类似:

- 打开或新建建立一个文件
- 读/写数据
- 关闭文件



文件的打开

在python中，使用open方法打开文件：

```
open(文件名，访问模式)
```

- “文件名”必须要填写
- “访问模式”是可选的

文件的打开

注意

如果使用`open`函数打开文件时，如果没有注明访问模式，则必须保证文件是存在的，否则会报异常。

```
Traceback (most recent call last):
```

```
File "<input>", line 1, in <module>
```

```
FileNotFoundError: [Errno 2] No such file or directory: 'test.txt'
```

文件的模式

访问模式	说明
r	默认模式。以只读方式打开文件，
w	打开一个文件只用于写入。
a	打开一个文件用于追加。
rb	以二进制格式打开一个文件用于只读。
wb	以二进制格式打开一个文件只用于写入。
ab	以二进制格式打开一个文件用于追加。

文件的模式

访问模式	说明
r +	打开一个文件用于读写, 文件指针会放在文件开头
w+	打开一个文件用于读写, 如已存在则覆盖
a+	打开一个文件用于读写, 如已存在, 则放末尾
rb+	以二进制格式打开一个文件用于读写
wb+	以二进制格式打开一个文件用于读写。如存在则覆盖
ab+	以二进制格式打开一个文件用于追加

文件的关闭

凡是打开的文件，切记要使用`close`方法关闭文件。

```
# 新建一个文件，文件名为:test.txt  
f = open(itheima.txt', 'w')  
# 关闭这个文件  
f.close()
```



过渡页



01 文件的打开和关闭

02 文件的读写

03 文件的重命名和删除

04 文件的相关操作

写文件

向文件写数据，需要使用**write**方法来完成，在操作某个文件时，每调用一次write方法，写入的数据就会追加到文件末尾。

```
f = open('itheima.txt', 'w')  
f.write('hello itheima, i am here!')  
f.close()
```



读文件

方式1：使用read方法读取文件

```
f = open('itheima.txt', 'r')
content = f.read(12)
print(content)
print("-"*30)
content = f.read()
print(content)
f.close()
```

运行结果

```
hello itheim
-----
a, i am here!
hello itheima, i am here!
hello itheima, i am here!
```


读文件

方式2：使用readlines方法读取文件

```
f= open('itheima.txt', 'r')
content = f.readlines()
i = 1
for temp in content:
    print("%d:%s" % (i, temp))
    i += 1
f.close()
```

运行结果

```
1:hello itheima, i am here!

2:hello itheima, i am here!

3:hello itheima, i am here!
```

读文件

方式3：使用readline方法一行一行读数据

```
f = open('itheima.txt', 'r')
content = f.readline()
print("1:%s"%content)
content = f.readline()
print("2:%s"%content)
f.close()
```

运行结果

```
1:hello itheima, i am here!

2:hello itheima, i am here!
```

文件的定位读写

方式1：使用tell方法来获取文件当前的读写位置

```
f = open("itheima.txt", "r")  
str = f.read(4)  
print("读取的数据是:", str)  
position = f.tell()  
print("当前文件位置:", position)
```

`tell()` 方法返回文件的当前位置，即文件指针当前位置。

文件的定位读写

方式2：使用seek方法来移动文件读取指针到指定位置

seek(offset, from)方法包含两个参数：

- offset:表示偏移量，也就是代表需要移动偏移的字节数
- from:表示方向，可以指定从哪个位置开始偏移
 - 0:表示文件开头（默认值）
 - 1:表示当前位置
 - 2:表示文件末尾

过渡页



01 文件的打开和关闭

02 文件的读写

03 文件的重命名和删除

04 文件的相关操作

文件的重命名

os模块中的`rename()`方法可以完成文件的重命名。

格式如下：

```
os.rename(需要修改的文件名, 新的文件名)
```

文件的删除

os模块中的`remove()`方法可以完成文件的删除操作。

格式如下：

```
os.remove(待删除的文件名)
```

过渡页



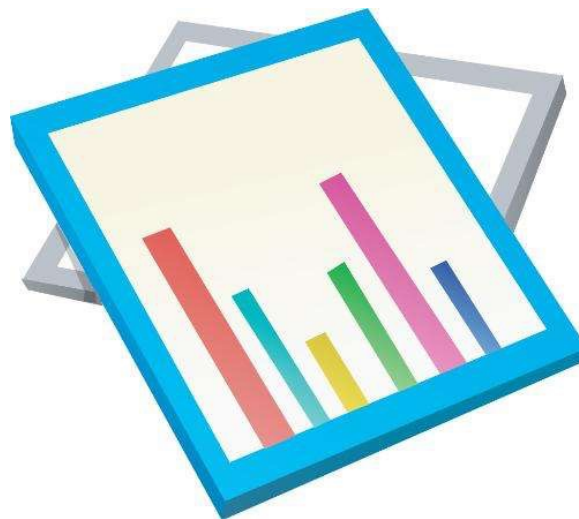
- 01** 文件的打开和关闭
- 02** 文件的读写
- 03** 文件的重命名和删除
- 04** 文件的相关操作

文件的相关操作

1. 创建文件夹

os模块的mkdir方法用来创建文件夹，示例如下：

```
import os  
os.mkdir("张三")
```

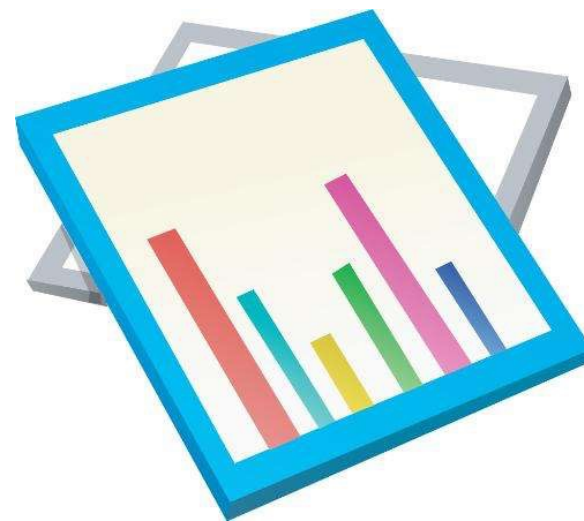


文件的相关操作

2. 获取当前目录

os模块的getcwd方法用来获取当前的目录，示例如下：

```
import os  
os.getcwd()
```

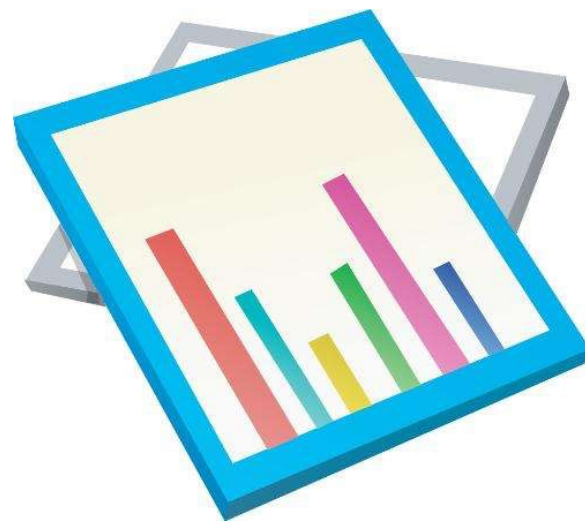


文件的相关操作

3. 改变默认目录

os模块的chdir方法用来改变默认目录，示例如下：

```
import os  
os.chdir("../")
```

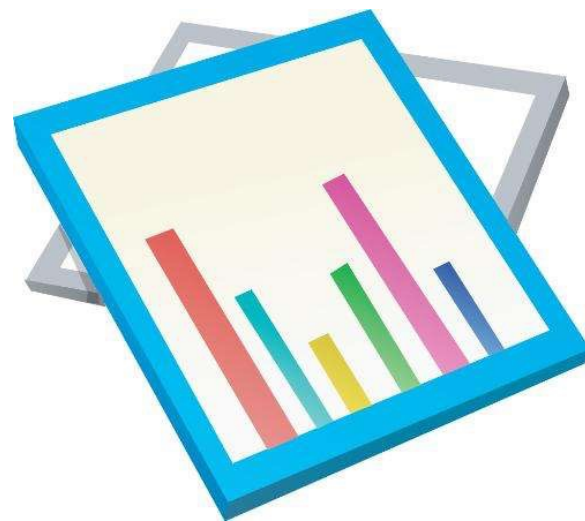


文件的相关操作

4. 获取目录列表

os模块的listdir方法用于获取目录列表，示例如下：

```
import os  
os.listdir("./")
```

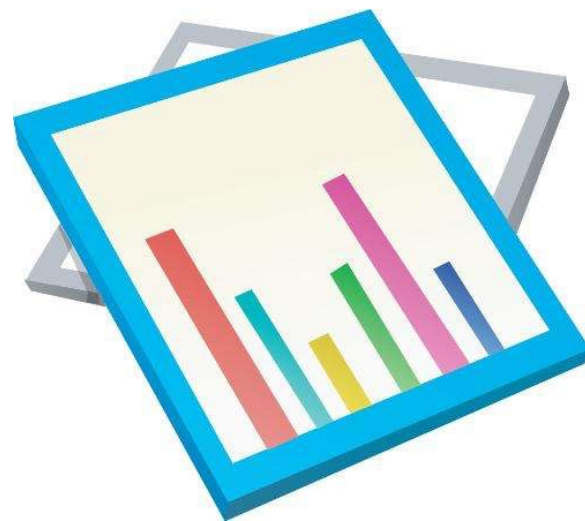


文件的相关操作

5. 删除文件夹

os模块的rmdir方法用于删除文件夹，示例如下：

```
import os  
os.rmdir ("张三")
```



本章小结

- 本章主要针对Python中的文件操作进行讲解，包括文件的打开关闭、文件的读写、文件的重命名、文件的删除等。通过本章的学习，希望大家可以认真掌握文件的相关操作，能够熟练使用相关方法实现功能。

第9章 异常

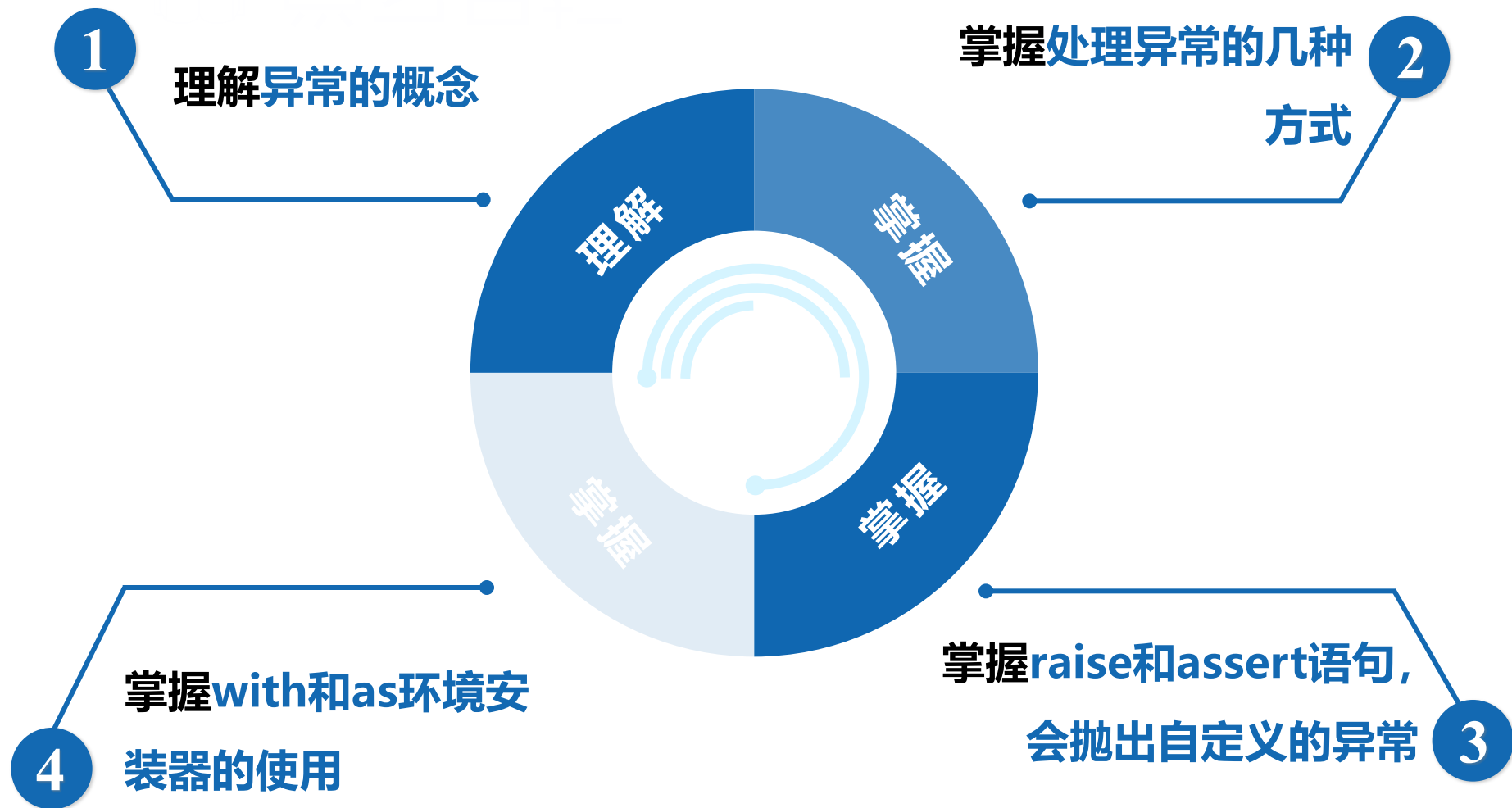
Python3

基础教程

- 异常类
- 异常处理
- with和as环境安装器
- 抛出异常
- 自定义异常



学习目标



目录页



- 01** 异常简介
- 02** 异常类
- 03** 异常处理
- 04** 抛出异常
- 05** 自定义异常
- 06** **with**和**as**环境安装器

过渡页



01 异常简介

02 异常类

03 异常处理

04 抛出异常

05 自定义异常

06 **with**和**as**环境安装器

异常介绍

在Python中，程序在执行的过程中产生的错误称为异常，比如列表索引越界、打开不存在的文件等。

```
print(a)  
open("123.txt","r")
```

这两行代码会报错吗？

异常介绍

报错信息：

`NameError: name 'a' is not defined`

`FileNotFoundError: [Errno 2] No such file or directory: '123.txt'`

- 第1个异常的类型为`NameError`（名称），描述信息为a没有定义；
- 第2个异常为`FileNotFoundError`，描述信息为没有找到123.txt文件

过渡页



01 异常简介

02 异常类

03 异常处理

04 抛出异常

05 自定义异常

06 **with**和**as**环境安装器

异常类

- 所有异常都是基类 **Exception** 的成员，它们都定义在 **exceptions** 模块中。
- 如果这个异常对象没有进行处理和捕捉，程序就会用所谓的回溯（`traceback`，一种错误信息）终止执行，这些信息包括错误的名称（例如 `NameError`）、原因和错误发生的行号。

异常类

1. NameError

尝试访问一个未声明的变量，会引发NameError。

```
Traceback (most recent call last):
```

```
File "D:/PythonCode/Chapter09/异常.py", line 1, in <module>
```

```
    print(foo)
```

```
NameError: name 'foo' is not defined
```

异常类

2. ZeroDivisionError

当除数为零的时候，会引发ZeroDivisionError异常。

Traceback (most recent call last):

File "D:/PythonCode/Chapter09/异常.py", line 1, in <module>

1/0

ZeroDivisionError: division by zero

异常类

3. SyntaxError

当解释器发现语法错误时，会引发SyntaxError异常

```
File "D:/PythonCode/Chapter09/异常.py", line 2
```

```
    for i in list
```

```
        ^
```

```
SyntaxError: invalid syntax
```

异常类

4. IndexError

当使用序列中不存在的索引时，会引发IndexError异常

```
Traceback (most recent call last):
```

```
File "D:/PythonCode/Chapter09/异常.py", line 2, in <module>
```

```
list[0]
```

```
IndexError: list index out of range
```

异常类

5. KeyError

当使用映射中不存在的键时，会引发KeyError异常。

Traceback (most recent call last):

File "D:/PythonCode/Chapter09/异常.py", line 2, in <module>

myDict['server']

KeyError: 'server'

异常类

6. FileNotFoundError

试图打开不存在的文件时，会引发FileNotFoundError

Traceback (most recent call last):

File "D:/PythonCode/Chapter09/异常.py", line 1, in <module>

f = open("test")

FileNotFoundError: [Errno 2] No such file or directory: 'test'

异常类

7. AttributeError

当尝试访问未知对象属性时，会引发AttributeError异常

Traceback (most recent call last):

File "D:/PythonCode/Chapter09/异常.py", line 6, in <module>

print(car.name)

AttributeError: 'Car' object has no attribute 'name'

过渡页



01 异常简介

02 异常类

03 异常处理

04 抛出异常

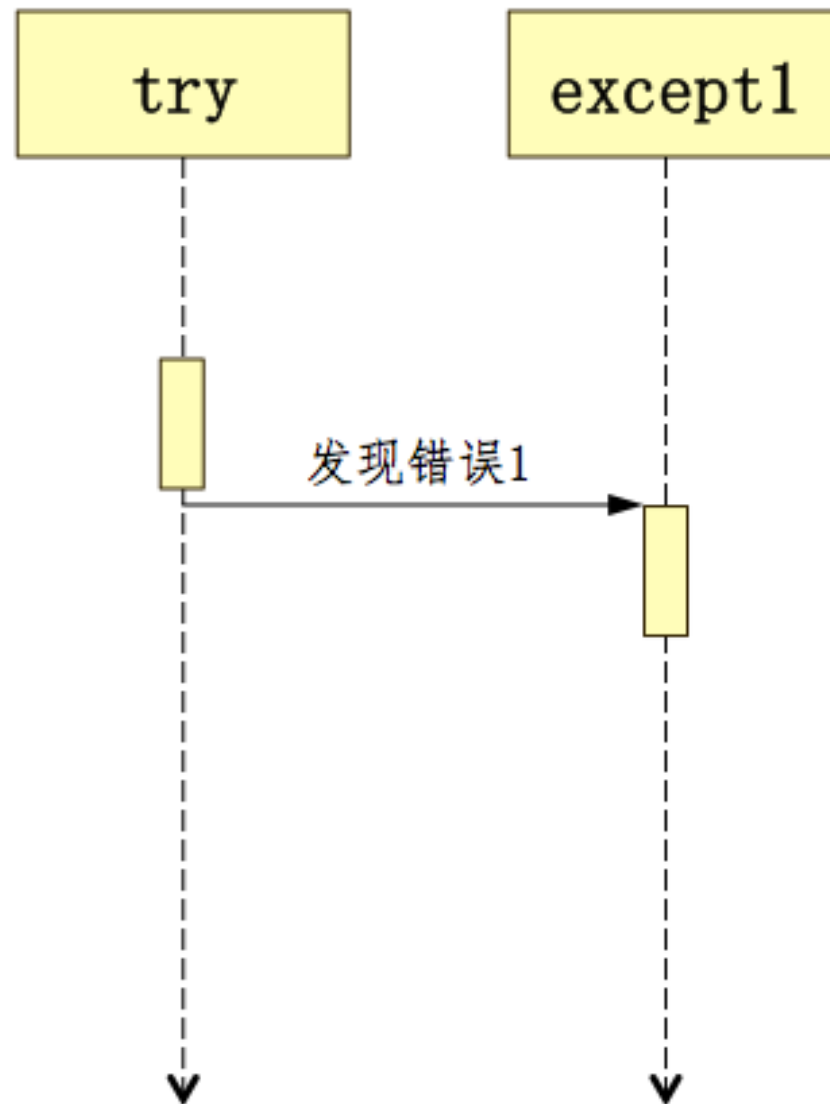
05 自定义异常

06 **with**和**as**环境安装器

捕获简单异常

try-except语句定义了监控异常的一段代码，并提供了处理异常的机制。

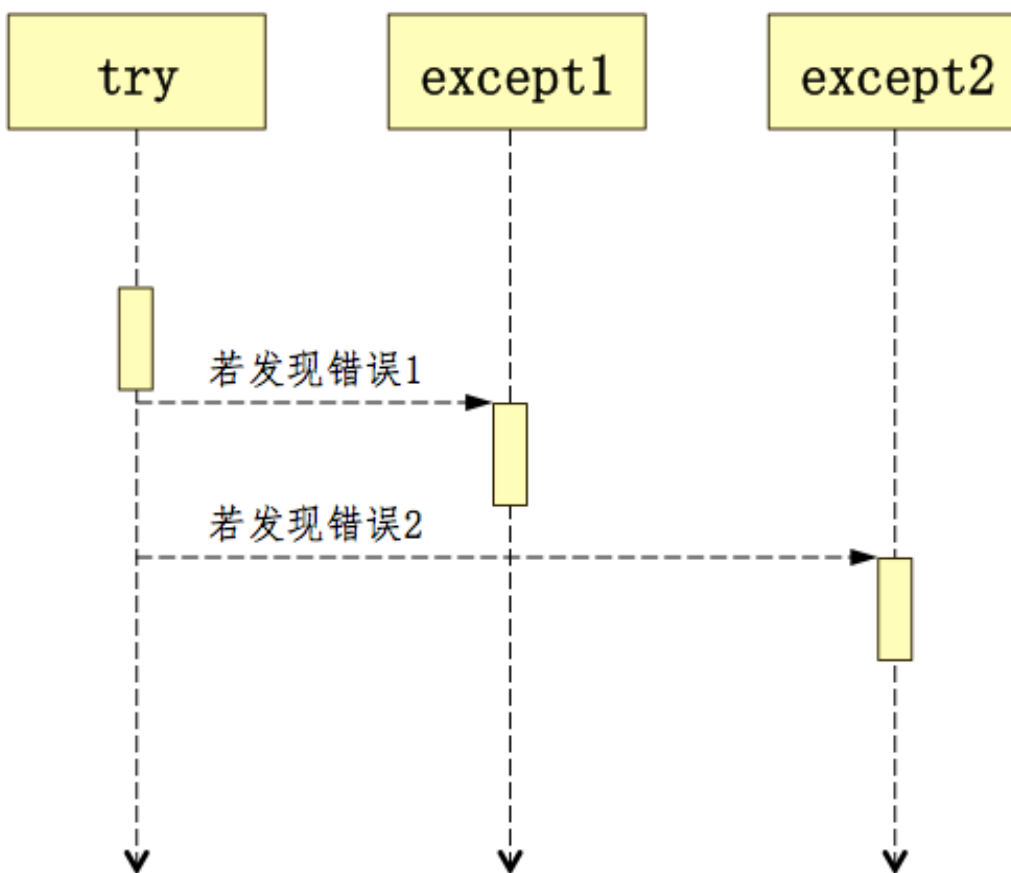
```
try:
    # 语句块
except:
    # 异常处理代码
```



捕获多个异常

处理多个异常的try-except语句格式如下：

```
try:  
    # 语句块  
except 异常名称1:  
    # 异常处理代码1  
except 异常名称2:  
    # 异常处理代码  
...
```



捕获异常的描述信息

当出现多种异常时，为了区分不同的错误信息，可以使用`as`获取系统反馈的信息。

```
# 获取描述信息
```

```
except (ZeroDivisionError, ValueError) as result:
```

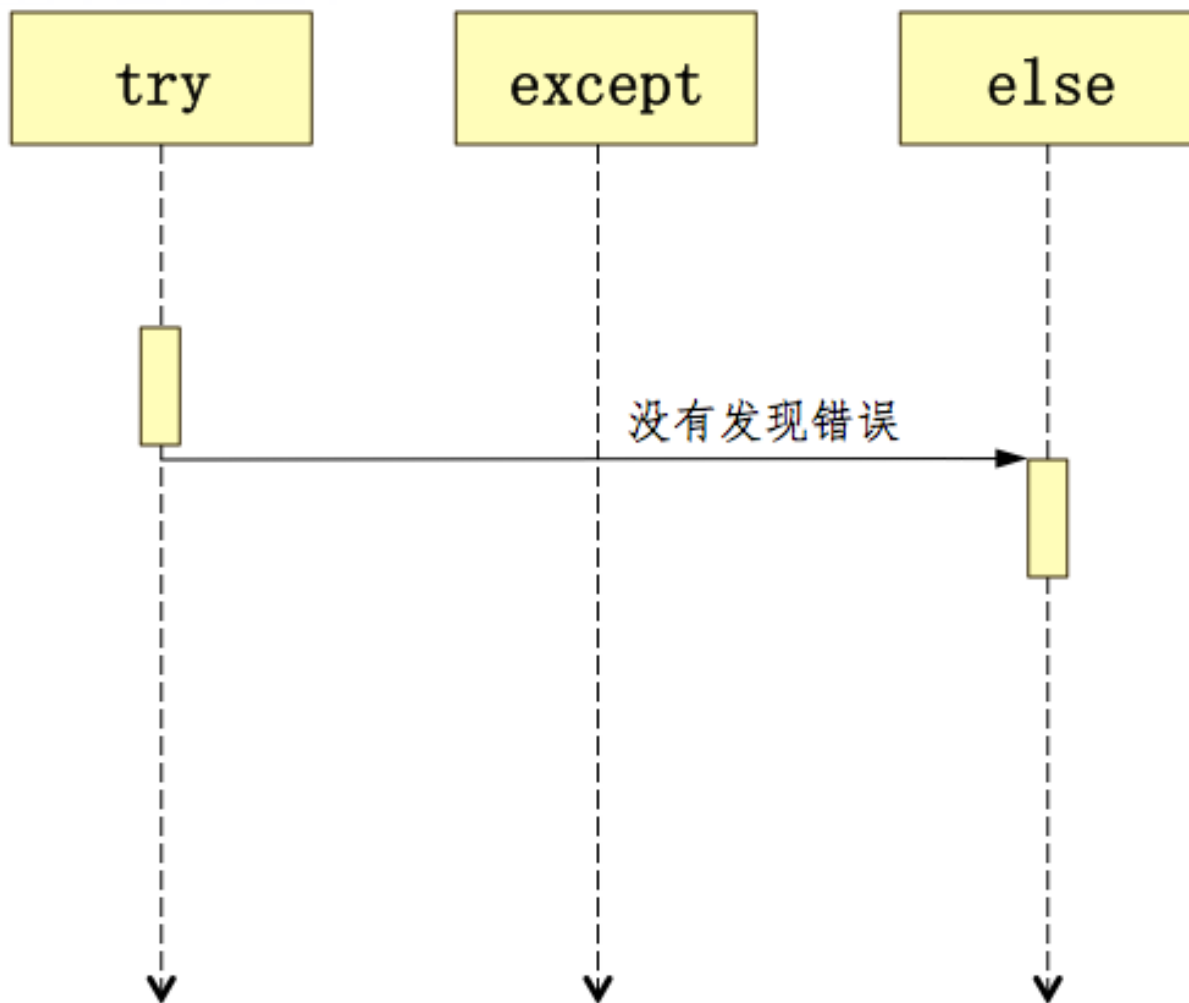
```
    print("捕捉到异常:%s"%result)
```

捕获所有的异常

当程序中出现大量异常时，捕获这些异常是非常麻烦的。这时，我们可以`在except子句中不指明异常的类型`，这样，不管发生何种类型的异常，都会执行except里面的处理代码。

没有捕获到异常(else)

如果try语句没有捕获到任何的错误信息，就不再执行任何except语句，而是会执行else语句。



终止行为 (finally)

在程序中，无论是否捕捉到异常，都必须要执行某件事情，例如关闭文件、释放锁等，这时可以提供 **finally** 语句处理。通常情况下，**finally** 用于释放资源。

过渡页



01 异常简介

02 异常类

03 异常处理

04 抛出异常

05 自定义异常

06 **with**和**as**环境安装器

raise语句

使用raise语句能显式地触发异常，格式如下：

- 1.raise 异常类名
 - 2.raise 异常类对象
 - 3.raise
- 引发指定异常类的实例
- 重新引发刚刚发生的异常

raise语句

1. 使用类名引发异常

当`raise`语句指定异常的类名时，会创建该类的实例对象，然后引发异常。

```
raise IndexError
```

Traceback (most recent call last):

File "D:/异常.py", line 1, in <module>

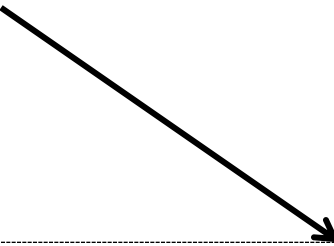
raise IndexError

IndexError

raise语句

2. 使用异常类的实例引发异常

```
index = IndexError()  
raise index
```



```
Traceback (most recent call last):  
  File "D:/异常.py", line 2, in <module>  
    raise index  
IndexError
```


raise语句

3. 传递异常

不带任何参数的`raise`语句，可以再次引发刚刚发生过的异常，作用就是向外传递异常。

```
try:  
    raise IndexError  
except:  
    print("出错了")  
    raise
```



出错了

```
File "D:/异常.py", line 2, in <module>  
    raise IndexError  
IndexError
```

raise语句

4. 指定异常的描述信息

```
raise IndexError("索引下标超出范围")
```



Traceback (most recent call last):

File "D:/异常.py", line 1, in <module>

raise IndexError("索引下标超出范围")

IndexError: 索引下标超出范围

raise语句

5. 异常引发异常

使用`raise... from...`可以在异常中抛出另外的异常。

```
try:
```

```
    num
```

```
except Exception as exception:
```

```
    raise IndexError("下标超出范围") from exception
```

try里面只定义了变量num，会引发NameError异常。except子句使用`raise... from...`抛出NameError异常后再抛出“下标越界”的异常。

assert语句

- `assert`语句又称作断言，指的是期望用户满足指定的条件。
- 当用户定义的约束条件不满足的时候，它会触发 `AssertionError` 异常，所以 `assert` 语句可以当做条件式的 `raise` 语句。

assert语句

assert语句格式如下：

assert 逻辑表达式, data



if not 逻辑表达式:
 raise AssertionError(data)

assert后面紧跟一个逻辑表达式，相当于条件。Data通常是一个字符串，当条件为false时作为异常的描述信息。

assert语句

断言的示例如下：

```
a = 0  
assert a!=0,"a的值不能为0"
```

Traceback (most recent call last):

File "D:/异常.py", line 2, in <module>

assert a!=0,"a的值不能为0"

AssertionError: a的值不能为0

过渡页



01 异常简介

02 异常类

03 异常处理

04 抛出异常

05 自定义异常

06 **with**和**as**环境安装器

自定义异常

- 创建一个继承`Exception`类的子类，就是自定义异常类。
- 当遇到自己设定的错误时，使用`raise`语句抛出自定义的异常。

过渡页



01 异常简介

02 异常类

03 异常处理

04 抛出异常

05 自定义异常

06 with和as环境安装器

with语句

现在有一个需求，打开foo.txt文本文件，读取完所有的数据以后关闭文件。 示例代码如下：

```
file = open("/tmp/foo.txt") # 打开文件  
data = file.read()         # 读取数据  
file.close()                # 关闭文件
```



文件读取出现问题怎么办？

忘记关闭文件出现问题怎么办？

with语句

为了避免在文件读取的过程中产生这些问题，可以在上述示例中增加处理异常的语句，加强版本如下：

```
file = open("/tmp/foo.txt")  
try:  
    data = file.read()  
finally:  
    file.close()
```

该代码虽然解决了产生异常的可能，但是这段代码过于冗长。此时，在示例中使用with语句处理上下文环境产生的异常，具体如下。

```
with open("/tmp/foo.txt") as file:  
    data = file.read()
```

with语句

Python 2.5开始，引入了with语句，**with语句适用于对资源进行访问的场合**，确保不管使用过程中是否发生异常都会执行必要的“清理”操作，释放资源。

with语句

with语句格式

```
with context_expr [as var]:  
    with_body
```

- context_expr: 需要返回一个上下文管理器对象, 该对象并不赋值给as子句中的 var。
- var: 可以是变量或者元组。
- with_body: with语句包裹的代码块。

with语句

with语句执行过程：

- (1) 执行context_expr, 生成上下文管理器context_manager;
- (2) 调用上下文管理器的__enter__()方法, 如果使用了as子句, 就把__enter__()方法的返回值赋值给 as 子句中的var;
- (3) 执行语句体with_body。
- (4) 无论在执行的过程中是否发生异常, 都会执行上下文管理器的__exit__()方法。该方法负责执行程序的“清理”工作, 如释放资源等。

with语句

with语句执行过程：

(5) 如果执行过程中没有出现异常，或者语句体中执行了break、continue或者return语句，则以None 作为参数调用__exit__()方法；如果执行过程中出现异常，则会使用sys.exc_info 得到的异常信息为参数调用__exit__()方法。

(6) 出现异常时，如果__exit__()方法返回的结果为False，则会重新抛出异常，让with 之外的语句逻辑来处理异常，这是通用做法；如果返回True，则忽略异常，不再对异常进行处理。

上下文管理器

- 要想使用with语句进行工作，前提是要有上下文管理器。
- 上下文管理器是Python 2.5开始支持的一种语法，用于规定某个对象的使用范围，一旦进入或者离开使用范围，会有特殊的操作被调用。

上下文管理器

1. 上下文管理协议

- `__enter__(self)`: 进入上下文管理器时调用此方法，其返回值被放入with-as语句中as说明符指定的变量中。
- `__exit__(self, type, value, tb)`: 离开上下文管理器调用此方法。如果有异常出现，type、value、tb分别为异常的类型、值和追踪信息；如果没有异常，3个参数均设为None。此方法返回值为True或者False，分别指示被引发的异常得到了还是没有得到处理，如果返回False，引发的异常会被传递出上下文。

上下文管理器

2. 上下文管理器

- 支持上下文管理协议的对象，用于实现`__enter__()`和`__exit__()`方法。上下文管理器定义执行`with`语句时要建立的运行时上下文，负责执行`with`语句块上下文中的进入与退出操作。通常情况下，使用`with`语句调用上下文管理器，也可以通过直接调用其方法来使用。

上下文管理器

3. 运行时上下文

- 由上下文管理器创建，通过上下文管理器的 `__enter__()` 和 `__exit__()` 方法实现。其中， `__enter__()` 方法在语句体执行之前进入运行时上下文， `__exit__()` 在语句体执行完后从运行时上下文退出。

本章小结

- 本章围绕着Python的异常进行介绍，包括异常类、抛出和捕捉系统内置的异常、抛出和捕捉自定义异常，以及with和as环境安装器。通过对本章的学习，大家应该深入了解了异常产生的原理，并知道如何在程序中运行它们。

Python3

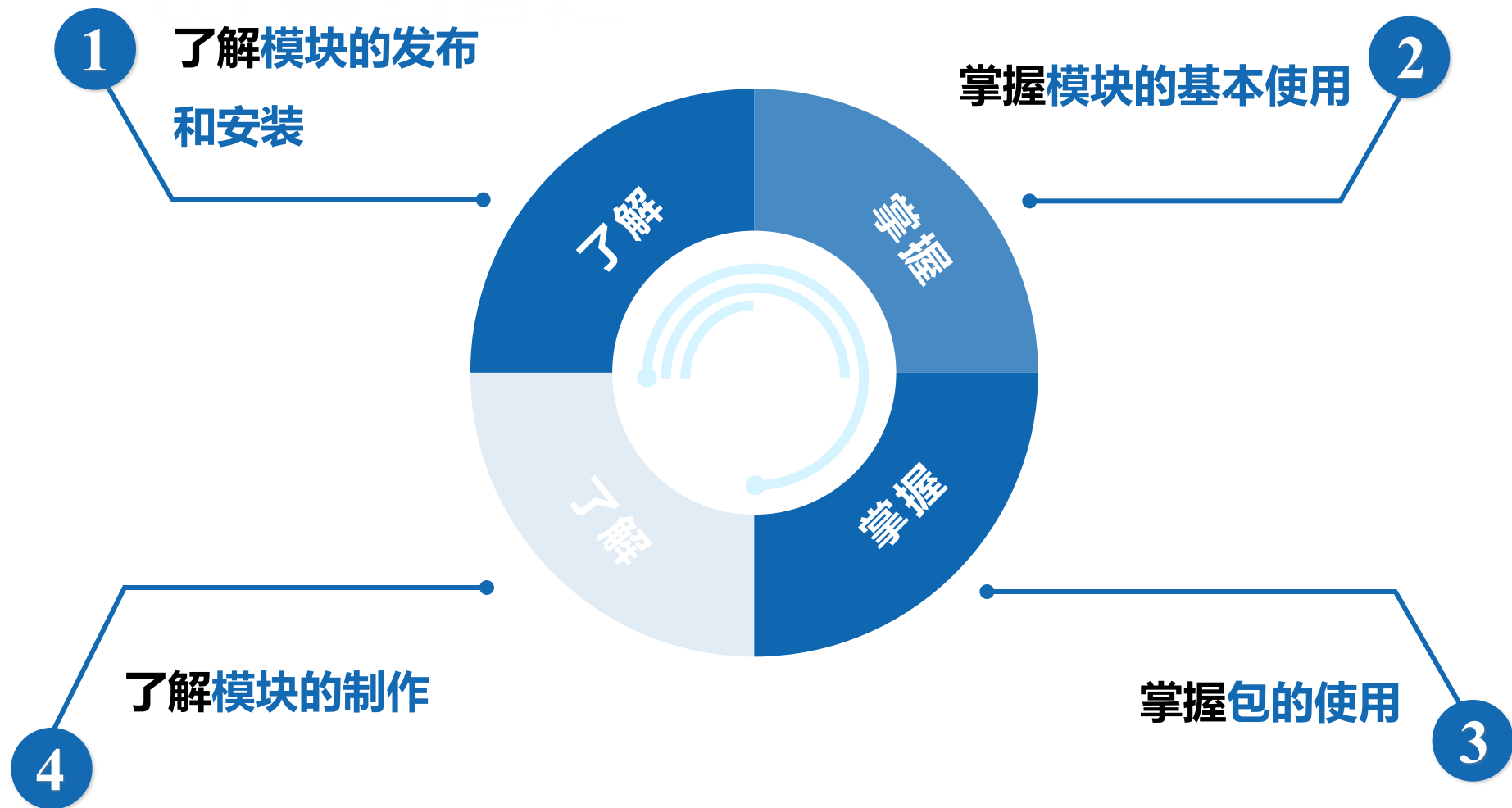
基础教程

第10章 模块

- 模块的使用
- 模块的制作
- 模块的发布
- 模块的安装
- 包的基本使用



学习目标



目录页



- 01** 模块的基本使用
- 02** 模块的制作
- 03** **Python**中的包
- 04** 模块的发布
- 05** 模块的安装

过渡页



01 模块的基本使用

02 模块的制作

03 **Python**中的包

04 模块的发布

05 模块的安装

模块的基本使用

在Python中用关键字import来引入某个模块，比如要引用模块math，就可以在文件最开始的地方用 **import** math来引入。基本格式如下：

```
import module1,module2...
```

模块的基本使用

如果要导入模块中的函数，其格式如下：

模块名.函数名

在调用模块中的函数时，之所以要加上模块名，是因为多个模块中，可能存在名称相同的函数，如果只是通过函数名来调用，解释器无法知道到底要调用哪个函数。

模块的基本使用

导入某个模块中的函数，格式如下：

```
from 模块名 import 函数名1, 函数名2...
```

导入某个模块中的所有函数，格式如下：

```
from 模块名 import *
```

模块的基本使用

Python解析器搜索模块位置的搜索顺序：

- (1) 搜索当前目录
- (2) 如果都找不到，会继续查看默认路径。
- (3) 搜索路径被存储在sys模块中的path变量

模块的基本使用

下列命令可以输出搜索路径：

```
import sys  
print(sys.path)
```

- `sys.path`输出是一个列表；
- 第一项输出的是当前目录，亦即我们执行python解释器的目录（对于脚本的话就是运行的脚本所在的目录）。



过渡页



01 模块的基本使用

02 模块的制作

03 Python中的包

04 模块的发布

05 模块的安装

模块的制作

test.py

```
def add(a,b):  
    return a+b  
# 用来进行测试  
ret = add(12,22)  
print('int test.py  
file,12+22=%d'%ret)
```

main.py

```
import test  
result = test.add(11,22)  
print(result)
```

运行main文件时，发现运行结果输出了test.py中的测试代码。

模块的制作

- python提供了一个__name__属性，每个模块都有一个__name__属性，当其值为 '__main__' 时，表明该模块自身在运行，否则是被引用。

```
if __name__ == '__main__':  
    ret=add(12,22)  
    print('int test.py file,__name__is %s'%__name__)
```


过渡页



01 模块的基本使用

02 模块的制作

03 **Python**中的包

04 模块的发布

05 模块的安装

Python中的包

- 为了组织好模块，通常会将多个模块放在一个包。
- 包是python模块文件所在的目录，且该目录下必须存在 `__init__.py` 文件（文件内容可以为空）。

```
package_a
├── __init__.py
├── module_a1.py
└── module_a2.py

package_b
├── __init__.py
├── module_b1.py
└── module_b2.py

main.py
```

Python中的包

如果main.py想要引用packagea中的模块modulea1, 可以使用下列语句实现:

```
from package_a import module_a1  
import package_a.module_a1
```

Python中的包

- 如果package_a中的modulea1需要引用package_b，那么默认情况下，Python是找不到package_b的。
- 可以在package_a中的__init__.py中添加
“`sys.path.append('../')`”

将该包下的所有module都添加 `import __init__` 即可。

过渡页



01 模块的基本使用

02 模块的制作

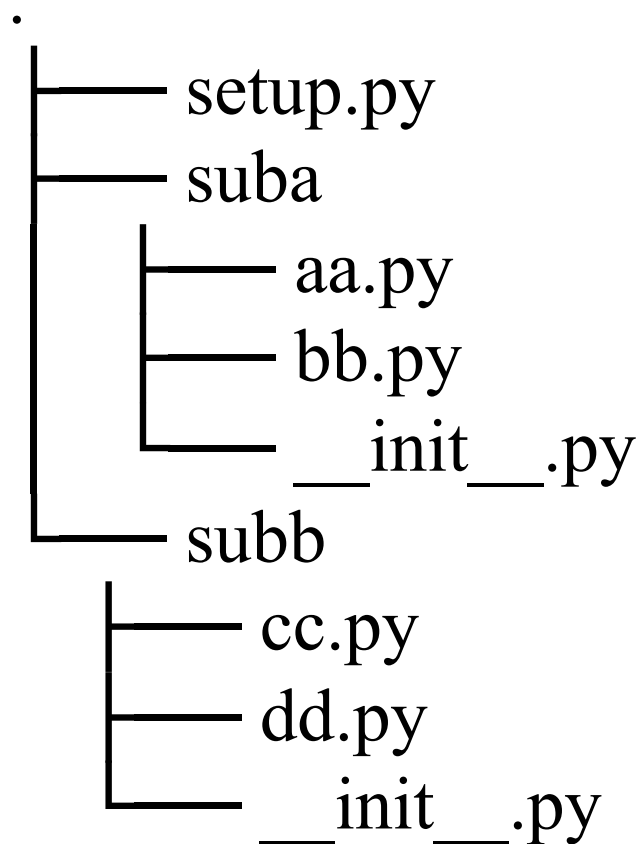
03 Python中的包

04 模块的发布

05 模块的安装

模块的发布

(1) 将模块放在setup.py包



模块的发布

(2) 编辑setup.py文件，指明需要包含哪些py文件。

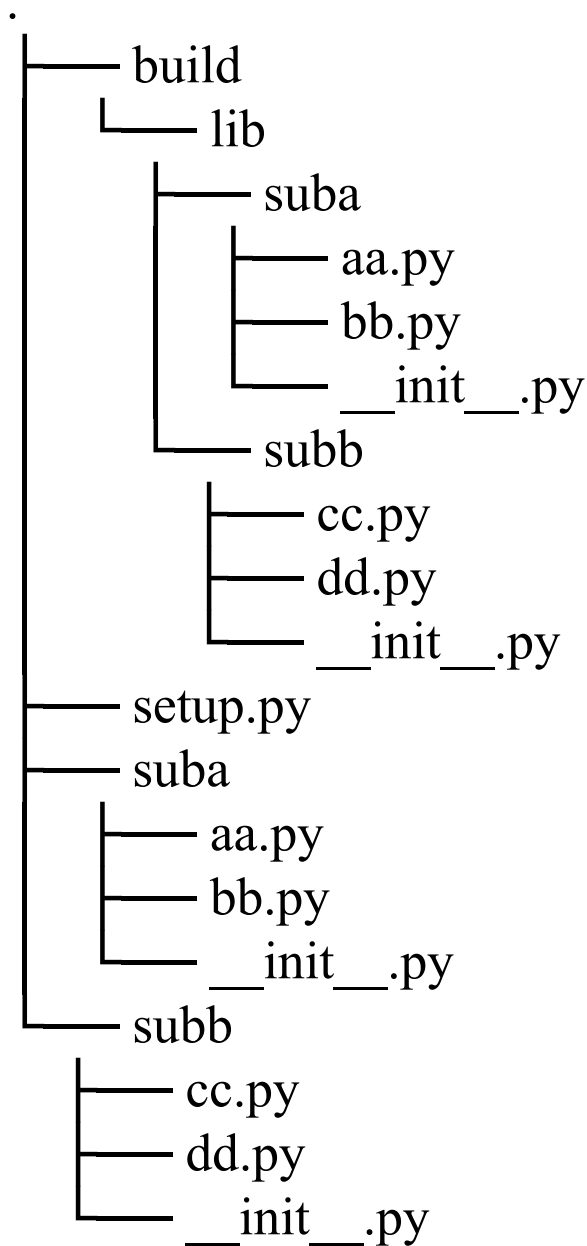
```
from distutils.core import setup
setup(
    name="itheima",
    version="1.0",
    description="itheima belongs to itcast",
    author="itcast",
    py_modules=['suba.aa', 'suba.bb', 'subb.cc', 'subb.dd'] )
```

模块的发布

(3) 构建模块

`python setup.py build`

构建后的目录结构→

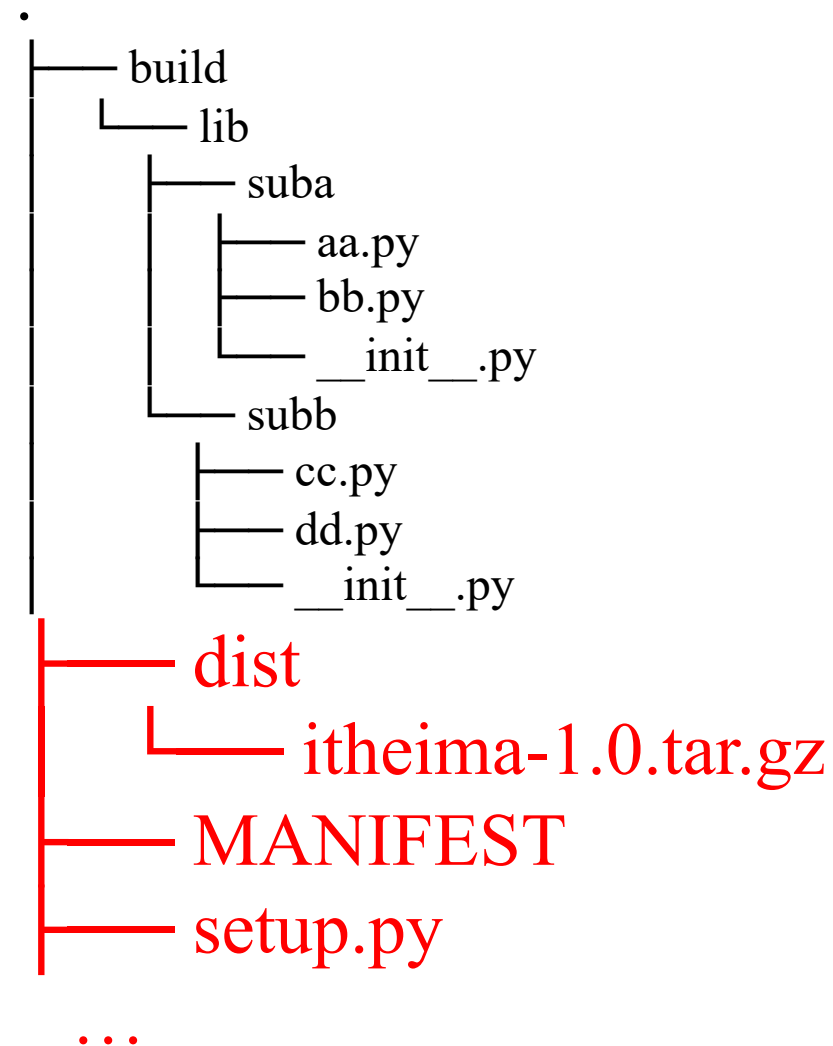


模块的发布

(4) 生成发布压缩包

```
python setup.py sdist
```

打包后的目录结构→



过渡页



01 模块的基本使用

02 模块的制作

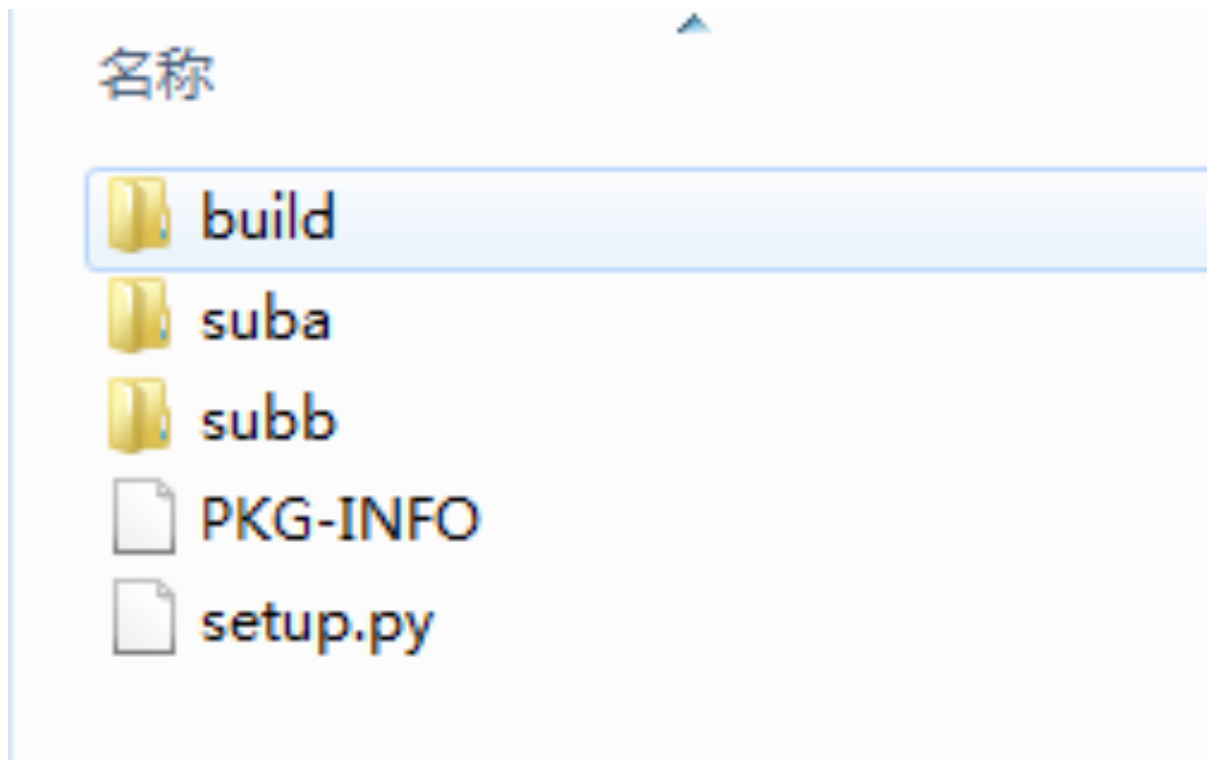
03 **Python**中的包

04 模块的发布

05 模块的安装

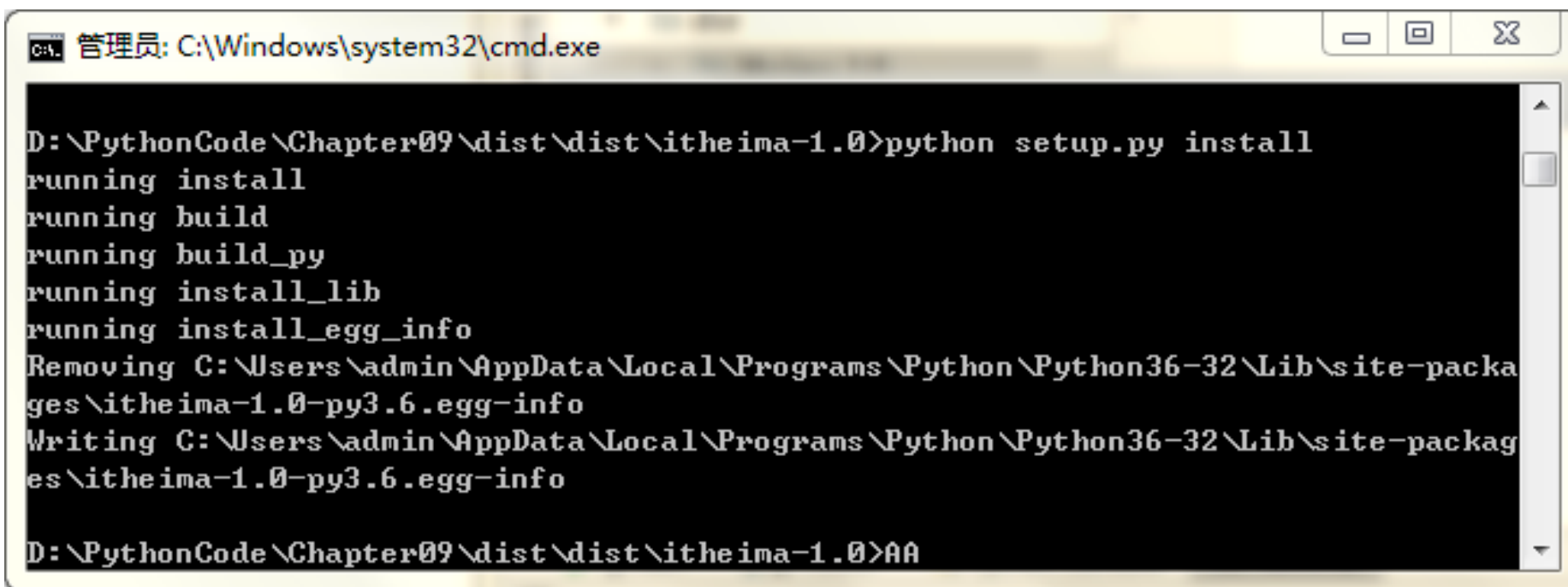
模块的安装

(1) 找到模块的压缩包，对压缩包进行解压



模块的安装

(2) 执行命令 “python setup.py install”



```
C:\> 管理员: C:\Windows\system32\cmd.exe

D:\PythonCode\Chapter09\dist\dist\itheima-1.0>python setup.py install
running install
running build
running build_py
running install_lib
running install_egg_info
Removing C:\Users\admin\AppData\Local\Programs\Python\Python36-32\Lib\site-packa
ges\itheima-1.0-py3.6.egg-info
Writing C:\Users\admin\AppData\Local\Programs\Python\Python36-32\Lib\site-packag
es\itheima-1.0-py3.6.egg-info

D:\PythonCode\Chapter09\dist\dist\itheima-1.0>AA
```

本章小结

- 本章主要针对Python中的模块进行了讲解，包括模块的制作、包、模块的发布、模块的安装和使用。希望通过本章的学习，大家能够理解使用模块的好处，并在以后的工作中灵活的借助模块完成想要的功能。

Python3

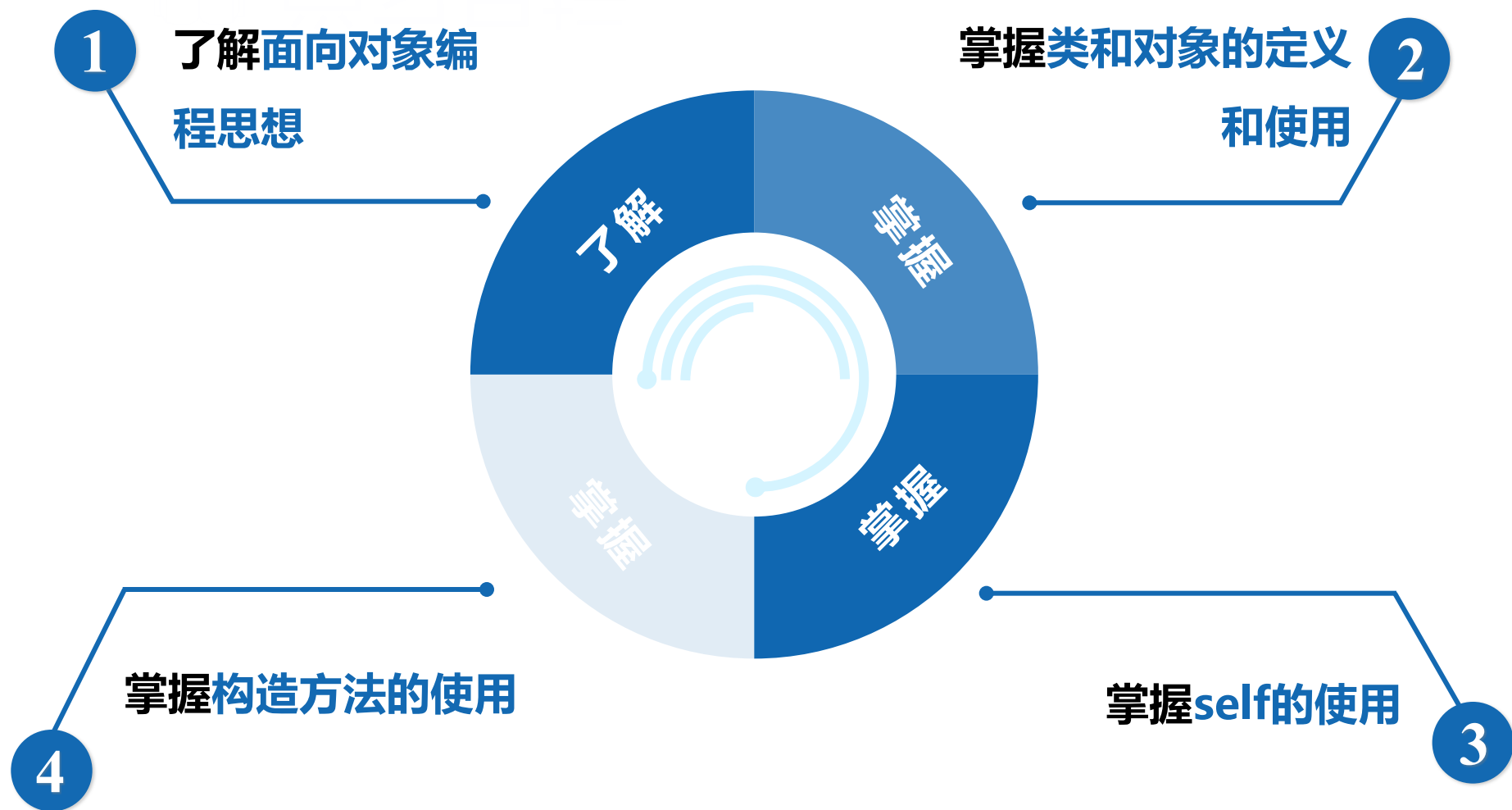
基础教程

第11章 面向对象编程（上）

- 面向对象编程
- 类和对象
- 构造方法和析构方法
- self的使用
- 运算符重载



学习目标



目录页



- 01** 面向对象编程概述
- 02** 类和对象
- 03** 构造方法和析构方法
- 04** **self**的使用
- 05** 运算符重载

过渡页



01 面向对象编程概述

02 类和对象

03 构造方法和析构方法

04 **self**的使用

05 运算符重载

面向对象

程序员 “面向对象”



在现实世界中存在各种不同形态的事物，这些事物之间存在着各种各样的联系。在程序中使用对象来映射现实中的事物，使用对象间的关系来描述事物之间的联系，**这种思想就是面向对象。**

面向对象

分别使用面向过程和面向对象来实现五子棋

编程思想	实现步骤	特点
面向过程	✓ 开始游戏	先分析解决问题的步骤 使用函数把这些步骤以此实现 使用的时候需要逐个调用函数
	✓ 黑子先走	
	✓ 绘制画面	
	✓ 轮到白子。	
	✓ 绘制画面	
	✓ 判断输赢	
	✓ 返回步骤2	
	✓ 输出最后结果	
面向对象	✓ 黑白双方：这两方的行为一样	把解决问题的事物分为多个对象 对象具备解决问题过程中的行为
	✓ 棋盘系统：负责绘制画面	
	✓ 规则系统：负责判断诸如犯规、输赢等。	

面向对象

若加入悔棋功能，面向过程和面向对象，分别怎么实现呢？



面向过程

从输入到判断到
显示的一系列步
骤都需要改动

面向对象

只需要改动棋盘
对象就可以

更简便！

过渡页



01 面向对象编程概述

02 类和对象

03 构造方法和析构方法

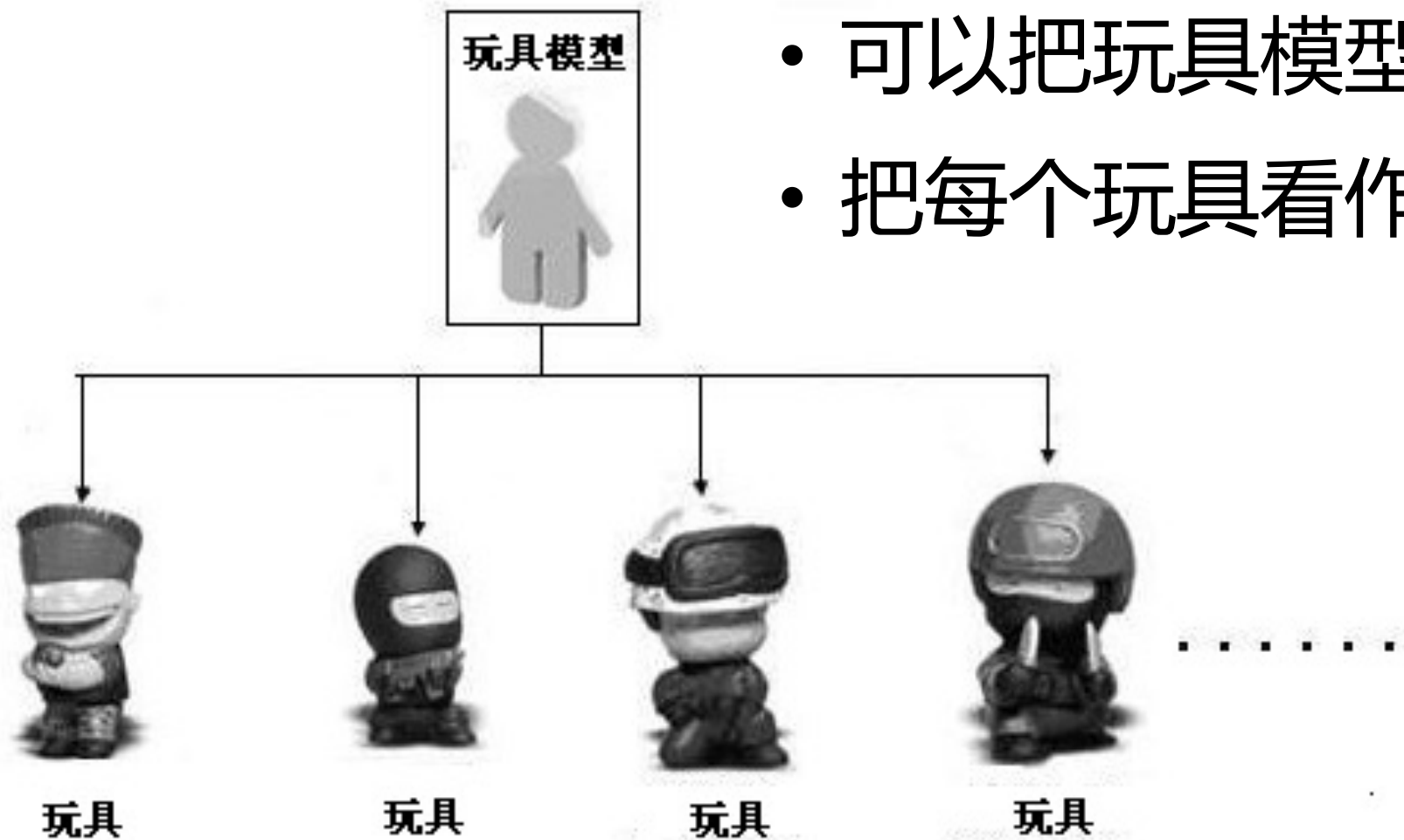
04 **self**的使用

05 运算符重载

类和对象的关系

- 面向对象编程有两个非常重要的概念：类和对象。
- 对象是面向对象编程的核心。
- 具有相似特征和行为的事物的集合统称为类
- 对象是根据类创建的，一个类可以对应多个对象。

类和对象的关系



- 可以把玩具模型看作一个类
- 把每个玩具看作一个对象

类的定义

类是由3部分组成的：

- 类的**名称**：类名。
- 类的**属性**：一组数据，比如性别。
- 类的**方法**：允许进行操作的方法，比如说话。

类的定义

使用**class**关键字来声明一个类，基本格式如下：

```
class 类名:  
    类的属性  
    类的方法
```



根据类创建对象

根据类**创建对象**的语法格式如下：

```
对象名 = 类名()
```

要想给对象**添加属性**，可以通过如下方式：

```
对象名.新的属性名 = 值
```

过渡页



01 面向对象编程概述

02 类和对象

03 构造方法和析构方法

04 `self`的使用

05 运算符重载

构造方法

- 构造方法指的是__init__方法。
- 当创建类的实例的时候，系统会自动调用构造方法，从而实现对类进行初始化的操作。

构造方法

- class **Car**:
- def **__init__**(*self*):
- *self.color* = '黑色'
- def **toot**(*self*):
- print("%s的车在鸣笛..."%(*self.color*))

```
bmw = Car()
bmw.toot()
```



析构方法

- 当删除一个对象来释放类所占用资源的时候，Python解释器默认会调用另外一个方法，这个方法就是`__del__()`方法。
- `__del__`方法被称为析构方法。

过渡页



01 面向对象编程概述

02 类和对象

03 构造方法和析构方法

04 **self**的使用

05 运算符重载

self的使用

- 在方法的列表中，第1个参数永远都是self。
- self的字面意思是自己，我们可以把它当做C++里面的this指针理解，表示的是对象自身。
- 当某个对象调用方法的时候，Python解释器会把这个对象作为第1个参数传给self，开发者只需要传递后面的参数就可以了。

self的使用

示例:

```
class Dog:
    def __init__(self, newColor):
        self.color = newColor
    def printColor(self):
        print("颜色为: %s"%self.color)
```

```
dog1 = Dog("白色")
dog1.printColor()
```

The diagram illustrates the execution of the provided code. A red line originates from the `printColor()` method call in the `dog1` object, travels down and then left, ending with an arrow pointing to the `self` parameter in the `printColor(self)` method definition. Another red line originates from the `Dog("白色")` constructor call, travels down and then left, ending with an arrow pointing to the `self` parameter in the `__init__(self, newColor)` method definition. These arrows demonstrate how the `self` parameter is passed to the corresponding method when it is called on an instance.

过渡页



01 面向对象编程概述

02 类和对象

03 构造方法和析构方法

04 **self**的使用

05 运算符重载

运算符重载

- **运算符重载**是通过实现特定的方法使类的实例对象支持Python的各种内置操作。例如：**+运算符是类里提供的__add__这个函数**，当调用+实现加法运算的时候，实际上是调用了__add__方法。

运算符重载

方法	说明	何时调用方法
<code>__add__</code>	加法运算	对象加法: <code>x+y</code> , <code>x+=y</code>
<code>__sub__</code>	减法运算	对象减法: <code>x-y</code> , <code>x-=y</code>
<code>__mul__</code>	乘法运算	对象乘法: <code>x*y</code> , <code>x*=y</code>
<code>__div__</code>	除法运算	对象除法: <code>x/y</code> , <code>x/=y</code>
<code>__getitem__</code>	索引, 分片	<code>x[i]</code> 、 <code>x[i:j]</code> 、没有 <code>__iter__</code> 的for循环等
<code>__setitem__</code>	索引赋值	<code>x[i]=值</code> 、 <code>x[i:j]=序列对象</code>
<code>__delitem__</code>	索引和分片删除	<code>del x[i]</code> 、 <code>del x[i:j]</code>

加法运算符重载

加法运算是通过调用 `__add__` 方法完成重载的，当两个实例对象执行加法运算时，自动调用 `__add__` 方法。

$z = x + y$

执行加法运算，实质是调用 `__add__` 方法

索引和分片重载

跟索引相关的重载方法包括如下3个：

- `__getitem__`：索引、分片；
- `__setitem__`：索引赋值；
- `__delitem__`：索引和分片删除。

索引和分片重载

1. `__getitem__`方法

在对实例对象执行索引、分片或者for迭代操作时，会自动调用`__getitem__`方法。

```
# 定义索引、分片运算符重载方法
def __getitem__(self, index):
    return self.data[index]
```

索引和分片重载

2. `__setitem__`方法

通过赋值语句给索引或者分片赋值时，调用 `__setitem__` 方法实现对序列对象的修改。

```
def __setitem__(self, index, value):  
    self.data[index] = value
```


索引和分片重载

3. `__delitem__`方法

当调用`del`方法时，实质上会调用`__delitem__`方法实现删除操作。

```
def __delitem__(self, index):  
    del self.data[index]
```

本章小结

- 本章首先介绍了面向对象编程的基本知识，包括面向对象概述、类和对象、根据类创建对象，然后介绍了构造方法和析构方法的使用，然后又介绍了self和__str__方法的简单使用，最后开发了一个反恐游戏的案例。通过对本章内容的学习，大家对面向对象应该有初步的了解，为后面面向对象深入的学习做好铺垫。

Python3

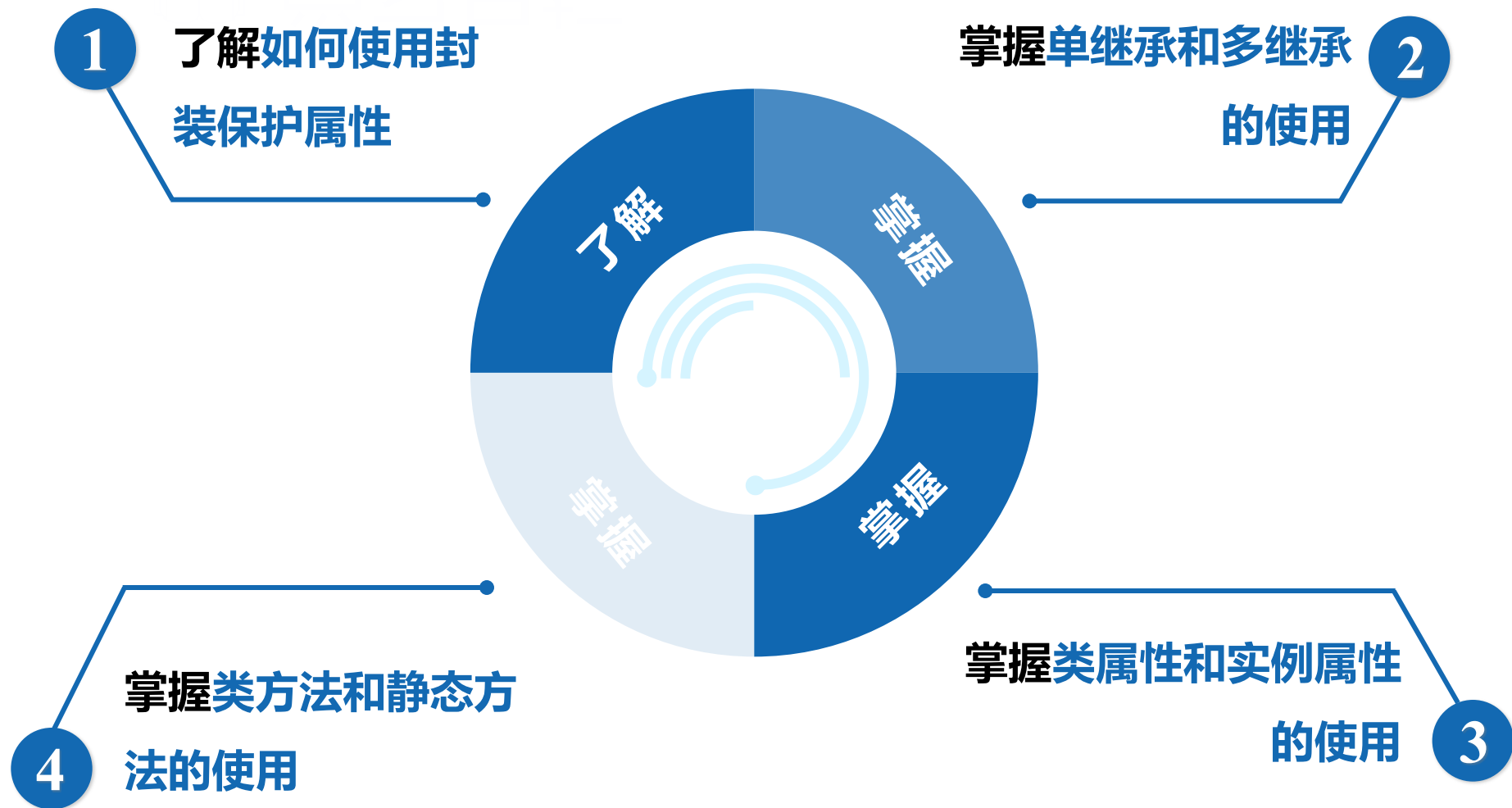
基础教程

第12章 面向对象编程（下）

- 封装
- 继承
- 多态
- 类属性和实例属性
- 类方法和静态方法



学习目标



目录页



01 封装

02 继承

03 多态

04 类属性和实例属性

05 类方法和静态方法

过渡页



01 封装

02 继承

03 多态

04 类属性和实例属性

05 类方法和静态方法

封装

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

laowang = Person("老王", 30)
laowang.age = 300
print(laowang.age)
```

人的年龄可以随便
设置，显然不可行。

封装

为了保护类里面的属性，可以采用如下方式解决：

1. 把属性定义为私有属性，即在属性名的前面加上两个下划线；
2. 添加用于设置或获取属性值的两个方法供外界调用。

过渡页



01 封装

02 继承

03 多态

04 类属性和实例属性

05 类方法和静态方法

单继承

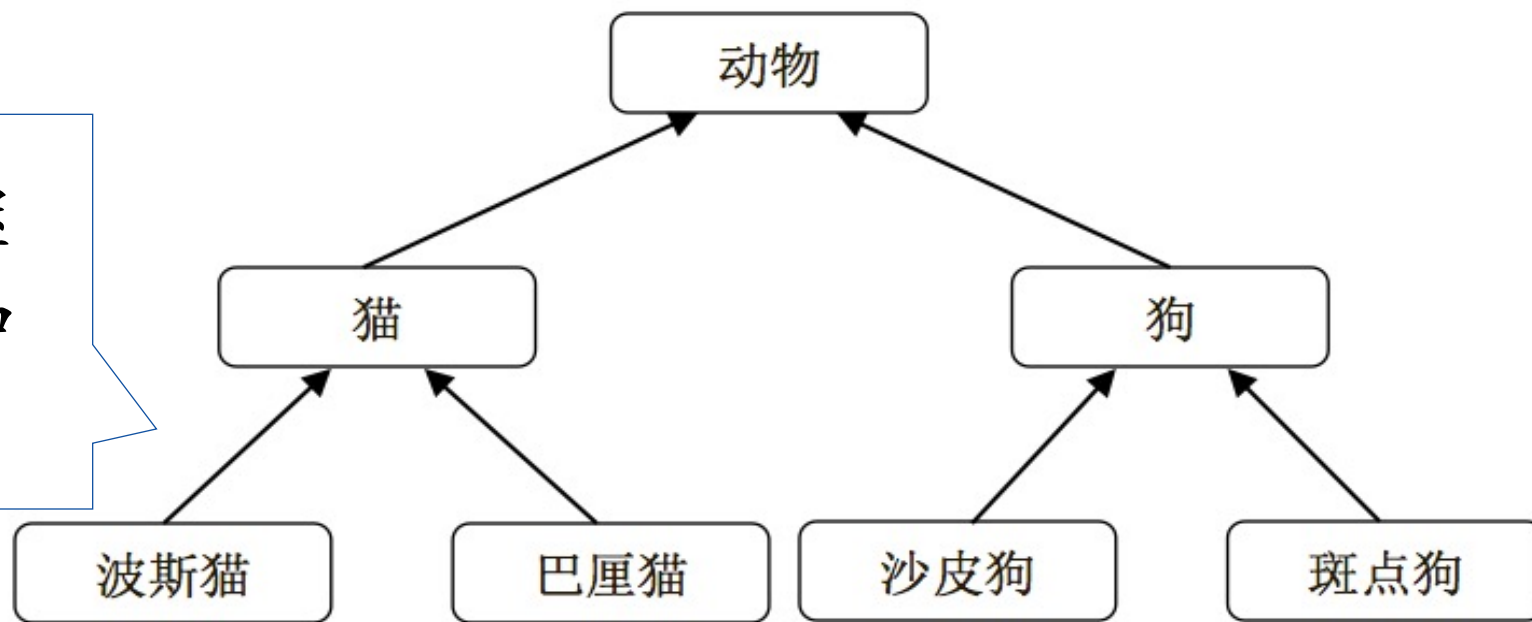


- 在现实生活中，继承一般指的是子女继承父辈的财产。
- 在程序中，继承描述的是事物之间的所属关系。

单继承

类的继承是指在一个现有类的基础上构建一个新的类，构建出来的新类被称作子类。

波斯猫和巴厘猫都继承自猫，而沙皮狗和斑点狗都继承狗。



单继承

Python程序中，继承使用如下语法格式标注：

```
class 子类名(父类名):
```

假设有一个类为A，A派生出来了子类B，示例如下：

```
class B(A):  
class A(object):
```

默认是继承自object的

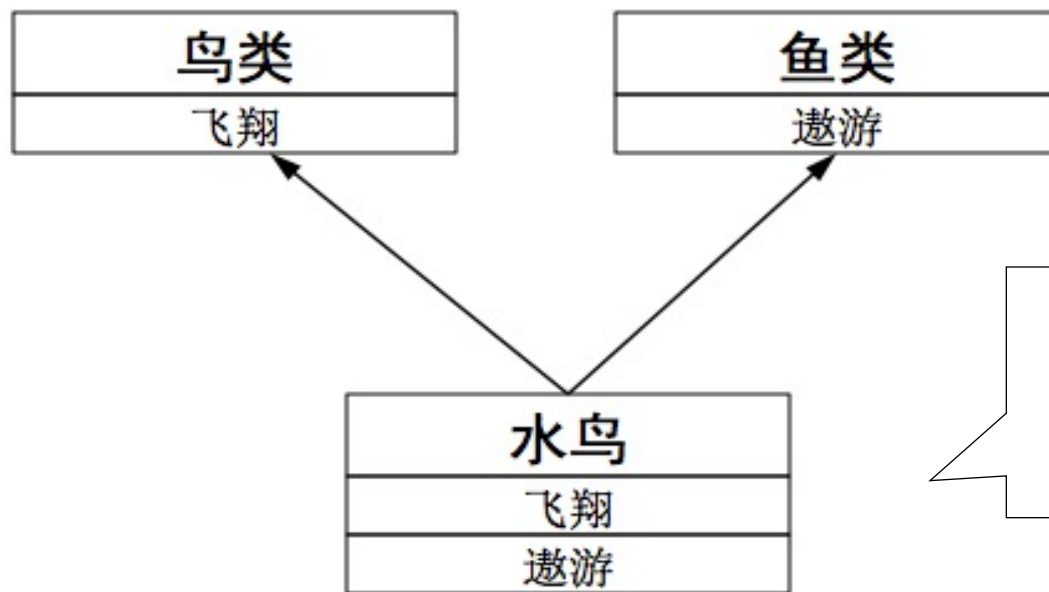
多继承

现实生活中，一个派生类往往会有多个基类。比如沙发床是沙发和床的功能的组合，这都是多重继承的体现。



多继承

Python支持多继承，多继承就是子类拥有多个父类，并且具有它们共同的特征，即子类继承了父类的方法和属性。



水鸟拥有了鱼和鸟的特征

多继承

多继承可以看做是单继承的扩展，语法格式如下：

```
class 子类名(父类1, 父类2...):
```

如果子类继承的多个父类间是平行的关系，子类先继承的哪个类就会调用哪个类的方法。

重写父类方法和调用父类方法

- 在继承关系中，子类会自动拥有父类定义的方法，但是有时子类想要按照自己的方式实现方法，即对父类中继承来的方法进行重写，使得子类中的方法覆盖掉跟父类同名的方法。
- 需要注意的是，在子类中重写的方法要和父类被重写的方法具有相同的方法名和参数列表。

过渡页



01 封装

02 继承

03 多态

04 类属性和实例属性

05 类方法和静态方法

多态

不同国家的人，打招呼的方式是不同的。



多态

A类

```
class A(object):  
    def test(self):  
        print("--A--test")
```

B类

```
class B(A):  
    def test(self):  
        print("--B--test")
```

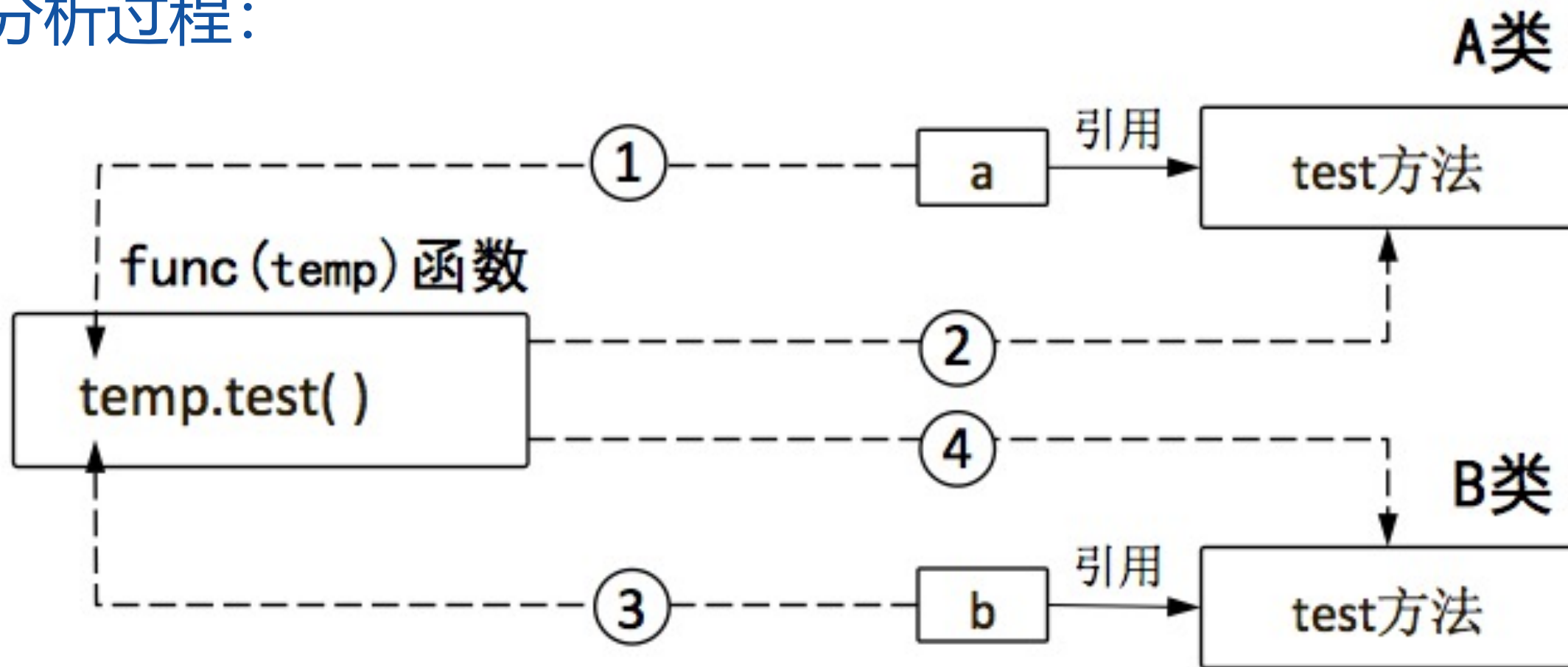
```
def func(temp):  
    temp.test()
```

```
a = A()  
b = B()  
func(a)  
func(b)
```

a、b的对象
两次调用func函数
结果不一样

多态

分析过程:



过渡页



01 封装

02 继承

03 多态

04 类属性和实例属性

05 类方法和静态方法

类属性

类属性是类所拥有的属性，它需要在类中进行显示地定义（位于类内部，方法的外面），它被所有类的实例对象所共有，在内存中只存在一个副本。

类属性示例代码：

```
class Cat(object):  
    #类属性  
    num = 0
```

实例属性

通过 “实例.属性” 添加属性的属性都是实例属性。

实例属性示例：

```
def __init__(self):  
    #实例属性  
    self.age = 1
```

过渡页



01 封装

02 继承

03 多态

04 类属性和实例属性

05 类方法和实例方法

类方法

使用修饰器@classmethod来标识类方法。

```
class 类名:  
    @classmethod  
    def 类方法名(cls):  
        方法体
```

要想调用类方法，既可以通过对象名调用类方法，又可以通过类名调用类方法，这两种方法没有任何区别。

静态方法

使用修饰器@staticmethod来标识静态方法。

```
class 类名:
```

```
    @staticmethod
```

```
    def 静态方法名():
```

```
        方法体
```



静态方法

- 静态方法是没有self参数，在静态方法中无法访问实例变量。
- 静态方法中不可以直接访问类属性，但是可以通过类名引用类属性。
- 静态方法跟定义它的类没有直接关系，只是起到了类似函数的作用。

本章小结

- 本章继续介绍了面向对象编程的知识，包括面向对象的三大特征（封装、继承和多态），然后介绍了类属性和实例属性，最后介绍了类方法和静态方法。通过对本章内容的学习，大家应该对面向对象有了深入地理解，为面向对象编程埋下了伏笔。